실전형 사이버 훈련장

버그헌팅 실습훈련 초급 과정

버그헌팅의 기초 및
공개형 버그바운티 참여

한국인터넷진흥원

SG SECURITY GYM
실전형사이버훈련장
Security-Gym

FIND THE GAP

1장. 버그헌팅 입문: 개념 이해와 올바른 마인드셋 1

2장. 버그헌팅의 역사적 진화와 프로그램 유형 완전 정복 6

3장. 취약점 공개 정책(VDP)의 이해와 국내외 버그바운티 플랫폼 11

4장. 윤리적 해킹의 기본과 법적 주의사항 15

5장. 버그헌팅의 이해: 취약점 평가(CVSS)와 성공적인 마인드셋 20

6장. 웹 애플리케이션의 작동 원리와 핵심 프로토콜 24

7장. 웹 클라이언트와 서버의 통신 원리 및 핵심 요소 28

8. HTTP 응답 코드의 비밀과 프록시 도구(Burp Suite) 활용 33

9. 웹 보안의 3대 핵심 요소: 인증, 인가, 입력 값 검증 37

10. OWASP Top 10 주요 취약점의 이해와 공격 원리 41

11. 시나리오 개발과 취약점 체이닝 45

12. 웹 취약점 버그헌팅 실제 사례 분석 및 실습 49

13. 웹 취약점 버그헌팅 실제 사례 분석 53

14. 버그헌팅 자동화 도구 활용 실습: SQLMap, JWT Crack, Hashcat 57

15. CVE의 이해와 메타스플로잇(Metasploit) 활용 실습 61

16. CVE 버그헌팅 사례 65

17. CVE 버그헌팅 사례 2 69

18. CVE 취약점 버그헌팅 사례 3 73

19. 버그헌팅 정보수집 전략: OSINT, Reconnaissance 77

20. ChatGPT를 활용한 버그헌팅 81

21. ChatGPT 활용 버그헌팅 및 리포트 작성 전략 85

마무리 키워드 퀴즈 정답표 89
[버그헌팅 초급 과정 필수 용어집(Glossary] 104

1장. 버그헌팅 입문: 개념 이해와 올바른 마인드셋

오늘부터 우리는 단순히 '해킹 기술'을 배우는 것이 아니라, 기업과 사용자의 자산을 지키며 그 가치를 인정받는 '버그헌팅'의 세계로 들어갈 것입니다.

---

1. 목적

- **지식 (Knowledge):** 버그헌팅, 버그바운티, VDP(취약점 공개 프로그램)의 정확한 정의와 차이를 구분하고, 기업과 해커가 이 생태계에 참여하는 핵심 동기를 이해한다.
- **기술 (Skills):** 버그헌팅의 전체 프로세스(대상 선정 → 범위 확인 → 탐색 및 발굴 → 제보 → 보상)를 파악하고, 포상금 산정의 기준이 되는 취약점 등급(Severity) 체계를 분석할 수 있다.
- **태도 (Attitude):** 합법적인 범위 내에서 활동하는 윤리적 해커로서의 책임감을 갖고, 실패를 두려워하지 않는 끈기와 끊임없이 의심하는 연구 자세를 함양한다.

2. 사전지식

이 강의를 완벽히 소화하기 위해 다음과 같은 기초 역량이 권장됩니다.

- **프로그래밍 기초:** 코드를 읽고 로직을 파악할 수 있는 수준의 코딩 실력
- **컴퓨터 공학 지식:** 컴퓨터 구조 및 운영체제(OS)에 대한 전반적인 이해
- **네트워크/웹 기초:** 웹 애플리케이션의 동작 원리와 네트워크 프로토콜에 대한 이해 (컴퓨터공학/정보보호학과 3~4학년 수준 권장)

3. 들어가기 전에

여러분, 혹시 '현상금 사냥꾼'이라는 말을 들어보셨습니까? 서부 영화에서나 볼 법한 이 단어가 IT 보안 분야에서는 '버그 헌터(Bug Hunter)'라는 이름으로 존재합니다.

소프트웨어는 사람이 만듭니다. 그래서 필연적으로 실수가 발생하고, 이 실수는 해커들이 파고들 '틈(Vulnerability)'이 됩니다. 과거에는 이 틈을 악용하는 블랙 해커들이 주류였다면, 이제는 여러분처럼 먼저 틈을 찾아 기업에 알려주고, 그 대가로 금전적 보상과 명예를 얻는 '화이트 해커'의 시대가 열렸습니다.

이번 강의에서는 버그헌팅이 정확히 무엇인지, 왜 구글이나 네이버 같은 대기업들이 앞다퉈 해커들에게 돈을 주는지, 그리고 여러분이 이 길을 걷기 위해 어떤 준비가 필요한지 아주 상세히 짚어보겠습니다.

4. 수업내용

4.1. 강의 소개 및 수강 가이드

- **강의 수준:** 본 과정은 버그헌팅에 관심은 있으나 아직 시도해보지 못했거나, 막 시작하려는 분들을 위한 **초급 과정**입니다.

- **교육 목표:** 웹, 네트워크, 시스템의 기본 이해를 바탕으로 대표적인 웹 취약점(OWASP Top 10 등)을 학습하고, CVE(공개된 취약점) 활용법, 정보 수집 전략, 도구 활용법, 그리고 리포트 작성까지의 전 과정을 다룹니다.
- **중요성:** 버그바운티 프로그램의 **90% 이상이 웹(Web)을 대상**으로 하므로, 웹 취약점 이해가 필수적입니다.

4.2. 핵심 용어 정리: 버그헌팅 vs 버그바운티

많은 분들이 혼용하는 용어를 명확히 정의합니다.

- **버그 (Bug):** 소프트웨어의 예상치 못한 결함, 오류, 오작동입니다.
- **버그헌팅 (Bug Hunting):** 'Hunting'은 행위입니다. 취약점을 찾아내어 보상이나 명예를 얻기 위해 수행하는 일련의 **활동** 자체를 말합니다.
- **버그바운티 (Bug Bounty):** 'Bounty'는 보상입니다. 기업이 자사 서비스의 취약점을 찾은 해커에게 **포상금을 지급하는 제도**입니다.
- **VDP (Vulnerability Disclosure Program):** 금전적 보상(Bounty)은 지급하지 않지만, 취약점을 제보받아 윤리적으로 해결하는 채널이나 프로그램을 뜻합니다.

4.3. 버그헌팅 참여 동기 (해커의 관점)

해외 최대 플랫폼 '해커원(HackerOne)'의 리포트에 따르면 해커들의 동기는 다음과 같습니다.

1. **배움의 기회:** 성공 하든 실패 하든 분석 과정 자체가 엄청난 공부가 됩니다. 가장 많은 해커가 꼽은 이유입니다.
2. **금전적 보상:** 돈은 강력한 동기부여입니다.
3. **커리어 및 재미:** 보안 업무 특성상 비공개 경력이 많은데, 버그헌팅 이력(명예의 전당 등)은 공개적으로 실력을 증명하는 훌륭한 간판이 됩니다.
4. **보안 기여:** 내가 찾은 취약점이 수많은 사용자를 보호한다는 뿌듯함을 느낍니다.
    - ***사례:*** 깃허브(GitHub) 프라이빗 페이지 취약점 제보로 약 4,600만 원($35,000)을 받은 사례가 있습니다. 이는 금전적 이득과 기술적 성취를 동시에 보여줍니다.

4.4. 기업이 버그바운티를 운영하는 이유

기업이 외부인에게 시스템을 개방하는 것은 위험해 보이지만, 실제로는 이득이 더 큽니다.

- **효율성:** 내부 팀이나 컨설팅 업체보다 수백 배 많은 전 세계 해커(집단지성)가 참여하므로, 짧은 기간에 훨씬 많은 취약점을 발견합니다.
- **비용 절감:** 유효한 취약점에 대해서만 비용을 지불하므로 고용 계약보다 경제적입니다.
- **보안 강화 및 마케팅:** 보안이 완벽한 소프트웨어는 없습니다. 외부 전문가를 통해 잠재 위협을 선제적으로 제거하고, "우리는 보안을 중요시한다"는 메시지를 고객에게 전달합니다.

4.5. 포상금(Bounty) 결정 구조 및 등급

포상금은 '프로그램의 예산 규모'와 '취약점의 심각도(Severity)'에 따라 결정됩니다.

- **심각도 5단계:** 일반적으로 **Critical, High, Medium, Low, Informational**로 나뉩니다.
    - **Critical:** 시스템 장악, 계정 탈취 등 가장 위험한 수준.
    - **High/Medium:** SQL 인젝션, 데이터 접근 등.
    - **Low/Info:** 단순 정보 노출 등 미미한 영향.
- **국내외 사례:**
    - **네이버:** 계정 탈취(Critical) 최대 $20,000 (약 2,600만 원), 단순 정보 유출(Low) 최대 $300.
    - **파인더갭/버그크라우드:** 플랫폼별, 기업별로 책정 기준이 다름.
- **전략:** 버그헌터는 자신이 찾은 취약점이 서비스에 미칠 영향력(Impact)을 증명하는 시나리오를 잘 작성해야 높은 등급과 보상을 받을 수 있습니다.

4.6. 흔한 오해와 진실 (Q&A)

- **Q: 버그헌팅은 불법인가요?**
    - **A:** 아닙니다. 기업이 허용한 **범위(Scope)** 내에서 규칙을 준수하며 진행하는 것은 합법이며 권장됩니다. 단, 범위를 벗어나면 법적 처벌을 받을 수 있습니다.
- **Q: 코딩이나 시스템 구조를 완벽히 알아야 시작할 수 있나요?**
    - **A:** 필수는 아닙니다. 기초 지식으로 시작해 부딪히며 배우는 것이 더 빠릅니다. 강사들도 처음엔 취약점을 못 찾았지만, 시도하는 과정 자체가 실력이 되었습니다.
- **Q: 기업 입장에서 해커에게 시스템을 여는 게 더 위험하지 않나요?**
    - **A:** 아닙니다. 어차피 시스템은 공격에 노출되어 있습니다. 화이트 해커에게 윤리적인 제보 채널을 열어주는 것이 음지에서의 공격을 예방하는 길입니다.

4.7. 버그헌팅 프로세스 및 필수 마인드셋

- **프로세스:** 프로그램 선택 > **범위(Scope) 확인(필수)** > 취약점 탐색 및 발굴 >제보(리포트) > 검증 및 보상
- **버그헌터의 4가지 필수 요소:**
    1. **탐구심:** 끊임없이 변화하는 보안 트렌드와 신기술을 학습하려는 의지
    2. **용기:** 시간과 노력을 투자해도 보상이 없을 수 있다는(No Guarantee) 두려움을 이기는 용기
    3. **의심:** "왜 이렇게 작동하지?", "다르게 입력하면 어떨까?"라는 끊임없는 의심
    4. **끈기:** 실패는 성공의 과정입니다. 포기하지 않고 한 단계씩 파고드는 각오

5. 핵심내용 (Summary)

1. **버그헌팅 vs 버그바운티:** 버그헌팅은 취약점을 찾는 '행위', 버그바운티는 그에 대한 '보상 제도'입니다.
2. **상호 이익:** 해커에게는 금전적 보상과 성장의 기회를, 기업에게는 효율적인 보안 강화 효과를 제공하는 윈-윈 구조입니다.
3. **범위(Scope) 준수:** 법적 보호를 받기 위해 기업이 지정한 대상과 규칙을 반드시 지켜야 합니다.
4. **Impact 증명:** 단순한 버그 발견을 넘어, 그 버그가 비즈니스에 미치는 심각한 영향을 시나리오로 증명해야 높은 보상을 받습니다.

6. 마무리 키워드 퀴즈

강의 내용을 제대로 이해했는지 확인해 봅시다.

- **Q1.** 기업이 자사 서비스의 보안 취약점을 찾은 해커에게 포상금을 지급하는 제도를 뜻하는 용어는 무엇인가요?
    -
- **Q2.** 취약점의 위험도를 평가할 때, 시스템을 장악하거나 계정 탈취할 수 있는 가장 높은 수준의 위험 등급은 무엇인가요?
    -
- **Q3.** 버그헌팅을 시작하기 전, 법적인 보호를 받고 불법 행위가 되지 않기 위해 가장 먼저 확인해야 할 '허용된 대상 및 규칙'을 무엇이라고 합니까?
    -

7. 다음 장 소개

오늘 우리는 버그헌팅의 기본 개념을 잡았습니다. 다음 시간에는 "버그헌팅 프로그램의 발전 역사"를 다루겠습니다. 1995년 넷스케이프에서 시작된 최초의 버그바운티가 구글, 페이스북을 거쳐 어떻게 지금의 플랫폼 형태로 진화했는지, 그리고 국내 시장은 어떻게 변화해왔는지 그 흐름을 파악해보겠습니다.

2장. 버그헌팅의 역사적 진화와 프로그램 유형 완전 정복

지난 시간에 버그헌팅의 기본 개념을 잡았다면, 이번에는 이 거대한 생태계가 **어떻게 탄생했고 발전해왔는지**, 그리고 여러분이 활동하게 될 **프로그램의 종류**에는 무엇이 있는지 아주 구체적으로 파고들어 보겠습니다. 역사를 알면 미래가 보이고, 판을 알면 전략이 생기는 법입니다.

---

1. 목적

- **지식 (Knowledge):** 1995년 넷스케이프부터 시작된 글로벌 버그바운티의 역사와 국내 시장의 흐름을 파악하고, 수동 분석에서 AI(ChatGPT) 활용까지 변화하는 해킹 기법의 트렌드를 이해한다.
- **기술 (Skills):** 퍼블릭(Public), 프라이빗(Private), 비공개 전용(Invitation Only) 프로그램의 차이를 명확히 구분하여 자신의 실력과 상황에 맞는 프로그램을 선정하는 안목을 기른다.
- **태도 (Attitude):** 단순한 기술자가 아닌, 집단지성(Crowdsourcing)의 일원으로서 기업 보안에 기여한다는 자부심과 변화하는 기술(AI 등)을 적극적으로 수용하려는 유연한 태도를 갖춘다.

2. 사전지식

- **버그헌팅 기초:** 버그헌팅과 버그바운티의 기본적인 정의 (1장 내용).
- **기본 IT 상식:** 웹 브라우저, 소프트웨어 업데이트, 클라우드 소싱에 대한 기초 개념.

3. 들어가기 전에

"과거의 버그헌팅은 단순히 소프트웨어의 결함을 찾는 일이었습니다. 하지만 지금은 그 의미가 완전히 달라졌습니다."

여러분, 1995년 넷스케이프(Netscape)라는 회사가 "우리 브라우저의 버그를 찾으면 돈을 주겠다"고 했을 때, 세상은 반신반의했습니다. 하지만 지금 구글, 페이스북, 미 국방부까지 이 방식을 채택하고 있습니다. 왜일까요? 기업들은 '내부 직원'만으로는 막을 수 없다는 것을 깨달았기 때문입니다.

이번 강의에서는 이 '보안의 패러다임'이 어떻게 바뀌었는지 역사의 흐름을 짚어보고, 특히 최근 화두가 되고 있는 **AI(ChatGPT)를 활용한 버그헌팅**의 가능성까지 엿보겠습니다. 또한, 여러분이 실제로 참여하게 될 프로그램이 '공개형'인지 '초대형'인지에 따라 어떤 전략을 취해야 하는지도 상세히 알려드리겠습니다.

4. 수업내용

4.1. 해외 버그바운티의 탄생과 황금기

버그바운티의 역사는 보안의 역사와 궤를 같이합니다.

- **태동기 (1995년):** 넷스케이프의 엔지니어 '자렛 리들'이 넷스케이프 내비게이터 2.0 베타 버전의 버그를 찾기 위해 최초의 버그바운티를 런칭했습니다. 당시엔 포상금이 크지 않아 큰 이슈는 아니었지만, 역사적인 첫걸음이었습니다.
- **성장기 (2002~2004년):** '아이디펜스(iDefense)'와 '모질라(Mozilla)'가 프로그램을 신설하며 명맥을 이었습니다.
- **확산기 (2010~2011년):** 구글(Chrome)과 **페이스북**이 참여하면서 판도가 바뀝니다. 이들은 특정 제품뿐만 아니라 전반적인 서비스 인프라로 범위를 넓히고 보상 규모를 키워 해커들에게 강력한 동기를 부여했습니다.
- **황금기 (2012년~현재):** 해커원(HackerOne), 버그크라우드(Bugcrowd) 같은 **전문 중개 플랫폼**이 등장했습니다. 이제 우버, 스타벅스, 미 국방부까지 참여하며 웹, IoT, 블록체인, 자동차 등 모든 분야로 확장되었습니다.

4.2. 국내 버그바운티의 발전 과정

해외에 비해 다소 늦었지만, 국내도 빠르게 변화하고 있습니다.

- **초기 (2006년):** KISA(한국인터넷진흥원)가 최초로 시작했습니다.
- **포상 제도 도입 (2012년):** KISA와 공동 운영사들이 협력하여 신고 포상제를 정식으로 운영하기 시작했습니다. 이것이 국내 버그바운티의 실질적인 시작입니다.
- **확장 및 플랫폼 등장 (2018년~):** 네이버, 카카오 등 대기업과 블록체인 기업들이 참여했고, 2021년부터는 **파인더갭, 패치데이, 제로웨일** 등 민간 플랫폼이 오픈하며 생태계가 활성화되었습니다.
- **한계와 과제:** 여전히 해외 대비 기업의 참여율과 포상금 규모는 낮은 편입니다. 기업들이 운영 부담과 서비스 장애를 우려하기 때문인데, 이를 개선하기 위한 노력이 지속되고 있습니다.

4.3. 버그헌팅 기법의 진화: 수동에서 AI까지

해커들의 무기도 시대에 따라 발전했습니다.

1) **수동적 접근 (Manual):** 초기 단계. 많은 시간과 노력이 들지만, 창의적인 취약점을 발견하는 데 필수적입니다.
2) **자동화 기술 (Automation):** 퍼징(Fuzzing) 기술과 정적/동적 분석 도구의 등장으로 효율성이 비약적으로 상승했습니다.
3) **집단지성과 커뮤니티:** 혼자 끙끙 앓는 것이 아니라, 커뮤니티를 통해 지식과 노하우를 공유하며 '함께' 해결하는 문화가 정착되었습니다.
4) **AI 기반 도구 (AI-Powered):** 최근 **ChatGPT, Copilot** 등 생성형 AI를 활용해 코드를 분석하고 공격 시나리오를 수립하는 고도화된 헌팅이 시도되고 있습니다.

4.4. 버그헌팅 프로그램의 종류 (VDP vs BBP)

프로그램은 보상 유무와 공개 범위에 따라 나뉩니다.

- **VDP (Vulnerability Disclosure Program): 포상금이 없습니다.** 대신 기업은 해커에게 법적 보호(Safe Harbor)를 약속하고, 해커는 윤리적으로 취약점을 제보합니다.
- **버그바운티 (BBP): 포상금이 있습니다.** 참여 대상에 따라 세 가지로 세분화됩니다.

4.5. 공개 범위에 따른 3가지 분류

기업은 보안성과 효과성을 고려해 다음 중 하나를 선택합니다.

1. **퍼블릭 (Public):**
    - **특징:** 누구나 참여 가능. 진입 장벽이 낮습니다.
    - **장점:** 가장 많은 해커가 참여하므로 다양한 취약점을 빠르게 찾을 수 있습니다.
    - **단점:** 신뢰할 수 없는 해커의 참여로 저품질 리포트가 많아질 수 있습니다.
2. **프라이빗 (Private):**
    - **특징:** 초대를 받은 검증된 해커만 참여 가능합니다.
    - **장점:** 보고서 품질이 높고, 특정 기능이나 제품을 집중 점검하기 좋습니다.
    - **공개 여부:** 프로그램 진행 사실 자체는 외부에 알리는 경우가 많습니다.
3. **인비테이션 온리 (Invitation Only):**
    - **특징:** 프라이빗과 유사하지만, **버그바운티 진행 사실조차 외부에 공개하지 않습니다.**

    - **대상:** 매우 민감한 기밀 프로젝트나 고도의 보안이 필요한 제품에 적용됩니다.

6. 마무리 키워드 퀴즈

강의의 핵심을 잘 파악했는지 확인해 봅시다.

- **Q1.** 1995년, 웹 브라우저의 안정성을 위해 역사상 최초로 버그바운티 프로그램을 시작한 기업의 이름은 무엇인가요?
    -
- **Q2.** 버그바운티 프로그램 중, 누구나 참여할 수 있어 잠재적으로 가장 많은 취약점을 발견할 수 있지만, 저품질 보고서가 접수될 확률도 높은 유형은 무엇인가요?
    -
- **Q3.** 최근 버그헌팅 트렌드 중 하나로, ChatGPT나 Copilot 등을 활용하여 취약점 탐색과 분석을 고도화하는 기술적 흐름은 무엇의 발전인가요?
    -

7. 다음 장 소개

우리는 이제 버그헌팅의 역사와 프로그램의 종류를 완벽히 이해했습니다. 다음 시간에는 "취약점 공개 정책(VDP)과 플랫폼"에 대해 더 깊이 알아보겠습니다. 해커가 법적으로 보호받기 위해 반드시 확인해야 할 '안전장치'인 VDP의 핵심 구성 요소와, 여러분이 활동하게 될 국내외 주요 플랫폼들의 특징을 비교 분석해 드리겠습니다.

3장. 취약점 공개 정책(VDP)의 이해와 국내외 버그바운티 플랫폼

앞선 강의에서 버그헌팅의 역사와 종류를 배웠다면, 이번에는 **"그래서 어디서, 어떻게, 안전하게 활동할 것인가?"**에 대한 실전적인 답을 찾을 차례입니다. 기업과 해커 사이의 약속인 **'취약점 공개 정책(VDP)'**과 여러분이 활동하게 될 '무대'인 다양한 플랫폼들을 분석해보겠습니다.

---

1. 목적

- **지식 (Knowledge):** 취약점 공개 정책(VDP)의 필요성과 핵심 구성 요소(범위, 법적 보호 등)를 이해하고, 국내외 주요 버그바운티 플랫폼별 특징과 성격을 비교 분석한다.
- **기술 (Skills):** VDP 템플릿을 해석하여 제보 가능한 대상과 규칙을 파악하고, 자신의 성향(오픈소스, 고수익, 커뮤니티 등)에 맞는 플랫폼을 선정할 수 있다.
- **태도 (Attitude):** 'Safe Harbor(법적 보호)' 조항의 중요성을 인지하고, 반드시 정책을 준수하며 합법적인 테두리 안에서 활동하려는 준법 정신을 갖춘다.

2. 사전지식

- **버그헌팅 기초:** 버그헌팅과 버그바운티의 차이점 (2장 내용)
- **기초 법규 이해:** 정보통신망법 등 해킹과 관련된 기본적인 법적 상식

3. 들어가기 전에

여러분이 길을 가다 이웃집 뒷문이 활짝 열려 있는 것을 봤다고 가정해 봅시다. 도둑이 들 것 같아 알려주고 싶은데, 집주인 연락처도 모르고 문을 두드렸다간 오히려 주거침입으로 오해받을까 두려워 그냥 지나치게 되죠.

사이버 세계도 마찬가지입니다. 보안 취약점을 발견해도 **"누구에게, 어떻게 신고해야 하는지"** 명확하지 않으면, 해커들은 법적 처벌이 두려워 침묵하거나 인터넷에 무단으로 공개해버려 더 큰 피해를 낳기도 합니다.

이를 해결하기 위해 등장한 것이 바로 VDP(취약점 공개 정책)입니다. "이 규칙만 지키면 신고해도 안전해"라고 약속하는 것이죠. 이번 시간에는 해커를 보호하는 안전장치인 VDP와, 여러분이 실력을 뽐낼 수 있는 전 세계의 다양한 '놀이터(플랫폼)'들을 샅샅이 파헤쳐 보겠습니다.

4. 수업내용

4.1. 취약점 공개 정책 (VDP: Vulnerability Disclosure Policy)

- **정의:** "뭔가 보이면 말해줘(If you see something, say something)"라는 디지털 이웃 감시와 같습니다. 윤리적 해커에게 보고를 위한 명확한 지침(연락처, 범위 등)을 제공하는 정책입니다.

- **필요성:** VDP가 없으면 해커는 신고 채널을 찾지 못해 포기하거나, SNS에 무단 공개하여 제로데이 위험을 초래할 수 있습니다. 기업 입장에서는 선의의 제보를 받을 기회를 놓치게 됩니다.
- **발전:** 2016년 미 국방부(DoD)가 최초로 VDP를 개발했고, 2020년 CISA(미 국토안보부)는 모든 연방 기관에 VDP 설립을 의무화했습니다.

4.2. VDP의 5가지 핵심 구성 요소 (CISA 템플릿 기반)

해커원(HackerOne) 등 주요 플랫폼에서 규정하는 VDP의 필수 요소입니다.

1. **목적/약속 선언:** 보안 연구원과 협력하겠다는 의지와 정책의 목적을 명시
2. **범위 (Scope):** 제보 가능한 시스템, 제품, 도메인을 명확히 규정
3. **세이프 하버 (Safe Harbor): 가장 중요한 항목**입니다. 정책을 준수하여 활동한 해커에게는 법적 조치나 처벌을 하지 않겠다는 '면책 선언'입니다.
4. **프로세스 설명:** 보고 방법(이메일, 웹 폼), 제출 형식, 필수 정보 등 기재
5. **평가 및 피드백:** 보고서 검토 소요 시간, 결과 통보 방식 등에 대한 안내

4.3. 버그바운티 플랫폼의 종류

크게 기업이 직접 운영하는 '자체 버그바운티'와 중개 역할을 하는 '플랫폼'으로 나뉩니다.

- **자체 운영:** 구글(Google), 애플(Apple), 네이버(Naver) 등은 자사 사이트에서 직접 제보를 받습니다.
- **중개 플랫폼:** 다수의 기업과 해커를 연결해주는 허브 역할을 합니다.

4.4. 해외 주요 버그바운티 플랫폼

1. **해커원 (HackerOne):**
    - **특징:** 세계 최대 규모 플랫폼. 미 국방부, 구글, MS 등 초대형 고객사 보유
    - **현황:** 해커 100만 명 돌파(2019), 누적 포상금 2억 3천만 달러(2022). 커뮤니티, 교육, 라이브 해킹 이벤트(H1-Challenge) 등이 활발함.
2. **버그크라우드 (Bugcrowd):**
    - **특징:** 해커원의 뒤를 잇는 대형 플랫폼. 테슬라, 아틀라시안 등 유치. 2021년 포브스 선정 유망 스타트업
3. **제로디움 (Zerodium):**
    - **특징: '프리미엄'** 플랫폼. 주로 iOS, 안드로이드, 서버 OS 등의 **제로데이(Zero-day)** 취약점을 고가에 매입합니다. 초보자보다는 고숙련 전문가에게 적합합니다.
4. **헌터 (Huntr):**
    - **특징: 오픈소스(Open Source)** 대상 플랫폼. 깃허브(GitHub) 등의 오픈소스 취약점을 수정하는 데 기여하면 포상합니다. 초보자가 CVE를 획득하고 경험을 쌓기에 가장 좋은 진입점입니다.

4.5. 국내 주요 버그바운티 플랫폼

1. **KISA (한국인터넷진흥원):**
    - **특징:** 공공기관 운영. '보안 취약점 신고포상제'를 통해 S/W 취약점 제보를 받으며, CVE 및 KVE 번호를 부여할 권한을 가집니다.
2. **파인더갭 (Find The Gap):**

    - **특징:** 삼성SDS 사내벤처 '해킹존'에서 분사. 국내 최대 규모이며 커뮤니티 활동, '화이트햇 투게더' 등 다양한 이벤트를 통해 생태계를 활성화하고 있습니다.
3. **패치데이 (PatchDay):**
    - **특징:** 보안 기업 '티오리(Theori)'가 런칭. 교육 플랫폼 '드림핵'과 연계되어 진입 장벽이 낮고, 업비트, 네이버 웨일 등의 프로그램을 운영 중입니다.
4. **제로웨일 (Zerowhale):**
    - **특징:** 파스텔플래닛 운영. 국내 플랫폼 중 상대적으로 **높은 포상금**을 지급하는 것으로 알려져 있습니다 (NHN Toast UI 등 진행).

5. 핵심내용 (Summary)

1. **VDP는 안전벨트:** VDP가 없는 곳을 공격하는 것은 법적 위험이 큽니다. 반드시 'Safe Harbor' 조항이 있는 곳, 혹은 허용된 범위 내에서만 활동해야 합니다.
2. **플랫폼 선택 전략:**
    - **입문자:** 오픈소스를 다루는 **Huntr**나 교육과 연계된 **패치데이**.
    - **고수익 목표:** 제로데이를 다루는 **제로디움**이나 포상금이 높은 **제로웨일/해커원**.
    - **커뮤니티/성장:** 다양한 이벤트가 있는 **해커원**이나 **파인더갭**.
3. **국내외 생태계 확장:** 과거 넷스케이프 단일 프로그램에서 시작해, 이제는 자동차, 블록체인, 오픈소스까지 모든 분야로 버그바운티가 확장되고 있습니다.

6. 마무리 키워드 퀴즈

강의 내용을 제대로 이해했는지 확인해 봅시다.

- **Q1.** 버그헌터가 법적 처벌을 받지 않고 안전하게 취약점을 제보할 수 있도록, 기업이 제보자에게 면책권을 보장하는 VDP의 핵심 조항을 무엇이라고 합니까?
    -
- **Q2.** iOS나 안드로이드 등의 고난이도 '제로데이(Zero-day)' 취약점을 주로 고가에 매입하는 프리미엄 버그바운티 플랫폼의 이름은?
    -
- **Q3.** 국내 공공기관인 KISA에서 운영하며, 신고된 취약점에 대해 CVE 또는 KVE 번호를 부여할 권한을 가진 제도는 무엇입니까?
    -

7. 다음 장 소개

자, 이제 우리는 어디서 활동해야 할지 무대를 알았습니다. 다음 시간에는 "윤리적 해킹의 기본"에 대해 다루겠습니다. 단순히 기술만 있다고 해커가 되는 것이 아닙니다. 블랙햇, 화이트햇, 그레이햇의 차이를 명확히 하고, 여러분이 '정보통신망법'을 준수하며 안전하게 활동하기 위해 반드시 지켜야 할 6가지 원칙을 상세히 짚어드리겠습니다.

4장. 윤리적 해킹의 기본과 법적 주의사항

앞선 강의에서 우리는 버그헌팅이 무엇인지, 어디서 활동해야 하는지를 배웠습니다. 이제는 '어떻게 활동해야 법적인 문제 없이 안전한가'에 대한 가장 중요한 주제를 다룰 차례입니다. 기술이 아무리 뛰어나도 윤리와 법을 어기면 그것은 단순한 범죄에 불과합니다. 이번 시간에는 여러분이 훌륭한 화이트 해커로 성장하기 위해 반드시 지켜야 할 '선'에 대해 명확히 알려드리겠습니다.

---

1. 목적

- **지식 (Knowledge):** 윤리적 해킹의 정의와 6가지 핵심 원칙(사전 동의, 개인정보 존중 등)을 이해하고, '정보통신망법' 등 위반 시 처벌받을 수 있는 법적 근거를 명확히 파악한다.
- **기술 (Skills):** 해킹 활동 수행 시 합법적인 범위(Scope)와 불법적인 침해 행위를 구분하고, 법적 리스크를 최소화하는 시나리오를 설계할 수 있다.
- **태도 (Attitude):** 블랙햇이나 그레이햇이 아닌 '화이트 해커(윤리적 해커)'로서의 정체성을 확립하고, 항상 법과 윤리를 준수하려는 책임감 있는 태도를 갖춘다.

2. 사전지식

- **버그헌팅 기초:** 버그헌팅과 버그바운티의 기본 개념 (이전 강의 내용).
- **기본적인 사회 규범:** 사생활 침해, 무단 침입 등이 범죄라는 기초적인 상식.

3. 들어가기 전에

여러분, 작년 말 뉴스에서 "보안 전문가가 아파트 월패드를 해킹했다가 처벌받았다"는 소식을 들어보셨나요? 보안 전문가로서 실력을 증명하거나 시스템의 취약점을 알리려는 의도였다 하더라도, **합법적인 절차와 동의 없이** 진행된 해킹은 명백한 범죄입니다.

버그헌터는 사실상 "윤리적으로 해킹하겠다"고 약속하고 참여하는 사람들입니다. 하지만 순간의 욕심이나 실수로 '범죄자'가 될 수도 있습니다. 좋은 의도가 면죄부가 되지는 않기 때문입니다. 이번 시간에는 여러분이 억울한 범법자가 되지 않도록, 꼭 지켜야 할 **6가지 원칙**과 **법적 가이드라인**을 아주 상세하게 짚어드리겠습니다.

4. 수업내용

4.1. 윤리적 해킹의 정의와 6대 원칙

윤리적 해킹(Ethical Hacking)이란 합법적이고 윤리적인 방식을 통해 보안 취약점을 찾고 개선하는 일련의 과정을 말합니다. 이를 위해 다음 6가지 원칙을 반드시 준수해야 합니다.

1. **사전 동의 (Prior Consent):**
    - 가장 기본적인 원칙입니다. 시스템 소유자나 관리자의 **명백한 허가** 없이 수행하는 모든 해킹은 불법입니다.
    - ***비유:*** 가족이나 친척 집이라도 허락 없이 마음대로 들어가면 '주거침입'이 되는 것과 같습니다. 허락된 범위와 시간, 규칙을 철저히 지켜야 합니다.
2. **개인정보 존중 (Respect for Privacy):**
    - 취약점을 증명하기 위해 무분별하게 개인정보를 수집해서는 안 됩니다.
    - ***주의:*** "영향력을 보여주려고 수백만 건의 데이터를 뽑았다"는 변명은 통하지 않습니다. 기업 입장에서 개인정보 유출은 치명적인 사고이므로, 증명에 필요한 최소한의 데이터(1~2건)만 확인해야 합니다.
3. **최소한의 침해 (Minimal Harm):**
    - 시스템을 테스트하는 것이 목표지, 파괴하는 것이 목표가 아닙니다.
    - ***금지:*** 원격 코드 실행(RCE) 취약점을 찾았다고 해서 `rm -rf /`(파일 전체 삭제) 같은 위험한 명령어를 실행해서는 안 됩니다. 이는 생태계 전체에 악영향을 끼치는 행위입니다.
4. **보고 의무 (Duty to Report):**
    - 발견한 취약점은 반드시 소유자에게 알리고, 가능한 경우 해결 방안까지 제시해야 합니다.
    - 취약점을 숨기고 나중에 악용하거나, 해결책 없이 문제만 던지는 것보다 **패치 방법**이나 **우회 공격 가능성**까지 제시하면 포상금이 높아질 수 있습니다.
5. **비밀 유지 (Confidentiality):**
    - 취약점 발굴 과정에서 알게 된 정보나 취약점 자체를 외부에 누설해서는 안 됩니다.
    - 기업의 허가 없이 블로그나 SNS에 공개하는 행위는 계약 위반이자 법적 처벌 대상입니다.
6. **법적 준수 (Legal Compliance):**
    - 해당 국가와 지역의 법률을 따라야 합니다. 해외 버그바운티 프로그램에 참여할 때도 그 나라의 법을 존중해야 합니다.

4.2. 국내 법규: 정보통신망법 (법적 처벌의 근거)

국내 활동 시 여러분의 발목을 잡을 수 있는 '정보통신망 이용촉진 및 정보보호 등에 관한 법률(정보통신망법)'의 핵심 조항입니다.

- **제48조 제1항 (침입 금지):** 정당한 접근 권한 없이 또는 허용된 범위를 넘어 정보통신망에 침입해서는 안 됩니다. (허가받지 않은 타겟 공격 금지)
- **제48조 제2항 (데이터 훼손 금지):** 정당한 사유 없이 데이터를 훼손, 멸실, 변경, 위조하거나 운용을 방해하는 프로그램(악성코드 등)을 유포해서는 안 됩니다.
- **제48조 제3항 (장애 발생 금지):** 정보통신망의 안정적 운영을 방해할 목적으로 대량의 신호(DDoS)를 보내거나 부정한 명령을 처리하게 하여 장애를 발생시켜서는 안 됩니다.
- **제49조 (비밀 보호):** 타인의 정보를 훼손하거나 타인의 비밀을 침해, 도용, 누설해서는 안 됩니다. (개인정보 유출 금지)
- **제71조 (벌칙):** 위 조항을 위반할 경우 **5년 이하의 징역 또는 5천만 원 이하의 벌금**에 처하며, **미수범**도 처벌받습니다.

4.3. 해커의 종류와 지향점

윤리적 관점에 따라 해커는 크게 세 가지로 분류됩니다.

1. **블랙 햇 (Black Hat):**
   - 악의적인 의도로 시스템을 파괴하거나 정보를 훔치는 해커.
   - ***사례:*** **케빈 미트닉(Kevin Mitnick)**. 과거 미 국방부 등을 해킹해 체포되었으나, 이후 화이트 해커로 전향하여 보안 컨설턴트로 활동 중입니다.
2. **화이트 햇 (White Hat):**
   - 취약점을 식별하고 방어 대책을 제시하는 '착한 해커'. 우리가 지향해야 할 모델입니다.
   - ***사례:*** **찰리 밀러(Charlie Miller)**. 유명 해킹 대회 Pwn2Own 우승자이자 애플 등의 취약점을 제보한 전문가입니다.
3. **그레이 햇 (Grey Hat):**
   - 악의적 의도는 없으나 허가받지 않은 해킹을 하는 등 윤리적으로 모호한 해커.
   - ***사례:*** **칼릴 슈레이터(Khalil Shreateh)**. 페이스북 취약점 제보 과정에서 응답이 없자 저커버그의 담벼락에 글을 써서 증명했습니다. 의도는 좋았으나 정책(사전 동의, 최소 침해 등)을 위반하여 포상금을 받지 못했습니다.

4.4. 최근의 용어 변화 (Trend)

- '블랙/화이트'라는 용어가 인종 차별적 의미를 담을 수 있다는 지적(구글 엔지니어링 부사장 등)에 따라, 최근에는 '윤리적 해커(Ethical Hacker)'와 '악의적 해커(Malicious Hacker)'라는 용어로 대체하는 추세입니다. 영국 NCSC 등에서도 화이트리스트/블랙리스트 용어 사용을 중단하고 있습니다.

5. 핵심내용 (Summary)

1. **6대 원칙 준수:** 사전 동의, 개인정보 존중, 최소한의 침해, 보고 의무, 비밀 유지, 법적 준수는 윤리적 해커의 필수 덕목입니다.
2. **정보통신망법 주의:** 접근 권한을 넘거나(제48조), 데이터를 훼손/유출(제49조)하는 행위는 강력한 처벌(제71조)을 받으므로 절대 금물입니다.

3. **화이트 해커 지향:** 실력 과시를 위해 법을 어기는 '그레이 햇'이 되지 말고, 합법적 테두리 안에서 가치를 증명하는 '화이트 해커(윤리적 해커)'가 되어야 합니다.
4. **증명의 한계:** 취약점 증명은 '최소한'으로 해야 합니다. 과도한 데이터 추출이나 시스템 파괴는 선의를 인정받기 어렵습니다.

6. 마무리 키워드 퀴즈

오늘 배운 내용을 점검해 봅시다.

- **Q1.** 윤리적 해커가 지켜야 할 6가지 원칙 중, 시스템 소유자로부터 명백한 허가를 받은 후에만 활동해야 한다는 원칙은 무엇인가요?
  -
- **Q2.** 악의적인 의도는 없지만, 허가받지 않은 시스템을 해킹하거나 규정을 위반하여 활동하는 해커를 모자 색깔에 비유하여 무엇이라고 합니까?
  -
- **Q3.** 국내법상 정당한 접근 권한 없이 정보통신망에 침입하거나, 허용된 범위를 넘어 침입하는 행위를 금지하고 있는 법률의 명칭은 무엇인가요?
  -

7. 다음 장 소개

이제 우리는 윤리적, 법적 준비를 마쳤습니다. 다음 시간에는 **"버그헌팅의 이해"** 파트로 넘어가겠습니다. 취약점의 위험도를 숫자로 계산하는 **CVSS(취약점 등급 시스템)**, 그리고 **모의해킹(Penetration Testing)과 버그헌팅의 결정적인 차이점**에 대해 배우며 실전 감각을 익혀보겠습니다.

5장. 버그헌팅의 이해: 취약점 평가(CVSS)와 성공적인 마인드셋

지금까지 우리는 버그헌팅의 정의, 역사, 플랫폼, 그리고 윤리적/법적 기준까지 숨 가쁘게 달려왔습니다. 이제 본격적인 기술 학습에 앞서, 가장 현실적이고 중요한 질문을 던져보겠습니다. **"내가 찾은 이 버그는 과연 얼마의 가치가 있을까?"**

단순히 "위험해 보입니다"라고 말하는 것은 아마추어입니다. 프로는 명확한 '점수'와 '기준'으로 위험을 증명합니다. 이번 시간에는 전 세계 공통의 취약점 평가 기준인 **CVSS**, **모의해킹과 버그헌팅의 결정적 차이**, 그리고 롱런하는 해커가 되기 위한 **마인드셋**을 다루겠습니다.

---

1. 목적

- **지식 (Knowledge):** 취약점의 위험도를 정량적으로 평가하는 CVSS(Common Vulnerability Scoring System)의 구성 요소와 역사를 이해하고, 전통적인 모의해킹(Penetration Testing)과 버그바운티의 차이점을 명확히 구분한다.
- **기술 (Skills):** CVSS v3.1 계산기를 활용하여 발견한 취약점의 공격 벡터(AV), 복잡도(AC), 영향도(C/I/A) 등을 입력해 베이스 스코어(Base Score)를 산출할 수 있다.
- **태도 (Attitude):** 자동화 도구에만 의존하지 않고 수동 분석을 병행하며, 단순 버그 발견을 넘어 '비즈니스 임팩트(Impact)'를 증명하려는 주도적인 연구 자세를 갖춘다.

2. 사전지식

- **기초 보안 용어:** 기밀성(Confidentiality), 무결성(Integrity), 가용성(Availability) - CIA 3요소.
- **이전 강의 내용:** 버그헌팅과 버그바운티의 개념 이해.

3. 들어가기 전에

여러분이 아주 심각한 보안 구멍을 찾았다고 칩시다. 기업 담당자에게 "이거 정말 위험해요!"라고 메일을 보냈는데, "그래서 점수가 몇 점입니까?"라는 답변이 온다면 당황스럽겠죠?

주관적인 '감'이 아니라 객관적인 '수치'로 위험을 말할 수 있어야 합니다. 이를 위해 전 세계적으로 쓰이는 표준이 바로 **CVSS**입니다. 또한, 많은 분들이 헷갈려 하는 '모의해킹'과 '버그헌팅'이 실무에서 어떻게 다른지, 그리고 수많은 실패 속에서도 포기하지 않고 '상위 1% 해커'가 되기 위해선 어떤 마음가짐이 필요한지 선배 해커들의 노하우를 빌려 설명해 드리겠습니다.

4. 수업내용

4.1. 취약점 리스크 정량화 (Vulnerability Risk Quantification)

- **필요성:** "얼마나 위험한가?"라는 질문에 대해 객관적이고 수치화된 기준을 제시하여, 보안 팀이 우선순위를 정해 효율적으로 자원을 배치할 수 있게 돕습니다.
- **주요 모델:**
  - **CVSS (Common Vulnerability Scoring System):** 가장 널리 쓰이는 표준 프레임워크.
  - **DREAD:** 마이크로소프트 개발 (Damage, Reproducibility, Exploitability, Affected Users, Discoverability).
  - **OWASP Risk Rating:** 웹 애플리케이션 보안 위험 평가 방법론.

4.2. CVSS의 발전과 구성 요소

CVSS는 v1(2004)에서 시작해 현재 v3.1(2019)이 주로 사용되며, v4도 개발 중입니다.

- **점수 체계: Base(기본), Temporal(시간적), Environmental(환경적)** 세 가지 그룹으로 나뉩니다. 버그바운티에서는 주로 취약점 고유의 특성인 **Base Score**를 사용합니다.

- **CVSS v3.1 핵심 매트릭 (Base Metrics):**

  - **공격 벡터 (Attack Vector, AV):** 공격 가능한 위치 (네트워크 > 인접 > 로컬 > 물리). 네트워크(Network)일수록 점수가 높습니다.
  - **공격 복잡도 (Attack Complexity, AC):** 공격 성공에 필요한 조건. 낮을수록(Low) 위험합니다.
  - **필요 권한 (Privileges Required, PR):** 공격자가 가져야 할 권한. 없음(None)일수록 위험합니다.
  - **사용자 상호작용 (User Interaction, UI):** 피해자의 행동(클릭 등) 필요 여부. 없음(None)이 위험합니다.
  - **영향 범위 (Scope, S):** 취약점이 다른 보안 영역(컴포넌트)까지 영향을 미치는지 여부 (Changed/Unchanged).
  - **CIA 영향도 (Confidentiality, Integrity, Availability):** 기밀성, 무결성, 가용성 침해 수준 (None, Low, High).
- **실제 사례 (Log4j vs 기타):**

  - **Log4j (Log4Shell):** 네트워크 접근 가능, 권한/상호작용 불필요, CIA 모두 High → **Critical 10.0점**.
  - **Log4j (관련 다른 취약점):** 로컬 접근 필요, 복잡도 높음 → 점수 낮음.

4.3. 모의해킹(Penetration Testing) vs 버그헌팅(Bug Hunting)

둘 다 취약점을 찾지만, 목적과 운영 방식이 다릅니다.

구분	모의해킹 (Pen-Testing)	버그헌팅/버그바운티 (Bug Bounty)

진행 주체	내부 팀 또는 계약된 컨설팅 회사 (소수)	전 세계의 불특정 다수 해커 (집단지성)
기간/범위	정해진 기간, 제한된 범위 내 집중 점검	기간 제한 없음(상시), 범위가 넓음
목표	전체적인 리스크 평가, 상세 보고서, 이행 점검	개별 취약점 발굴, 유효한 리포트 획득
보상	컨설팅 용역 비용 (시간/인력 기반)	건당 포상금 (성과 기반)
특징	깊이 있는 진단, 체계적 관리 가능	다양한 시각, 지속적인 점검 가능

4.4. 성공적인 버그헌팅을 위한 마인드셋 (by Yassine Aboukir)

상위 랭커 해커의 조언을 통해 배우는 4가지 핵심 전략입니다.

1. **깊이 있는 정찰 (In-depth Reconnaissance):**
   - 단순히 서브도메인만 찾지 말고, 모바일 앱 디컴파일, JS 파일 분석 등을 통해 숨겨진 파라미터나 API를 찾아야 합니다.
2. **수동 진단의 중요성 (Manual Testing):**
   - 자동화 도구는 누구나 돌립니다. 도구가 찾지 못하는 **비즈니스 로직(Logic Flow)** 오류나 **SSO(싱글사인온) 우회** 같은 복잡한 취약점은 수동 분석으로만 찾을 수 있습니다.
3. **임팩트(Impact) 증명:**
   - 단순 XSS(스크립트 실행) 제보에 그치지 말고, 이를 통해 '세션 토큰을 탈취'하거나 '계정을 탈취'하는 시나리오까지 연결해야 높은 보상(Critical)을 받을 수 있습니다. 기술적/시나리오적으로 파급력을 높이는 것이 핵심입니다.
4. **지속적인 학습과 즐기는 태도:**
   - 처음엔 중복(Duplicate)이나 실패가 많을 수 있습니다. 포상금보다 '새로운 것을 배웠다'는 성취감에 집중하고, 다른 해커들의 리포트(Write-up)를 보며 끊임없이 연구해야 합니다.

5. 핵심내용 (Summary)

1. **CVSS는 공통 언어:** 취약점의 위험도는 '감'이 아닌 CVSS 점수(Base Score)로 소통합니다. 공격의 난이도가 낮을수록, 피해 영향도가 클수록 점수가 높아집니다.
2. **상호 보완적 관계:** 모의해킹은 '깊이 있고 체계적인 진단'을, 버그바운티는 '다양한 시각의 지속적인 진단'을 제공하므로 기업은 두 가지를 병행하는 것이 가장 효과적입니다.
3. **Impact가 돈이다:** 단순히 버그의 존재를 알리는 것을 넘어, 그 버그가 비즈니스에 미칠 실질적인 피해(계정 탈취, 정보 유출 등)를 시나리오로 증명해야 합니다.
4. **자동화 < 수동화:** 자동화 도구는 보조 수단일 뿐입니다. 남들이 보지 못하는 로직의 허점은 끈기 있는 **수동 분석**에서 나옵니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 취약점의 심각도를 평가하는 국제 표준 점수 체계로, 공격 벡터, 복잡도, 필요한 권한 등의 요소를 통해 0~10점 사이의 점수를 매기는 시스템은 무엇인가요?
    -
- **Q2.** CVSS 평가 항목 중, 공격자가 취약점을 악용하기 위해 피해자(사용자)의 특정 행동(클릭 등)이 필요한지 여부를 나타내는 지표는 무엇인가요?
    -
- **Q3.** 정해진 기간과 인력으로 수행하는 '모의해킹'과 달리, 불특정 다수의 해커가 참여하여 성과(취약점 발견)에 따라 보상을 받는 방식은 무엇인가요?
    -

7. 다음 장 소개

이론적인 무장을 마쳤으니, 이제 전장으로 나갈 차례입니다. 다음 시간에는 **"웹 기본 이해"** 파트로 들어갑니다. 웹 애플리케이션이 어떻게 작동하는지, HTTP 프로토콜과 패킷의 구조는 어떻게 되어 있는지, 그리고 프록시 도구(Burp Suite)를 어떻게 사용하는지 기초부터 탄탄히 다져보겠습니다.

6장. 웹 애플리케이션의 작동 원리와 핵심 프로토콜

우리는 앞서 버그헌팅의 개념과 역사, 그리고 중요한 윤리적 기준들을 배웠습니다. 이제부터는 진짜 기술의 영역으로 들어갑니다. 건물을 짓기 위해선 기초 공사가 튼튼해야 하듯, 웹 해킹을 잘하기 위해서는 **'웹이 어떻게 작동하는지'** 그 근본 원리를 완벽하게 이해해야 합니다.

---

1. 목적

- **지식 (Knowledge):** 웹을 구성하는 핵심 요소(HTTP, HTML/CSS/JS, URI)의 정의와 역할을 이해하고, DNS의 작동 원리 및 레코드 유형(A, CNAME)을 설명할 수 있다.
- **기술 (Skills):** HTTP 메시지의 구조(Header, Body, CRLF)를 분석하고, URI의 각 구성 요소(Scheme, Host, Port, Path, Query, Fragment)를 식별하여 취약점 분석에 활용할 수 있다.
- **태도 (Attitude):** 화려한 공격 기술 이전에 기본 프로토콜(Protocol)을 이해하는 것이 보안의 시작임을 인지하고, 사소해 보이는 설정(DNS 등)에서도 취약점을 찾아내려는 세밀한 관찰력을 기른다.

2. 사전지식

- **인터넷 기초:** IP 주소와 도메인에 대한 기본적인 개념.
- **기본 컴퓨터 상식:** 클라이언트(사용자 PC)와 서버의 역할에 대한 기초 이해.

3. 들어가기 전에

"버그바운티의 90% 이상은 웹(Web)을 대상으로 합니다."

여러분이 아무리 뛰어난 해킹 툴을 가지고 있어도, 그 대상인 '웹'이 어떤 언어로 대화하고 어떻게 길을 찾는지 모른다면 무용지물입니다. 우리가 매일 쓰는 인터넷 주소창의 URL 하나에도 수많은 규칙과 약속이 숨어 있습니다. 개발자들의 아주 사소한 실수나 설정 오류가 훗날 치명적인 '서브도메인 테이크오버(Subdomain Takeover)' 같은 취약점으로 이어지기도 하죠.

이번 시간에는 웹의 3대 요소인 **HTTP, URI, DNS**를 아주 깊이 있게, 해커의 관점에서 뜯어보겠습니다. 기초라고 가볍게 여기지 마십시오. 30년 경력의 전문가로서 단언컨대, 가장 치명적인 취약점은 가장 기본적인 원리 속에 숨어 있습니다.

4. 수업내용 (영상 시간)

4.1. 웹 애플리케이션의 기본 구조 및 작동 원리

- **웹(Web)의 정의:** HTTP 프로토콜을 이용하여 정보를 공유하는 서비스입니다. 정보를 주는 쪽을 **웹 서버**, 받는 쪽을 **웹 클라이언트**라고 합니다.
- **웹의 진화와 보안의 중요성:**
  - 초기에는 단순 문서 공유 형태였으나, 현재는 금융, 쇼핑, SNS 등 복잡한 서비스로 진화했습니다.
  - 집 주소, 카드 번호 등 민감한 개인정보(PII)가 오가기 때문에, 정보 유출 시 심각한 피해를 막기 위한 웹 보안의 중요성이 대두되었습니다.
- **프론트엔드(Front-end)와 웹 리소스:**
  - 사용자의 요청을 받는 부분으로, 웹 리소스(Web Resource)로 구성됩니다.
  - **404 Not Found:** 리소스(Resource)를 찾지 못했다는 의미입니다. 우리가 URL로 접근하는 것은 서버에 있는 특정 리소스(파일 등)를 가져오라는 명령입니다.

4.2. 웹 리소스의 3대 요소 (HTML, CSS, JS)

프론트엔드를 구성하는 핵심 언어들입니다.

1. **HTML (Hypertext Markup Language):** 웹 문서의 **뼈대**를 담당합니다. 태그와 속성을 통해 구조화된 문서를 작성합니다.
2. **CSS (Cascading Style Sheets):** 웹 문서의 생김새(시각화)를 담당합니다. 글자 크기, 색상, 배경 등을 지정합니다.
3. **JavaScript (JS):** 웹 문서의 **동작**을 정의합니다. 클릭, 스크롤 반응, 데이터 전송 등을 구현합니다.
   - ***전문가 Tip:*** 클라이언트 사이드 취약점(XSS 등)을 발굴하려면 **JavaScript와 HTML**에 대한 이해가 필수적입니다.

4.3. URI (Uniform Resource Identifier)의 구조

- **정의:** 인터넷 자원을 고유하게 식별하고 위치를 지정하는 문자열입니다. URL은 URI의 하위 집합입니다.
- **상세 구조 분석:**
  - **Scheme (스킴):** 프로토콜을 나타냅니다. (예: `http`, `https`, `ftp`).
  - **User (유저):** 사용자 이름과 비밀번호(대부분 생략됨).
  - **Host (호스트):** 도메인 이름이나 IP 주소.
  - **Port (포트):** 서비스 포트 번호. 생략 시 기본값(HTTP: 80, HTTPS: 443)이 사용됩니다.
  - **Path (패스):** 서버상의 자원 경로.
  - **Query (쿼리):** 서버에 전달하는 추가 정보(파라미터). 보통 `?key=value` 형태입니다.
  - **Fragment (프래그먼트):** 문서 내 특정 부분을 가리키는 식별자. `#`(해시태그) 뒤에 옵니다.
  - ***중요성:*** 개발자가 URI 구조를 잘못 이해하거나 처리할 때 취약점이 발생하므로, 해커는 이 구조를 완벽히 파악해야 합니다.

4.4. HTTP 프로토콜과 메시지 구조

- **HTTP (HyperText Transfer Protocol):** 클라이언트의 요청(Request)과 서버의 응답(Response)으로 이루어진 데이터 교환 규칙입니다.
- **메시지 구조:** `Header(헤더)`와 `Body(바디)`로 구성됩니다.
  - **CRLF:** 헤더의 각 줄은 `\r\n` (Carriage Return + Line Feed)으로 구분됩니다.
  - **Header:** 필드와 값으로 구성되며 속성을 나타냅니다. 헤더의 끝은 CRLF 한 줄로 표시됩니다.
  - **Body:** 헤더가 끝난 뒤에 오는 데이터 부분입니다.
- **HTTPS와 TLS:** HTTP는 평문 통신이라 도청 위험이 있습니다. 이를 보완하기 위해 **TLS(Transport Layer Security)** 프로토콜을 도입하여 메시지를 암호화한 것이 HTTPS입니다.

4.5. DNS (Domain Name System)의 원리

- **정의:** 사람이 읽을 수 있는 도메인 이름(예: `google.com`)을 컴퓨터가 이해하는 IP 주소(예: `8.8.8.8`)로 변환해주는 시스템입니다.
- **작동 프로세스 (예: www.google.com 접속 시):**
  - 브라우저가 **Local DNS 서버**에 IP를 물어봅니다. (캐싱되어 있으면 바로 반환)
  - 없다면 **Root DNS 서버**에 `.com`을 관리하는 서버 위치를 묻습니다.
  - `.com` 서버에게 `google.com`의 정보를 묻습니다.
  - 최종적으로 `www.google.com`의 IP를 받아 접속합니다.
- **주요 레코드:**
  - **A Record:** 도메인을 서버의 IP와 **직접 연결**합니다. 속도가 빠릅니다.
  - **CNAME (Canonical Name):** 도메인을 다른 도메인 이름(별칭)과 연결합니다. IP가 유동적인 환경에서 유용하지만 단계를 한 번 더 거칩니다.
  - ***전문가 Tip:*** DNS 설정 미흡은 '서브도메인 테이크오버'와 같은 심각한 취약점으로 연결될 수 있어 버그헌팅에서 중요한 제보 대상입니다.

5. 핵심내용 (Summary)

1. **웹의 기초:** 웹은 HTTP 프로토콜 위에서 작동하며, HTML(구조), CSS(표현), JS(동작) 세 가지 리소스로 화면을 구성합니다.
2. **URI 해부:** 스킴, 호스트, 포트, 패스, 쿼리, 프래그먼트 등 URI의 각 요소는 서버와 통신하는 정밀한 주소 체계이며, 이를 조작하여 취약점을 찾을 수 있습니다.
3. **HTTP vs HTTPS:** 평문으로 전송되는 HTTP의 보안 위협을 해결하기 위해 TLS 암호화를 적용한 것이 HTTPS입니다.
4. **DNS와 보안:** 도메인을 IP로 바꿔주는 DNS 시스템(A레코드, CNAME 등)의 원리를 이해해야 하며, 이는 인프라 관련 취약점 발굴의 핵심 열쇠가 됩니다.

6. 마무리 키워드 퀴즈

강의 내용을 제대로 이해했는지 확인해 봅시다.

- **Q1.** 웹 문서의 '동작'을 정의하며, 클라이언트 사이드 취약점 분석을 위해 필수적으로 알아야 하는 언어는 무엇인가요?
  -
- **Q2.** HTTP 메시지에서 헤더(Header)와 바디(Body)를 구분하거나, 헤더의 각 줄을 구분하기 위해 사용되는 특수 문자는 무엇인가요?
  -
- **Q3.** 도메인 이름을 IP 주소로 연결할 때, IP 주소가 아닌 '다른 도메인 이름(별칭)'으로 연결하는 DNS 레코드 방식은 무엇인가요?
  -

7. 다음 장 소개

웹의 겉모습과 주소 체계를 익혔으니, 이제는 실제로 데이터를 주고받는 대화법을 배울 차례입니다. 다음 시간에는 "웹 클라이언트와 서버의 통신 원리"에 대해 깊이 파고들어, 실제 요청(Request)과 응답(Response)이 어떻게 이루어지는지 알아보겠습니다.

7장. 웹 클라이언트와 서버의 통신 원리 및 핵심 요소

지난 시간에는 웹의 겉모습과 주소 체계(URL), 그리고 DNS에 대해 배웠습니다. 이번 시간에는 **"사용자의 클릭 한 번이 어떻게 서버를 거쳐 다시 우리 눈앞에 나타나는지"** 그 보이지 않는 여정을 기술적으로 파헤쳐 보겠습니다. 이 통신 과정을 이해하는 것은 버그헌팅의 '기본기'이자 가장 강력한 무기입니다.

---

1. 목적

- **지식 (Knowledge):** 웹 서버(Web Server)와 웹 애플리케이션 서버(WAS), DB로 이어지는 3계층 구조를 이해하고, HTTP Method(GET, POST 등)의 특징과 차이점, 그리고 주요 헤더(Cookie, User-Agent)의 보안적 의미를 설명할 수 있다.
- **기술 (Skills):** HTTP 요청/응답 패킷을 분석하여 데이터의 흐름을 파악하고, 각 Method(PUT, DELETE 등)와 헤더 값을 변조하여 잠재적인 취약점을 탐색하는 시나리오를 구상할 수 있다.
- **태도 (Attitude):** 단순히 화면에 보이는 결과에 만족하지 않고, 보이지 않는 패킷의 이면을 분석하려는 탐구 자세와 보안 설정(HttpOnly 등)의 중요성을 인지하는 전문가적 마인드를 갖춘다.

2. 사전지식

- **HTTP 프로토콜 기초:** 6장에서 학습한 HTTP 메시지 구조(Header, Body)에 대한 이해.
- **기본 IT 용어:** 클라이언트, 서버, 데이터베이스에 대한 기초 개념.

3. 들어가기 전에

여러분, 우리가 쇼핑몰에서 '결제하기' 버튼을 누르면 어떤 일이 벌어질까요? 단순히 화면이 바뀌는 것 같지만, 그 뒤에서는 클라이언트(브라우저)와 서버가 수많은 대화를 주고받습니다.

"짜장면 주세요"라고 주문(Request)하면 주방장(WAS)이 요리해서 웨이터(Web Server)가 가져다주(Response)는 식당과 비슷합니다. 하지만 만약 주문서(Packet)를 중간에 조작해서 "계산은 안 하고 음식만 달라"고 한다면 어떨까요?

이번 시간에는 이 통신 과정의 각 단계에서 어떤 취약점이 발생할 수 있는지, 해커의 관점에서 통신 원리를 낱낱이 해부해 보겠습니다. 특히 **GET과 POST의 차이**, **쿠키의 보안 옵션**, **User-Agent를 이용한 속임수**는 실전 버그헌팅에서 매우 빈번하게 활용되는 핵심 개념입니다.

4. 수업내용

4.1. 웹 아키텍처: 클라이언트와 서버의 역할 분담

웹은 중앙 집중식 구조를 가집니다. 클라이언트가 요청하고 서버가 응답합니다.

- **웹 서버 (Web Server):** 가장 앞단에서 요청을 받습니다. (예: Nginx, Apache)
  - **역할:** 정적 콘텐츠(이미지, CSS, JS 등)는 직접 처리하여 빠르게 반환하고, 동적 요청은 뒤쪽의 WAS로 넘깁니다.
- **웹 애플리케이션 서버 (WAS):** 실제 비즈니스 로직을 처리합니다. (예: Tomcat, Django, Node.js)
  - **역할:** 사용자 요청에 따라 DB에서 데이터를 조회하거나 파일을 처리한 후, 결과를 만들어 웹 서버에 돌려줍니다.
- **데이터베이스 (DB):** 데이터를 저장하고 관리합니다. WAS가 필요할 때마다 접속하여 데이터를 가져옵니다.

4.2. HTTP GET 요청: 정보 조회

- **특징:** 서버로부터 정보를 '요청(조회)'하는 가장 일반적인 방법입니다.
- **데이터 전송:** URL 뒤에 **쿼리 파라미터(Query Parameter)** 형태(`?key=value`)로 데이터를 붙여 보냅니다.
- **장점 및 특성:**
  - **캐싱(Caching):** 동일한 요청의 결과를 저장해두어 속도가 빠릅니다.
  - **기록 및 공유:** 브라우저 히스토리에 남고, 북마크(즐겨찾기)가 가능합니다.
  - **멱등성(Idempotency):** 여러 번 요청해도 서버의 상태를 변경하지 않고 항상 같은 결과를 반환합니다.
  - ***주의:*** URL 길이 제한이 있어 대용량 데이터 전송에는 부적합하며, 민감 정보가 URL에 노출될 수 있어 보안상 주의가 필요합니다.

4.3. HTTP 응답(Response) 구조

서버가 클라이언트에게 보내는 답변입니다.

1. **상태 코드 (Status Code):** 처리 결과를 3자리 숫자로 알립니다. (예: 200 성공, 404 찾을 수 없음, 500 서버 오류).
2. **헤더 (Header):** 응답 데이터의 정보(메타데이터)를 담습니다. (콘텐츠 타입, 캐싱 정책, 쿠키 설정 등).
3. **본문 (Body):** 실제 요청한 데이터(HTML, JSON 등)가 들어있습니다.

4.4. HTTP POST 요청: 상태 변경

- **특징:** 서버에 데이터를 '전송'하여 리소스를 생성하거나 상태를 변경할 때 사용합니다.
- **데이터 전송:** URL이 아닌 요청 본문(Body)에 데이터를 담아 보냅니다.
- **특성:**
  - **대용량 전송:** 데이터 크기 제한이 없습니다.
  - **캐싱/기록 안 됨:** 보안상의 이유로 히스토리에 남지 않고 캐싱되지 않습니다.
  - **비멱등성:** 같은 요청을 여러 번 보내면 서버에 여러 개의 글이 생성되는 등 상태가 계속 변합니다.
  - ***참고:*** 새로고침 시 "양식 다시 제출" 경고가 뜨는 이유는 중복 결제나 중복 글쓰기를 방지하기 위함입니다.

4.5. 기타 HTTP Method와 버그헌팅 전략

버그헌터는 다양한 메서드를 시도하여 숨겨진 취약점을 찾습니다.

- **PUT:** 데이터 업데이트용. 권한 검증 미흡 시 타인의 정보를 수정할 수 있는지 테스트합니다.
- **DELETE:** 리소스 삭제용. 권한 없는 사용자가 삭제 요청을 보냈을 때 처리되는지 확인합니다.
- **OPTIONS:** 서버가 지원하는 메서드 목록을 확인하여, 허용되지 않아야 할 메서드(PUT, DELETE 등)가 열려있는지 정찰합니다.
- **TRACE:** 요청이 서버에 도달하는 경로를 추적(루프백)하여, 헤더 변조 여부 등을 디버깅합니다.

4.6. 핵심 헤더의 이해 및 활용

- **Set-Cookie & HttpOnly:**
  - 서버는 로그인 성공 시 `Set-Cookie` 헤더로 세션 ID를 발급합니다.
  - **HttpOnly 옵션:** 이 옵션이 설정되면 자바스크립트(`document.cookie`)로 쿠키에 접근할 수 없습니다. 이는 **XSS(크로스 사이트 스크립팅)** 공격으로 세션을 탈취당하는 것을 막는 매우 중요한 보안 장치입니다. 버그헌팅 시 이 설정이 누락되었는지 반드시 확인해야 합니다.
- **User-Agent:**
  - 클라이언트의 정보(브라우저 종류, OS 등)를 서버에 알립니다.
  - ***버그헌팅 Tip:*** 서버가 모바일/PC 환경에 따라 다르게 반응하거나, 특정 브라우저만 허용하는 경우 User-Agent를 조작하여 접근 통제를 우회하거나 숨겨진 취약점을 찾을 수 있습니다.

5. 핵심내용 (Summary)

1. **3단 구조:** 웹 통신은 Web Server(정적) - WAS(동적/로직) - DB(데이터)의 협력으로 이루어지며, 각 단계에서 취약점이 발생할 수 있습니다.
2. **GET vs POST:**
   - **GET:** 조회용, URL에 데이터 노출, 캐싱됨, 멱등성 O.
   - **POST:** 변경용, Body에 데이터 숨김, 캐싱 안 됨, 멱등성 X.
3. **보안 헤더: HttpOnly** 설정은 XSS 공격 시 쿠키 탈취를 막는 핵심 방어 기제이므로, 점검 1순위 대상입니다.
4. **입력값 검증:** 헤더(User-Agent)나 메서드(PUT/DELETE) 등 사용자가 보낼 수 있는 모든 입력값은 조작 가능하며, 이를 통해 서버의 예외 처리를 테스트해야 합니다.

6. 마무리 키워드 퀴즈

강의 내용을 제대로 이해했는지 확인해 봅시다.

- **Q1.** HTTP 응답 헤더 중 하나로, 자바스크립트를 통한 쿠키 접근을 차단하여 XSS 공격으로부터 세션 ID를 보호하는 보안 옵션은 무엇인가요?
  -
- **Q2.** 서버에 데이터를 전송하여 상태를 변경할 때 주로 사용되며, 요청 본문(Body)에 데이터를 담아 보내고 실행 결과가 매번 달라질 수 있는(비멱등성) HTTP 메서드는 무엇인가요?
  -
- **Q3.** 클라이언트의 브라우저 및 운영체제 정보를 담고 있는 헤더로, 버그헌팅 시 이를 조작하여 모바일 환경으로 위장하거나 접근 제한을 우회할 때 사용하는 헤더는 무엇인가요?
  -

7. 다음 장 소개

웹 통신의 기본 원리를 익혔으니, 이제 본격적으로 웹 애플리케이션을 위협하는 대표적인 취약점들을 공략할 차례입니다. 다음 시간에는 "OWASP Top 10 주요 취약점"으로 넘어가, 실제 현장에서 가장 많이 발견되는 위험한 취약점들의 원리와 공격 기법을 실습과 함께 상세히 다루도록 하겠습니다.

8장. HTTP 응답 코드의 비밀과 프록시 도구(Burp Suite) 활용

지난 강의에서 웹의 통신 구조(HTTP, Request/Response)를 머리로 이해했다면, 이번에는 그 통신 과정을 **'눈으로 보고 손으로 조작하는'** 실전 기술을 익힐 차례입니다. 해커에게 웹 브라우저 화면은 빙산의 일각일 뿐입니다. 수면 아래에서 오고 가는 '진짜 정보'를 장악하는 자만이 취약점을 찾아낼 수 있습니다.

---

1. 목적

- **지식 (Knowledge):** HTTP 상태 코드(200, 300, 400, 500번대)의 의미를 정확히 해석하고, 프록시(Proxy) 서버가 클라이언트와 서버 사이에서 어떤 역할을 수행하는지 이해한다.
- **기술 (Skills):** 대표적인 웹 프록시 도구인 **Burp Suite**를 설치 및 구동하여 트래픽을 캡처(Intercept)하고, **Repeater**와 **Intruder** 기능을 통해 패킷을 변조 및 재전송하는 실습 능력을 갖춘다.
- **태도 (Attitude):** 404, 500 등의 에러 코드를 단순한 오류로 보지 않고 서버의 구조나 취약점을 유추하는 '단서'로 활용하는 분석적 태도를 함양한다.

2. 사전지식

- **HTTP 프로토콜:** 요청(Request)과 응답(Response)의 기본 구조 (이전 강의 내용).
- **기본 네트워크 지식:** IP 주소와 포트(Port)에 대한 기초 이해.

3. 들어가기 전에

여러분, "아는 만큼 보인다"는 말이 있죠. 일반 사용자는 웹사이트에서 '404 Not Found'를 보면 "페이지가 없네" 하고 뒤로 가기를 누릅니다. 하지만 우리 같은 버그헌터는 "아, 이 경로에는 파일이 없지만 서버가 살아있고, URL 패턴이 이런 식이니 다른 파일을 넣어볼까?"라고 생각합니다. 에러 메시지는 서버가 우리에게 보내는 은밀한 신호입니다.

또한, 여러분이 식당에서 주문을 할 때 웨이터(Proxy)가 주방장(Server)에게 주문서를 전달하기 전에 내용을 몰래 바꿔치기한다면 어떻게 될까요? "짜장면"을 "탕수육"으로 바꿔서 전달했는데 주방장이 그대로 요리해준다면? 이것이 바로 우리가 오늘 배울 **프록시를 이용한 패킷 변조**의 핵심입니다. 이제 이 강력한 무기를 다루는 법을 배워봅시다.

4. 수업내용

4.1. HTTP 응답 코드(Status Code) 완벽 분석

서버가 클라이언트의 요청을 어떻게 처리했는지 알려주는 3자리 숫자입니다. 이를 해석하는 능력은 버그헌팅의 기초입니다.

- **200번대 (성공):**

    - **200 OK:** 요청이 정상적으로 처리되었고 필요한 정보가 반환되었습니다.
- **300번대 (리다이렉션):**
    - 클라이언트의 요청을 완료하기 위해 추가 동작(이동)이 필요합니다.
    - **302 Found:** 리소스가 다른 위치로 이동했으니 그곳으로 다시 요청하라는 의미입니다.
- **400번대 (클라이언트 에러):**
    - 요청 자체에 문제가 있는 경우입니다.
    - **400 Bad Request:** 요청 형식이 잘못됨.
    - **401 Unauthorized:** 인증이 필요함 (로그인 안 됨).
    - **403 Forbidden:** 접근 권한이 없음 (관리자 페이지 등).
    - **404 Not Found:** 요청한 리소스를 찾을 수 없음.
- **500번대 (서버 에러):**
    - 서버 내부의 문제로 요청을 처리하지 못한 경우입니다.
    - **500 Internal Server Error:** 서버 로직 에러 (SQL 인젝션 시 자주 발생).
    - **503 Service Unavailable:** 서버 과부하 등으로 잠시 서비스 불가.

4.2. 버그헌팅 관점에서의 응답 코드 해석

단순한 상태 확인을 넘어, 취약점 발굴의 신호로 해석해야 합니다.

- **이상 징후 포착:** 인증이 필요 없는 공개 페이지 요청에 `401 Unauthorized`가 뜬다면, 서버의 인증 로직이 꼬여있을 가능성이 있습니다.
- **404 에러의 가치:**
    - `Home`을 `HOOME`로 오타를 냈을 때 404가 뜬다면, 서버가 해당 경로를 인식하고 있다는 뜻입니다.
    - 이를 통해 `user/1`, `user/2` 같은 URL 패턴을 유추하고, `user/admin` 같은 숨겨진 경로를 예측(Fuzzing)하여 접근 통제 미흡 취약점을 찾을 수 있습니다.
    - **에러 페이지 노출:** 기본 404 페이지가 아닌, 개발자가 설정한 에러 페이지에서 서버 버전이나 내부 IP 등 민감 정보가 노출되는지 확인해야 합니다.

4.3. 프록시(Proxy)의 개념과 역할

- **일반적인 통신:** 클라이언트(브라우저) $\leftrightarrow$ 서버 (직접 연결).
- **프록시 통신:** 클라이언트 $\leftrightarrow$ **프록시(Proxy)** $\leftrightarrow$ 서버.
    - 프록시는 중간에서 모든 트래픽을 **중계**합니다.
    - **해커의 이점:** 요청을 서버로 보내기 전에 **가로채서(Intercept)** 내용을 수정하거나, 서버의 응답을 사용자에게 보여주기 전에 조작할 수 있습니다.

4.4. 버프슈트(Burp Suite) 실습 및 핵심 기능

전 세계 해커들이 가장 많이 사용하는 프록시 도구입니다.

- **설정 및 구동:**
    - `Next` $\rightarrow$ `Start Burp`로 실행합니다.
    - **Proxy -> Open Browser:** 별도의 브라우저 프록시 설정 없이, 내장된 브라우저(Chromium)를 열어 바로 패킷을 잡을 수 있습니다.
- **인터셉트(Intercept):**

    - `Intercept is on` 상태에서 네이버 등에 검색어를 입력하면, 요청이 서버로 가지 않고 버프슈트에 멈춥니다.
    - ***Tip:*** 기본적으로 Request만 잡히므로, Response도 보고 싶다면 옵션(Options)에서 `Intercept Server Responses` 항목을 체크해야 합니다.
- **리피터 (Repeater):**
    - 캡처한 패킷을 '재사용'하는 기능입니다. 우클릭 $\rightarrow$ `Send to Repeater`.
    - 매번 브라우저에서 버튼을 누를 필요 없이, 리피터 탭에서 파라미터 값(예: 검색어 'test' $\rightarrow$ 'asdasd')만 바꿔가며 `Send`를 눌러 서버의 반응을 즉시 확인할 수 있습니다.
- **인트로더 (Intruder):**
    - **'자동화 공격'** 도구입니다. 우클릭 $\rightarrow$ `Send to Intruder`.
    - **설정 방법:**
        1. 변조하고 싶은 부분(파라미터 값)을 드래그하여 `Add` 버튼으로 지정합니다 (Payload Position).
        2. `Payloads` 탭에서 대입할 문자열 리스트(예: 123, 234, test...)를 입력합니다.
        3. `Start Attack`을 누르면 설정한 값들을 자동으로 대입하여 서버의 응답을 리스트로 보여줍니다. 무차별 대입(Brute Force)이나 퍼징(Fuzzing)에 필수적입니다.

5. 핵심내용 (Summary)

1. **상태 코드는 나침반:** 200(성공), 302(이동), 400(클라이언트 실수), 500(서버 실수)의 의미를 파악하고, 특히 404나 500 에러를 통해 서버 구조와 로직 오류를 유추해야 합니다.
2. **프록시는 수술대:** 프록시(Burp Suite)는 클라이언트와 서버 사이의 통신을 수술대 위에 올려놓고 해부(분석)하고 성형(변조)하는 도구입니다.
3. **Repeater & Intruder:**
    - **Repeater:** 수동으로 값을 정밀하게 바꿔가며 취약점을 검증할 때 사용.
    - **Intruder:** 다수의 값을 자동으로 대입하여 숨겨진 경로나 취약점을 탐색할 때 사용.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 클라이언트의 요청에는 문제가 없으나, 서버 내부의 로직 에러나 DB 오류 등으로 인해 요청을 처리할 수 없을 때 반환되는 HTTP 상태 코드 대역은 무엇인가요?
    -
- **Q2.** 웹 프록시 도구인 Burp Suite에서, 캡처한 요청 패킷을 하나씩 수동으로 변조하여 서버로 재전송하고 응답을 분석하는 데 특화된 탭(기능)의 이름은 무엇인가요?
    -
- **Q3.** 클라이언트가 요청한 리소스가 서버에 존재하지 않을 때 발생하는 에러 코드로, 해커가 이를 통해 서버의 디렉터리 구조나 URL 패턴을 유추하는 데 활용하는 코드는 무엇인가요?
    -

7. 다음 장 소개

이제 우리는 웹의 언어(HTTP)를 배웠고, 대화(패킷)를 조작할 도구(Burp Suite)도 손에 넣었습니다. 다음 시간에는 이 도구들을 활용하여 웹 애플리케이션의 가장 치명적인 약점인 **"OWASP Top 10"** 취약점들을 하나씩 공략해보겠습니다. 실제 사이트에서 어떤 방식으로 해킹이 일어나는지 본격적인 공격 기법을 다룰 예정이니 기대해 주십시오.

9장. 웹 보안의 3대 핵심 요소: 인증, 인가, 입력 값 검증

지난 시간까지 웹의 통신 구조와 도구(Burp Suite)를 익혔다면, 이제는 웹 애플리케이션 보안을 지탱하는 **세 가지 기둥**을 세울 차례입니다. 아무리 튼튼한 성벽(방화벽)을 쌓아도, 문지기가 신분증을 확인하지 않거나(인증 실패), 아무에게나 열쇠를 주거나(인가 실패), 독이 든 음식(악성 입력값)을 들여보낸다면 그 성은 무너집니다.

---

1. 목적

- **지식 (Knowledge):** 인증(Authentication)과 인가(Authorization)의 명확한 차이를 구분하고, 세션(Session)과 쿠키(Cookie), JWT, OAuth 2.0의 작동 원리와 장단점을 이해한다.
- **기술 (Skills):** 세션 하이재킹(Session Hijacking), 무차별 대입(Brute Force), MFA 우회 등 인증 과정의 취약점과 디렉터리 트래버설(Directory Traversal) 등 입력 값 검증 부재로 인한 취약점의 발생 원리를 파악한다.
- **태도 (Attitude):** "사용자가 입력하는 모든 데이터는 오염될 수 있다"는 전제하에, 블랙리스트 방식보다는 화이트리스트 방식을 지향하는 안전한 검증 마인드를 갖춘다.

2. 사전지식

- **HTTP 프로토콜:** Stateless(상태를 저장하지 않음) 특성에 대한 이해.
- **웹 기초:** 클라이언트와 서버의 기본 통신 구조 (이전 강의 내용).

3. 들어가기 전에

여러분, "내가 누구인가?"를 증명하는 것과 "내가 무엇을 할 수 있는가?"는 완전히 다른 문제입니다. 신분증을 보여주는 것(인증)과, 그 신분증으로 통제 구역에 들어가는 것(인가)의 차이죠. 현업에서 발견되는 수많은 취약점은 복잡한 기술적 오류가 아니라, 이 기본적인 논리(Logic)가 꼬여서 발생합니다.

특히 최근에는 SNS 로그인(OAuth)이나 모바일 앱(JWT) 사용이 늘면서 인증 체계가 복잡해졌고, 해커들은 바로 이 '연결 고리'를 노립니다. 이번 시간에는 웹 보안의 근본이 되는 이 세 가지 요소가 실무에서 어떻게 구현되고, 또 어떻게 뚫리는지 공격자의 시선으로 분석해 보겠습니다.

4. 수업내용

4.1. 인증 (Authentication): 신원 확인의 관문

- **정의:** 사용자가 본인이 주장하는 사람(ID/PW 등)이 맞는지 시스템이 확인하는 과정입니다.
- **세션 관리의 필요성:** 웹(HTTP)은 상태를 저장하지 않는(Stateless) 특성이 있어, 요청 때마다 사용자를 식별하기 위해 쿠키(Cookie)와 세션(Session)을 사용합니다.
    1. **쿠키:** 사용자 브라우저에 저장되는 텍스트 파일. 변조 위험이 있습니다.
    2. **세션:** 서버 측에 정보를 저장하고, 클라이언트에는 '세션 ID'만 쿠키로 부여합니다. 보안성이 상대적으로 높습니다.
- **주요 인증 취약점:**
    1. **세션 하이재킹 (Session Hijacking):** 공격자가 사용자의 세션 ID(쿠키)를 탈취하여 해당 사용자로 위장하는 공격입니다. 주로 XSS나 Sniffing을 통해 이루어집니다.
    2. **무차별 대입 (Brute Force):** 가능한 모든 비밀번호 조합을 대입해보는 공격입니다. 비밀번호 복잡도가 낮을수록 위험합니다.
    3. **MFA 우회:** 이중 인증(지문 등)을 사용하더라도, 로직 설계 실수(예: 다른 기기에서 지문 재등록 허용)로 인해 우회될 수 있습니다.

4.2. 인가 (Authorization): 권한 부여의 규칙

- **정의:** 인증된 사용자에게 특정 리소스나 기능(파일 읽기, 쓰기 등)을 사용할 수 있는 권한을 부여하는 과정입니다.
- **RBAC (Role-Based Access Control):** '관리자', '일반 사용자' 등 역할(Role)에 따라 권한을 제어하는 방식이 주로 쓰입니다.
- **최신 인가 기술:**
    1. **JWT (JSON Web Token):**
        - **특징:** 사용자 정보를 토큰 자체에 담아 서명(Signature)한 뒤 전달합니다. 서버가 별도의 저장소(Stateful)를 두지 않아도 되므로 **Stateless**하고 확장성이 좋습니다.
        - **구조:** Header(헤더) + Payload(내용) + Signature(서명).
    2. **OAuth 2.0:**
        - **개념:** 비밀번호 공유 없이 타사 애플리케이션(예: 플렉스)에 내 계정 권한(예: 구글 캘린더)을 위임하는 개방형 표준입니다.
        - **구성 요소:** Resource Owner(사용자), Client(앱), Resource Server(API 서버), Authorization Server(인증 서버).
        - **프로세스:** 권한 요청 > 인증 코드 발급 > **Access Token** 교환 >API 이용.

4.3. 입력 값 검증 (Input Validation): 데이터의 무결성 확보

- **필요성:** 사용자 입력값은 절대 신뢰해서는 안 됩니다. 검증되지 않은 데이터는 SQL 인젝션, XSS 등의 공격 통로가 되며, 시스템 오류를 유발합니다.
- **취약점 예시 (Directory Traversal):** 파일명 입력값에 `../`(상위 디렉터리 이동) 문자를 포함시켜 시스템의 민감한 파일(`/etc/passwd` 등)을 열람하는 공격입니다.

- **검증 전략:**
    1. **블랙리스트 (Blacklist):** '위험한 것(SELECT, script 등)'만 막는 방식. 우회 가능성이 높아 권장되지 않습니다.
    2. **화이트리스트 (Whitelist):** '허용할 것(알파벳, 숫자 등)'만 정의하고 나머지는 모두 차단하는 방식. 보안상 훨씬 안전합니다.
- **필터링 기법:** 정규 표현식(Regular Expression)을 사용하거나, 데이터 타입/길이를 엄격하게 제한하여 구현합니다.

5. 핵심내용 (Summary)

1. **인증 vs 인가:** 인증은 "누구인가(Who)", 인가는 "무엇을 할 수 있는가(What)"를 다룹니다.
2. **세션 보안:** 세션 ID는 제2의 비밀번호입니다. 탈취당하면(Session Hijacking) 계정이 통째로 넘어갑니다.
3. **최신 트렌드:** 모바일과 MSA 환경에서는 서버 부하를 줄이기 위해 세션 대신 **JWT**를, 소셜 로그인 등을 위해 **OAuth 2.0**을 적극 활용합니다.
4. **입력 검증의 철칙:** 모든 입력값은 의심해야 하며, 보안을 위해서는 블랙리스트보다 **화이트리스트** 방식의 검증을 적용해야 합니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 클라이언트의 상태 정보를 서버에 저장하지 않고, 토큰 자체에 정보를 담아 서명하여 검증하는 'Stateless'한 인증 방식의 토큰 표준은 무엇인가요?
    -
- **Q2.** 사용자가 비밀번호를 제공하지 않고도, 구글이나 페이스북 같은 서비스의 자원 접근 권한을 다른 애플리케이션에 위임할 수 있게 해주는 표준 프로토콜은 무엇인가요?
    -
- **Q3.** 입력 값 검증 방식 중, 허용할 안전한 리스트(예: 숫자만 허용)를 미리 정의하고 그 외의 모든 입력을 차단하는 보안성이 높은 방식은 무엇인가요?
    -

7. 다음 장 소개

웹 보안의 기초 체력을 길렀으니, 이제는 실전 공격 기술을 배울 차례입니다. 다음 시간에는 웹 애플리케이션 보안 취약점의 표준이자 해커들의 교과서인 "OWASP Top 10"에 대해 알아보겠습니다. 실제 현장에서 가장 많이 발생하고 위험한 10가지 취약점을 하나씩 파헤쳐 보겠습니다.

10장. OWASP Top 10 주요 취약점의 이해와 공격 원리

지난 시간까지 웹의 기본 구조와 통신 원리를 익히며 기초 체력을 다졌습니다. 이제는 실제 공격 기술, 즉 '무기'를 다룰 차례입니다. 웹 애플리케이션 보안의 바이블이라 불리는 **OWASP Top 10**의 핵심 취약점들을 상세히 분석하고, 공격자가 이를 어떻게 악용하는지 그 메커니즘을 파헤쳐 보겠습니다.

---

1. 목적

- **지식 (Knowledge):** IDOR, XSS, CSRF, SQL Injection, File Inclusion(LFI/RFI), SSRF 등 주요 웹 취약점의 정의와 발생 원리, 그리고 공격 유형을 정확히 이해한다.
- **기술 (Skills):** 파라미터 변조(IDOR), 스크립트 삽입(XSS), 쿼리 조작(SQLI), 경로 조작(LFI) 등의 공격 벡터를 식별하고, 각 취약점이 발생하는 코드 패턴과 시나리오를 구상할 수 있다.
- **태도 (Attitude):** 단순한 호기심이 아닌, 시스템에 치명적인 영향을 줄 수 있는 취약점의 위험성을 인지하고, 이를 방어하기 위한 개발자의 관점까지 고려하는 전문가적 자세를 갖춘다.

2. 사전지식

- **웹 통신 기초:** HTTP 요청/응답 구조 및 파라미터(Parameter) 전달 방식.
- **기본 프로그래밍:** HTML, JavaScript, SQL 구문에 대한 기초적인 독해 능력.

3. 들어가기 전에

여러분, 혹시 남의 택배 배송 조회를 하다가 URL 끝의 숫자만 바꿨는데 다른 사람의 주소가 나온 적이 있나요? 아주 단순한 실수 같지만, 이것이 바로 IDOR(부적절한 접근 제어)라는 치명적인 취약점입니다.

이번 시간에는 해커들이 가장 많이 노리고, 또 가장 빈번하게 발견되는 웹 취약점들을 다룹니다. "내가 짠 코드는 안전하겠지"라는 방심이 어떻게 시스템을 무너뜨리는지, 공격자의 시선에서 6가지 핵심 취약점을 낱낱이 해부해 보겠습니다.

4. 수업내용

4.1. IDOR (부적절한 직접 객체 참조)

- **정의:** 사용자가 입력한 파라미터(ID 등)가 서버의 데이터베이스 키나 파일 등 내부 객체를 직접 참조할 때, 권한 검증 없이 이를 처리하여 발생하는 취약점입니다. 주로 접근 제어(Access Control) 실패와 관련 있습니다.
- **공격 유형:**
  - **수평적 권한 상승 (Horizontal):** 나와 동일한 권한을 가진 다른 사용자의 데이터(예: 1001번 유저가 1003번 유저 정보 열람)를 탈취.
  - **수직적 권한 상승 (Vertical):** 일반 사용자가 관리자 권한의 기능을 수행.
- **트리거 포인트:** 숫자형 ID 변조가 가장 흔하지만, 사용자 이름, 파일명 변경, HTTP 메서드(GET/POST) 변경, 콘텐츠 타입(JSON/XML) 변경 등으로도 발생할 수 있습니다.

4.2. XSS (크로스 사이트 스크립팅)

- **정의:** 공격자가 웹 페이지에 악성 스크립트(JS)를 삽입하여, 이를 열람하는 희생자의 브라우저에서 실행되게 만드는 공격입니다. 정보 탈취나 세션 하이재킹에 주로 쓰입니다.
- **주요 3가지 유형:**
  1. **Reflected XSS (반사형):** 악성 스크립트가 포함된 URL을 사용자가 클릭하면, 서버가 이를 반사하여 브라우저에서 실행됩니다. (일회성).
  2. **Stored XSS (저장형):** 게시판 등에 악성 스크립트를 작성하여 서버 DB에 저장하고, 해당 글을 읽는 모든 사용자에게 피해를 줍니다. 파급력이 가장 큽니다.
  3. **DOM-based XSS:** 서버를 거치지 않고, 브라우저 내의 DOM 객체(예: `location.hash`)를 처리하는 과정에서 발생하는 클라이언트 측 취약점입니다. 서버 로그에 남지 않아 탐지가 어렵습니다.

4.3. CSRF (사이트 간 요청 위조)

- **정의:** 사용자가 자신의 의지와 무관하게 공격자가 의도한 행위(비밀번호 변경, 계좌 이체 등)를 웹사이트에 요청하게 만드는 공격입니다.
- **메커니즘:** 사용자가 이미 로그인된 상태(세션/쿠키 보유)라는 점을 악용합니다. 해커가 보낸 링크나 이미지를 클릭하는 순간, 사용자 브라우저의 쿠키를 이용해 몰래 요청이 전송됩니다.

4.4. SQL Injection (SQL 삽입)

- **정의:** 입력값 검증 부재를 틈타 SQL 쿼리문의 일부를 조작하여, DB 정보를 탈취하거나 조작하는 공격입니다.
- **공격 기법 (6가지):**
  1. **Boolean-based Blind:** 참/거짓 결과에 따라 데이터를 추측.
  2. **Error-based:** 일부러 에러를 유발하여 에러 메시지에 포함된 DB 정보를 획득.
  3. **Union-based:** `UNION` 연산자를 이용해 다른 테이블의 정보를 합쳐서 조회.
  4. **Stacked Query:** 세미콜론(;)으로 쿼리를 닫고 새로운 악성 쿼리를 추가 실행.
  5. **Time-based Blind:** `sleep()` 등을 이용해 응답 시간 지연 여부로 참/거짓을 판별.
  6. **Inline Query:** 쿼리 내부에 또 다른 쿼리를 삽입-.
- **팁:** 입력값에 `-1`을 했을 때 계산된 결과가 나온다면, 해당 값이 정수형으로 연산되고 있음을 유추할 수 있습니다.

4.5. File Inclusion (파일 포함 공격)

- **LFI (Local File Inclusion):** 서버 내부의 로컬 파일(`/etc/passwd` 등)을 읽어오거나 실행합니다.
  - ***RCE 연계:*** 파일 업로드 기능으로 웹쉘을 올리거나, 서버 로그(Log Poisoning)에 PHP 코드를 심은 뒤 LFI로 불러와 실행시킬 수 있습니다.
- **RFI (Remote File Inclusion):** 외부 서버에 있는 악성 파일(웹쉘 등)을 원격으로 불러와 서버에서 실행시킵니다. 공격자가 자신의 서버에 `shell.txt`를 두고 타겟 서버가 이를 include 하게 만듭니다.

4.6. SSRF (서버 사이드 요청 위조)

- **정의:** 공격자가 서버를 조종하여 서버가 내부 시스템(Internal Network)이나 외부 시스템에 요청을 보내게 만드는 취약점입니다.
- **위험성:** 방화벽으로 보호받는 내부망 스캔, 클라우드 환경(AWS 등)의 메타데이터 서비스(`169.254.169.254`)에 접근하여 자격 증명(Credential)을 탈취하는 데 주로 악용됩니다.

5. 핵심내용 (Summary)

1. **IDOR의 단순함과 위험성:** 복잡한 해킹 기술 없이 파라미터 변조만으로 타인의 정보를 볼 수 있는, 가장 빈번하고 파괴적인 취약점입니다.
2. **XSS와 CSRF의 차이:** XSS는 '스크립트 실행'을 통해 정보를 탈취하고, CSRF는 '사용자의 권한'을 도용하여 원치 않는 행위를 시킵니다.
3. **Injection 계열의 위협:** SQL Injection은 DB를, Command Injection이나 File Inclusion은 서버 자체를 장악할 수 있는 치명적인 공격입니다.
4. **SSRF와 클라우드:** 최근 클라우드 도입이 늘면서, 서버를 경유해 내부 클라우드 자원에 접근하는 SSRF 공격의 중요도가 매우 높아졌습니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 게시판 등에 악성 스크립트를 작성하여 서버 DB에 저장하고, 해당 글을 열람하는 다수의 사용자에게 스크립트가 실행되게 만드는 XSS 유형은 무엇인가요?
  -
- **Q2.** 공격자가 웹 애플리케이션의 로직을 악용하여, 서버가 방화벽 내부의 시스템이나 클라우드 메타데이터 서비스 등 공격자가 직접 접근할 수 없는 곳으로 요청을 보내도록 만드는 취약점은 무엇인가요?
  -
- **Q3. 사용자가 입력한 파라미터(예:** user_id=1234)를 서버가 적절한 검증 없이 내부 객체의 키값으로 사용하여, 타인의 정보에 접근할 수 있게 되는 취약점을 무엇이라고 합니까?
  -

7. 다음 장 소개

주요 취약점들의 공격 메커니즘을 확실히 이해하셨나요? 다음 시간에는 이번에 배운 취약점들이 실제 비즈니스 환경에서 **어떤 영향(Impact)과 위협**을 초래하는지, 그리고 이를 바탕으로 어떻게 효과적인 시나리오를 구성할 수 있는지 심화 내용을 다루도록 하겠습니다.

11장. 시나리오 개발과 취약점 체이닝

앞선 강의들에서 개별적인 취약점(XSS, SQL Injection 등)의 원리와 공격법을 익혔습니다. 하지만 현업에서 "여기 XSS가 있어요"라고 제보하면 "그래서요? 피해가 없네요(Informative)"라며 거절당하기 일쑤입니다.

진정한 해커는 하나의 작은 틈새를 비집고 들어가 시스템 전체를 장악하는 '시나리오'를 만듭니다. 이번 시간에는 버그헌팅의 꽃이라 불리는 **'취약점 체이닝(Vulnerability Chaining)'** 기술과, 단순한 버그를 치명적인 위협으로 바꾸는 **'스토리텔링'** 전략을 전수해 드리겠습니다.

---

1. 목적

- **지식 (Knowledge):** 단일 취약점의 한계를 극복하기 위해 두 개 이상의 취약점을 연결하는 체이닝(Chaining)의 개념과, 낮은 위협도(Low Severity)의 버그(예: Self-XSS)가 어떻게 고위험(Critical)으로 발전할 수 있는지 이해한다.
- **기술 (Skills):** 발견한 취약점(XSS 등)이 보안 설정(HttpOnly)으로 막혔을 때, 정보 노출 등 다른 취약점과 연계하여 우회 시나리오를 설계하고, 실질적인 피해(Impact)를 증명하는 PoC(Proof of Concept)를 구성할 수 있다.
- **태도 (Attitude):** 단순히 "취약점이 존재한다"는 사실 전달에 그치지 않고, "이것이 비즈니스에 어떤 피해를 주는가"를 설득력 있게 제시하려는 **스토리텔링(Storytelling)** 마인드를 갖춘다.

2. 사전지식

- **기본 웹 취약점:** XSS, CSRF, IDOR 등의 기본 공격 원리 (5장~10장 내용).
- **웹 보안 메커니즘:** HttpOnly 쿠키 옵션, Same Origin Policy 등 기본적인 방어 기제에 대한 이해.

3. 들어가기 전에

여러분, 튼튼한 금고가 있는데 열쇠 구멍에 작은 흠집(Low 취약점)이 있다고 해서 은행이 돈을 줄까요? 절대 아닙니다. 하지만 그 흠집을 이용해 경비원의 시선을 돌리고(1단계), 그 사이에 복제한 열쇠(2단계)로 문을 연다면 이야기가 달라집니다.

버그헌팅은 기술적인 발견을 넘어선 '스토리텔링'입니다. 기업 보안 담당자에게 "이 버그는 당신의 회사를 망하게 할 수 있습니다"라고 납득시켜야 합니다. 오늘은 별것 아닌 것처럼 보이는 'Self-XSS'나 '정보 노출' 조각들을 모아, 계정 탈취(Account Takeover)나 원격 코드 실행(RCE)이라는 거대한 퍼즐을 완성하는 상위 1% 해커들의 전략을 배워보겠습니다.

4. 수업내용

4.1. 시나리오 개발의 중요성: 버그헌팅은 스토리텔링이다

- **단순 제보의 한계:** "스크립트가 실행됩니다" 정도의 보고서는 낮은 평가를 받거나 거절될 수 있습니다. 담당자를 설득하려면 발생 가능한 결과(Impact)를 보여줘야 합니다.
- **상황 예시 (XSS와 HttpOnly):**
  - **상황:** XSS 취약점을 찾았으나, 세션 쿠키에 **HttpOnly** 옵션이 설정되어 있어 `document.cookie`로 세션을 탈취할 수 없는 경우.
  - **하수:** "XSS가 있습니다"라고 제보 $\rightarrow$ Low 또는 Medium 등급.
  - **고수 (시나리오 개발):** 사이트를 더 뒤져서 개발용 페이지나 에러 페이지에서 **세션 정보가 평문으로 노출**되는 곳을 찾아냅니다. 그 후 XSS를 이용해 해당 페이지를 비동기(AJAX/XHR)로 호출하고, 응답값에서 세션을 파싱해 공격자 서버로 전송하는 코드를 짭니다.
  - **결과:** 단순 스크립트 실행이 **'세션 탈취 및 계정 장악'** 시나리오로 격상되어 High~Critical 포상금을 받습니다.

4.2. 취약점 체이닝 (Vulnerability Chaining)의 개념

- **정의:** 두 개 이상의 상대적으로 중요하지 않은 보안 이슈를 결합하여 강력한 공격 시나리오를 구축하는 기법입니다.
- **효과:** 단일 취약점으로는 제한적인 권한만 얻을 수 있지만, 체이닝을 하면 그 효과가 기하급수적으로 증가합니다.
- **이론적 체이닝 예시 (File Write + LFI):**
  1. **Arbitrary File Write:** 서버에 파일을 쓸 수 있지만, 방화벽 때문에 실행하거나 접근할 수 없는 상태.
  2. **LFI (Local File Inclusion):** 파일을 실행할 수 있지만, 업로드할 방법이 없는 상태.
  3. **Chain:** 파일 쓰기 취약점으로 악성 PHP 코드를 서버에 심고, LFI 취약점을 이용해 그 파일을 불러와 실행(RCE)시킵니다.

4.3. Self-XSS의 이해와 한계

- **정의:** 공격자가 아닌 피해자 본인의 브라우저 콘솔 등에 스크립트를 직접 입력해야만 실행되는 XSS입니다.
- **한계:** 피해자가 스스로 공격 코드를 입력해야 하므로, 이는 기술적 취약점이라기보다 피싱(Social Engineering)에 가깝습니다. 따라서 대부분의 버그바운티 프로그램에서 포상 제외 대상(Out of Scope)입니다.
- **반전:** 하지만 이 Self-XSS도 다른 취약점과 결합되면 강력한 무기가 됩니다.

4.4. 실전 체이닝 사례: 우버(Uber) 계정 탈취 (2016)

본질적으로 위험도가 낮은 두 취약점을 연결하여 치명적인 결과를 만든 유명한 사례입니다.

- **취약점 1: Self-XSS**
  1. 우버 모바일 사이트(m.uber.com) 프로필 수정 기능에 XSS가 있었으나, 본인이 로그인했을 때만 실행되어 위협이 되지 않았습니다.
- **취약점 2: 헤더 조작 (Logout/Login CSRF 유사)**

    1. 로그아웃 등의 기능에서 파라미터를 조작해 응답 헤더(`Set-Cookie`)를 변조할 수 있는 취약점 발견. 공격자가 피해자의 브라우저 쿠키를 강제로 공격자의 세션으로 덮어씌울 수 있었습니다.
- **Chaining Scenario (계정 탈취):**
    1. 공격자는 자신의 계정 프로필에 XSS 공격 코드(피해자의 OAuth 토큰을 훔치는 코드)를 심어놓습니다.
    2. 피해자에게 헤더 조작 취약점이 있는 URL을 보냅니다.
    3. 피해자가 클릭하면 `Set-Cookie` 헤더에 의해 **강제로 공격자의 계정으로 로그인**됩니다.
    4. 로그인되자마자 공격자가 심어둔 XSS 코드가 피해자 브라우저에서 실행됩니다.
    5. 이 XSS 코드는 피해자의 원래 쿠키 저장소 등에 남아있던 민감한 **OAuth 토큰** 등을 탈취하여 공격자에게 전송합니다.
- **결과:** 단순 버그 2개를 엮어 **원클릭 계정 탈취**를 증명했습니다.

5. 핵심내용 (Summary)

1. **임팩트(Impact) 중심:** 버그헌팅의 핵심은 "문제가 있다"는 통보가 아니라, "이 문제로 **어떤 피해**를 입힐 수 있다"는 증명입니다. 시나리오가 포상금의 액수를 결정합니다.
2. **취약점 체이닝:** 낮은 등급의 취약점(Self-XSS, Open Redirect, Info Leak 등)들을 버리지 말고 기록해 두었다가, 서로 연결하여 **Critical**한 공격 벡터를 만들어야 합니다.
3. **시스템 이해:** 체이닝을 위해서는 시스템의 비즈니스 로직과 데이터 흐름을 완벽히 이해하고, "내가 공격자라면 어떻게 이 서비스를 무너뜨릴까?"라고 끊임없이 질문해야 합니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 단일 취약점으로는 위협이 낮지만, 두 개 이상의 보안 이슈를 연결하여 강력한 공격 시나리오를 만드는 기법을 무엇이라고 합니까?
    -
- **Q2.** 사용자가 스스로 브라우저 콘솔에 악성 코드를 입력하게 유도해야 실행되는 XSS 유형으로, 단독으로는 포상 대상에서 제외되는 경우가 많은 취약점은?
    -
- **Q3.** XSS 공격 시 자바스크립트로 세션 쿠키(document.cookie)에 접근하지 못하도록 막는 보안 설정 옵션은 무엇입니까? (체이닝을 통해 이를 우회하는 시나리오가 필요함)
    -

7. 다음 장 소개

이제 우리는 기술적 공격법과 이를 엮어내는 전략까지 모두 익혔습니다. 하지만 아무리 훌륭한 해킹을 했어도, 이를 보고서로 잘 표현하지 못하면 포상금을 받을 수 없습니다.

다음 시간에는 여러분의 노력이 헛되지 않도록, "버그헌팅 리포트 작성 전략"을 다루겠습니다. 플랫폼별 권장 포맷, 마크다운(Markdown) 작성법, 그리고 심사관(Triage)이 감탄하는 '좋은 리포트'의 조건을 예시와 함께 상세히 알려드리겠습니다. 이것이 초급 과정의 마지막 관문입니다.

12장. 웹 취약점 버그헌팅 실제 사례 분석 및 실습

이론으로 배운 취약점이 실제 현장에서는 어떻게 발견되고, 또 어떻게 치명적인 공격으로 이어지는지 궁금하셨을 겁니다. 이번 12장에서는 전 세계를 놀라게 했던 실제 버그헌팅 사례(YouTube, Sony 등)를 통해 해커의 사고방식을 훔쳐보고, 자동화 도구를 활용한 실습의 기초를 다져보겠습니다. "설마 이런 곳에 취약점이 있겠어?"라는 방심이 어떻게 뚫리는지 생생하게 보여드리겠습니다.

---

1. 목적

- **지식 (Knowledge):** 글로벌 기업(Google, Sony)과 오픈소스(Gnuboard)에서 발견된 주요 취약점(IDOR, XSS, SQL Injection)의 발생 원리와 공격 시나리오를 이해한다.
- **기술 (Skills):** 취약점 발굴을 위해 서브도메인 정찰 도구(Amass, Sublist3r)를 활용하고, 일반적인 파라미터 외에 **HTTP 헤더(User-Agent)** 등을 변조하여 우회 공격을 시도하는 기술을 익힌다.
- **태도 (Attitude):** "대기업 서비스는 안전할 것이다"라는 고정관념을 버리고, 서드파티(Third-party) 연동 구간이나 로그 수집 구간 등 개발자가 간과하기 쉬운 '사각지대'를 집요하게 파고드는 창의적인 자세를 갖춘다.

2. 사전지식

- **OWASP Top 10:** XSS, SQL Injection, IDOR의 기본 개념 (이전 강의 내용).
- **HTTP 프로토콜:** POST 요청 구조와 User-Agent 헤더의 역할.

3. 들어가기 전에

여러분, 구글의 유튜브(YouTube) 비공개 영상은 절대 볼 수 없을까요? 소니(Sony) 같은 대기업 서버는 난공불락일까요? 정답은 '아니오'입니다.

2019년, 한 해커는 유튜브의 '광고 설정' 기능을 이용해 남의 비공개 영상을 훔쳐보는 기발한 방법을 찾아냈습니다. 또한, 소니 사이트에서는 단순한 SQL 인젝션을 원격 코드 실행(RCE)까지 발전시켜 서버를 장악한 사례도 있습니다.

이들의 공통점은 남들이 보지 않는 곳, 즉 '틈(Gap)'을 노렸다는 것입니다. 이번 강의에서는 이 전설적인 사례들을 기술적으로 낱낱이 해부하여, 여러분의 해킹 감각을 한 단계 끌어올려 드리겠습니다.

4. 수업내용

4.1. 버그헌팅 통계와 트렌드

- **지속적인 위협 XSS:** 보안 기술이 발전하고 프레임워크단에서 방어 로직을 갖추었음에도 불구하고, XSS(교차 사이트 스크립팅)는 여전히 가장 많이 제보되고 발견되는 취약점입니다. 해커들이 필터링을 우회하는 새로운 기법을 계속 찾아내기 때문입니다.
- **권한 관리의 허점:** IDOR(부적절한 직접 객체 참조)와 같은 인증 및 접근 통제 관련 취약점 또한 빈번하게 보고되며, 보상 규모도 증가하는 추세입니다.

4.2. IDOR 사례: 유튜브 비공개 영상 탈취 (Google)

- **배경:** 유튜브에는 '비공개(Private)' 영상 설정이 있어 본인 외에는 볼 수 없어야 합니다.
- **발견 지점:** 유튜브 자체가 아닌 연동된 **'구글 애드(Google Ads)'** 서비스에서 취약점이 발견되었습니다. 유튜버가 광고 삽입 시점을 설정하는 '마크 모먼트(Mark Moment)' 기능이 문제였습니다.
- **공격 원리:**
    1. '마크 모먼트' 버튼을 누르면 `GetThumbnails`라는 POST 요청이 전송됩니다.
    2. 이 요청의 Body에는 **영상 고유 ID**와 타임스탬프(밀리초)가 포함됩니다.
    3. 서버는 해당 시점의 썸네일 이미지를 Base64로 인코딩하여 응답합니다.
    4. **Exploit:** 해커는 타겟 비공개 영상의 ID를 알아낸 뒤, 타임스탬프를 **33ms(프레임 간격)** 단위로 증가시키며 반복 요청을 보냈습니다. 이렇게 수집한 썸네일들을 이어 붙여 **비공개 영상을 복원**하는 데 성공했습니다.
- **교훈:** 메인 서비스가 안전하더라도, 서드파티(연동 서비스)에서 발생하는 논리적 허점을 통해 우회 공격이 가능합니다.

4.3. XSS 사례: 그누보드 (국내 오픈소스 CMS)

- **유형:** Reflected XSS (2023년 4월 패치 사례).
- **발생 원인:** `bbs/search.php` 페이지에서 `wr_table` 파라미터에 대한 검증이 미흡했습니다.
- **코드 분석:**
    - 사용자가 입력한 `wr_table` 값이 별도 필터링 없이 변수에 저장됩니다.
    - 이 변수가 `get_paging` 함수의 인자로 전달되고, 다시 HTML `<a>` 태그 내부에 그대로 삽입되어 출력되면서 스크립트가 실행됩니다.

- **패치:** 해당 파라미터에 정규식(`preg_replace`)을 적용하여 특수문자가 올 수 없도록 제한하여 해결했습니다.

4.4. SQL Injection 사례: 소니 (Sony) RCE 연계

- **정찰(Recon)의 중요성:** 제보자는 처음에 `Sublist3r`를 썼으나 유효한 도메인을 못 찾았고, `Amass`라는 도구를 사용하여 구글 검색에도 나오지 않는 숨겨진 서브도메인을 발견했습니다.
- **취약점 탐지:**
  1. 로그인 페이지에서 `username` 파라미터에 작은따옴표(')를 넣었더니 500 에러와 함께 SQL 구문 오류가 노출되었습니다 (DB가 MS-SQL임이 확인됨).
  2. 하지만 `username` 파라미터 자체는 필터링이 되어 있어 공격이 불가능했습니다.
- **우회 및 에스컬레이션 (User-Agent):**
  1. 제보자는 개발자들이 보통 **HTTP 헤더**에 대한 검증은 소홀하다는 점을 노렸습니다.
  2. `User-Agent` 헤더 값에 SQL 인젝션 페이로드를 삽입하자 공격이 성공했습니다.
  3. **Time-based SQLi** (WAITFOR DELAY)로 취약점을 확인한 후, **Stacked Query** 기법(세미콜론으로 쿼리 연결)을 이용해 `xp_cmdshell`(시스템 명령 실행)을 활성화했습니다.
  4. 결과적으로 단순 SQL 인젝션을 넘어 서버를 장악하는 RCE(원격 코드 실행)까지 성공했습니다.

5. 핵심내용 (Summary)

1. **서드파티의 위험성:** 유튜브 사례처럼 본체(Main Service)가 튼튼해도 연동된 서비스(Google Ads)의 취약점을 통해 데이터가 유출될 수 있습니다. 창의적인 공격 벡터 탐색이 중요합니다.
2. **헤더 인젝션:** 입력 폼(Input field)뿐만 아니라, **User-Agent, Cookie, Referer** 등 보이지 않는 HTTP 헤더 값도 서버의 DB나 로직에서 처리될 수 있으므로 반드시 점검해야 합니다 (소니 사례).
3. **도구의 적절한 활용:** 정찰 단계(Recon)에서 `Amass` 같은 도구를 활용해 숨겨진 자산(Subdomain)을 찾는 것이 버그헌팅의 시작점입니다.
4. **Impact 극대화:** 단순한 에러 발견에 그치지 않고, 데이터를 재조립하거나(영상 복원), 권한을 상승시켜(RCE) 파급력(Impact)을 증명해야 높은 포상금을 받을 수 있습니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 유튜브 비공개 영상 탈취 사례에서, 해커가 영상을 프레임 단위로 가져오기 위해 공략했던 연동 서비스의 기능은 무엇인가요?
  -
- **Q2.** 소니(Sony) 해킹 사례에서, 사용자의 입력값(ID/PW) 필터링을 우회하기 위해 공격자가 SQL 인젝션 코드를 삽입한 HTTP 헤더의 이름은 무엇인가요?
  -
- **Q3.** 소니 사례에서 초기 정찰(Recon) 시 `Sublist3r`로는 찾지 못한 숨겨진 서브도메인을 발견하게 해 준 오픈소스 정보 수집 도구의 이름은 무엇인가요?
  -

7. 다음 장 소개

실제 사례들을 보니 가슴이 뛰지 않으십니까? 다음 시간에는 취약점의 공식 명칭이자 훈장과도 같은 "CVE(Common Vulnerabilities and Exposures)의 이해"에 대해 알아보겠습니다. CVE가 무엇인지, 어떻게 발급되고 활용되는지, 그리고 이를 이용해 어떻게 더 효율적으로 취약점을 찾을 수 있는지 깊이 있게 다뤄보겠습니다.

13장. 웹 취약점 버그헌팅 실제 사례 분석

앞선 강의들에서 각 취약점의 이론적인 원리를 배웠습니다. 하지만 이론과 실전은 다릅니다. 실제 기업 환경은 방화벽(WAF), 필터링 로직 등 다양한 방어 체계가 존재하기 때문입니다. 이번 13장에서는 전 세계를 놀라게 했던 실제 버그헌팅 사례(Sony, GitLab, Dropbox)를 통해, 상위 1% 해커들이 어떻게 방어벽을 우회하고 치명적인 공격(RCE)으로 연결했는지 그 '생각의 흐름'을 추적해 보겠습니다.

---

1. 목적

- **지식 (Knowledge):** MSSQL의 `xp_cmdshell`, GitLab의 API 구조, AWS 메타데이터 서비스 등 특정 환경에서 발생할 수 있는 심화 공격 벡터를 이해한다.
- **기술 (Skills):** 단순 SQL Injection을 RCE(원격 코드 실행)로 확장하는 기법, 경로 조작(Path Traversal)을 통한 시스템 파일 덮어쓰기, 리다이렉션(Redirection)을 이용한 SSRF 필터링 우회 기술을 습득한다.
- **태도 (Attitude):** 취약점을 발견했을 때 단순 제보에 그치지 않고, "어떻게 하면 더 큰 영향력(Impact)을 증명할 수 있을까?"를 고민하며 끝까지 파고드는 끈기(Persistence)와 창의적 우회 전략을 수립하는 자세를 갖춘다.

2. 사전지식

- **SQL Injection 기초:** Time-based Blind SQLi의 개념.
- **시스템/네트워크 기초:** SSH 키 인증 방식, AWS 클라우드 메타데이터 IP(`169.254.169.254`)의 역할.
- **HTTP 상태 코드:** 리다이렉션(3xx)의 동작 원리.

3. 들어가기 전에

여러분, 소니(Sony)나 드롭박스(Dropbox) 같은 글로벌 IT 기업들은 보안이 매우 강력할 것 같죠? 하지만 해커들은 아주 작은 틈새를 찾아내어 기어이 서버를 장악해냅니다.

단순히 SQL 문법 에러를 찾는 것은 누구나 할 수 있습니다. 하지만 진정한 고수는 그 에러를 통해 방화벽이 차단하는 특정 단어를 우회하고, 결국 시스템 명령어를 실행시켜 서버의 주인 행세를 합니다. 이번 강의에서는 **Sony의 SQL Injection이 RCE로 변하는 과정**, **GitLab의 SSH 키 탈취**, **Dropbox의 SSRF 방어 우회**라는 세 가지 전설적인 사례를 통해 '해커의 집요함'이 무엇인지 보여드리겠습니다.

4. 수업내용

4.1. [Sony] SQL Injection을 통한 RCE (Time-based Blind & WAF 우회)

- **탐지 (Detection):**
  - 로그인 페이지에서 `username` 파라미터에 작은따옴표(')와 주석(`--`)을 입력했을 때, MSSQL 특유의 주석 처리 방식(공백 불필요)과 에러 메시지를 통해 SQL 인젝션 가능성을 확인했습니다.
  - **이후** `WAITFOR DELAY '00:00:05'` 구문을 주입하여 응답이 5초 지연되는 것을 통해 **Time-based SQL Injection**을 확정했습니다.
- **확장 (Escalation to RCE):**
  - 단순 데이터 유출에 만족하지 않고, 시스템 명령 실행을 위해 MSSQL의 **`xp_cmdshell`** 저장 프로시저를 활성화했습니다 (`sp_configure` 사용).
  - `ping` 명령으로 외부 연결을 확인한 뒤, `whoami` 결과를 파라미터에 담아 외부 서버로 전송하는 방식으로 RCE(원격 코드 실행)를 증명했습니다.
- **우회 (Bypass WAF):**

- 패치 후 `EXEC xp_cmdshell` 문자열이 차단되자, 해커는 `DECLARE @x`로 변수를 선언하여 문자열을 분리한 뒤 실행하는 방식으로 방화벽을 다시 뚫었습니다.

4.2. [GitLab] Path Traversal을 통한 RCE (SSH Key Overwrite)

- **취약점 배경:** GitLab 엔터프라이즈 버전의 '패키지 레지스트리' 기능에서 발생했습니다.
- **공격 원리:**
  - 공격자는 `PUT` 요청을 통해 Maven 패키지를 업로드할 때, URL 경로에 `../../`와 같은 상위 디렉터리 이동 문자를 삽입했습니다.
  - **코드 분석:** GitLab이 사용하는 프레임워크(Grape)는 `file_name` 파라미터에 대해서만 정규식 검사를 수행하고, 실제 경로인 `path` 부분은 검증하지 않는다는 허점을 이용했습니다-.
- **임팩트 (Impact):**
  - 공격자는 자신의 SSH 공개키(`id_rsa.pub`)를 서버의 `authorized_keys` 파일 위치로 덮어씌워 업로드했습니다.
  - 이로 인해 비밀번호 없이 SSH로 서버에 접속하여 시스템을 장악(RCE)하는 데 성공했습니다.

4.3. [Dropbox] SSRF 방어 우회 (HTTP Redirect 기법)

- **탐지 (Detection):**
  - Dropbox의 문서 불러오기 기능(HelloSign)에서 외부 클라우드(Google Drive 등)를 연동하는 과정에서 취약점을 찾았습니다.
  - 단순히 외부 URL을 입력했을 때는 차단(404)되었으나, 파라미터를 조작(D -> O)하여 방어 로직의 허점을 찾았습니다.
- **목표 설정 (Targeting AWS):**
  - 해당 서비스가 AWS를 사용함을 확인하고, AWS EC2 인스턴스의 메타데이터 서비스(`169.254.169.254`)에 접근하여 **IAM 자격 증명**을 탈취하려 시도했습니다.
- **우회 (Redirect Bypass):**
  - 직접적인 내부 IP 호출이 차단되자, 공격자는 자신의 서버에 악성 스크립트를 호스팅했습니다.
  - Dropbox 서버가 공격자의 서버에 접속하면, 공격자 서버는 **HTTP 303 Redirect** 응답을 보내 목적지를 `169.254.169.254`로 변경시켰습니다.
  - 결국 Dropbox 서버는 리다이렉션을 따라 스스로 내부 메타데이터 서비스에 접속하고 그 결과를 공격자에게 반환했습니다.

5. 핵심내용 (Summary)

1. **임팩트의 극대화:** 세 사례 모두 단순한 버그 발견에 그치지 않고, 시스템 장악(RCE)이나 클라우드 키 탈취와 같은 최상위 위험(Critical)으로 연결했습니다.
2. **창의적 우회:** 방화벽이나 필터링이 있다고 포기하지 않았습니다. 문자열을 변수에 담아 실행하거나(Sony), 리다이렉션을 이용해 서버를 속이는(Dropbox) 등 기발한 방법으로 보안 장치를 무력화했습니다.
3. **깊이 있는 분석:** GitLab 사례처럼 소스 코드나 프레임워크의 동작 방식을 깊이 이해해야만 발견할 수 있는 논리적 허점(Logic Flaw)들이 존재합니다.

4. **끈기(Persistence):** 예상한 결과가 나오지 않아도 다른 파라미터를 조작하거나 우회 방법을 끝까지 연구하는 자세가 버그헌팅 성공의 열쇠입니다.

6. 마무리 키워드 퀴즈

오늘 배운 실제 사례들을 되짚어 봅시다.

- **Q1.** Sony 해킹 사례에서, MSSQL 데이터베이스 환경에서 시스템 운영체제 명령어를 실행하기 위해 공격자가 활성화하고 사용한 저장 프로시저의 이름은 무엇인가요?
    -
- **Q2.** GitLab 사례에서 공격자는 Path Traversal 취약점을 이용하여 자신의 SSH 공개키를 서버의 특정 파일에 덮어씌웠습니다. SSH 접속 시 인증된 키 목록을 담고 있는 이 파일의 이름은 무엇인가요?
    -
- **Q3.** Dropbox SSRF 사례에서, 공격자가 내부망 IP(169.254...) 직접 호출이 차단되자 이를 우회하기 위해 자신의 서버에서 사용한 HTTP 응답 헤더 기술은 무엇인가요?
    -

7. 다음 장 소개

실제 해커들의 공격 시나리오가 얼마나 치밀한지 확인하셨나요? 다음 시간에는 이러한 취약점들에 부여되는 고유 식별 번호인 "CVE(Common Vulnerabilities and Exposures)의 이해"에 대해 알아보겠습니다. CVE가 무엇인지, 어떻게 검색하고 분석하여 버그헌팅에 활용할 수 있는지 체계적으로 다뤄보겠습니다.

14장. 버그헌팅 자동화 도구 활용 실습: SQLMap, JWT Crack, Hashcat

앞선 강의들에서 다양한 웹 취약점의 원리와 실제 사례를 깊이 있게 다루었습니다. 이제는 이러한 취약점을 '효율적'으로 찾아내는 방법을 배울 차례입니다. 버그헌팅은 시간과의 싸움이기도 합니다. 모든 파라미터를 수동으로 하나하나 대입하는 것은 정확하지만 속도가 느릴 수 있습니다.

이번 14장에서는 전 세계 버그헌터들이 애용하는 자동화 도구(SQLMap, JWT Cracker, Hashcat)의 사용법과, 도구를 사용할 때 전문가로서 갖춰야 할 주의사항에 대해 상세히 정리해 드리겠습니다. 도구는 여러분의 손발이 되어줄 것입니다.

---

1. 목적

- **지식 (Knowledge):** SQL Injection 자동화 도구인 **SQLMap**의 주요 옵션과 작동 원리, **JWT**의 구조와 서명 취약점, 그리고 해시(Hash)의 개념과 크래킹 원리를 이해한다.
- **기술 (Skills):** SQLMap을 활용해 DB 정보를 추출하거나 시스템 쉘(OS Shell)을 획득하고, 취약한 비밀키로 서명된 JWT를 크랙하며, Hashcat을 이용해 암호화된 비밀번호를 복구하는 실습 능력을 갖춘다.
- **태도 (Attitude):** 자동화 도구는 보조 수단일 뿐임을 인지하고, 도구 사용 전 수동으로 취약점 유무를 판단하여 **운영 시스템에 미칠 부하와 리스크를 최소화**하려는 신중한 태도를 갖춘다.

2. 사전지식

- **SQL Injection:** 공격 원리에 대한 이해 (10장, 13장 내용).
- **인증/인가 기초:** JWT(JSON Web Token) 구조와 해시 알고리즘(MD5, SHA256 등)에 대한 이해.

3. 들어가기 전에

여러분, 30년 전에는 모든 해킹을 수작업으로 했습니다. 하지만 지금은 훌륭한 도구들이 많습니다. "도구를 쓰면 스크립트 키디(Script Kiddie) 아닐까요?"라고 묻는 학생들이 있는데, 천만의 말씀입니다. **도구의 원리를 알고 적재적소에 쓰는 사람은 전문가**이고, 아무것도 모르고 엔터만 치는 사람이 스크립트 키디입니다.

SQLMap 하나만 잘 다뤄도 며칠 걸릴 DB 분석을 몇 분 만에 끝낼 수 있습니다. 하지만 잘못 쓰면 서비스 장애를 일으켜 법적 책임을 질 수도 있습니다. 이번 시간에는 이 '양날의 검'을 어떻게 안전하고 강력하게 휘두를 수 있는지, 제 실무 노하우를 담아 알려드리겠습니다.

4. 수업내용

4.1. 버그헌팅과 자동화 도구의 필요성

- **효율성 증대:** 버그헌팅은 시간 소모가 큰 작업입니다. 대상 시스템을 체계적으로 탐색하거나 반복적인 테스트를 수행할 때 자동화 도구는 필수적입니다.
- **역할:** 도구는 해커를 대신해 취약점을 찾아주는 마법 지팡이가 아니라, 해커가 **자신의 작업을 더 효율적으로 수행하도록 돕는 비서**입니다.

4.2. SQLMap: SQL Injection 자동화의 끝판왕

- **개요:** MySQL, Oracle, MSSQL 등 다양한 DBMS를 지원하는 오픈소스 도구입니다.
- **주요 옵션 (운영 환경 필수):**
  - `--delay`**:** 쿼리 실행 간격을 설정하여 서버 부하를 줄이고 차단을 방지합니다.
  - `--risk`**:** 공격의 위험도를 관리합니다. 운영 중인 서비스에서는 이 옵션을 낮게 설정하여 치명적인 영향을 막아야 합니다.
  - `--tor`**:** 공격자의 IP를 숨기고 익명성을 유지하기 위해 Tor 네트워크를 사용합니다.
  - `-r`**:** 캡처한 HTTP 요청(덤프 파일)을 그대로 불러와 분석합니다. 복잡한 헤더나 데이터 구조를 일일이 입력할 필요가 없어 매우 유용합니다.
- **공격 기능 (Exploit):**
  - **--file-read:** SQL 인젝션이 확인된 후, 서버 내부의 파일(`/etc/passwd` 등)을 읽어옵니다.
  - **--os-shell:** 시스템 쉘에 접근합니다. 백엔드 언어(PHP, JSP 등)에 맞는 웹쉘을 자동으로 업로드하여 원격 명령 실행(RCE)을 가능하게 합니다.
- **주의사항 (Expert Tip):**
  - SQLMap은 고정된 쿼리를 사용하므로 비즈니스 로직을 이해하지 못합니다. **수동으로 취약점을 먼저 확인한 후**, 착취(Exploitation) 단계에서 도구를 사용하는 것을 권장합니다.

4.3. JWT (JSON Web Token) 크래킹

- **JWT 구조:** `Header.Payload.Signature`로 구성됩니다. 정보는 Base64로 인코딩되어 누구나 볼 수 있지만, **무결성은 서명(Signature)으로 보장**됩니다.
- **취약점:** 서명에 사용되는 비밀키(Secret Key)가 약하거나 하드코딩된 경우, 무차별 대입(Brute Force) 공격으로 키를 찾아낼 수 있습니다.
- **공격 도구 및 원리:**
  - `jwt_tool`, `jwtcrack` 등을 사용합니다.
  - 비밀키를 찾아내면 해커는 관리자 권한(`admin`)을 가진 위조된 토큰을 생성하여 **계정을 탈취**하거나 심지어 **RCE**까지 연결할 수 있습니다.

4.4. Hashcat: 패스워드 복구 도구

- **개요:** 해싱된 문자열(암호문)의 원본을 찾아내는 크래킹 도구입니다. 침투 테스트 시 내부망에서 획득한 관리자 패스워드 해시를 푸는 데 매우 유용합니다.
- **식별 도구:** `CyberChef`의 'Hash Analyzer' 등을 이용해 해시의 종류(MD5, SHA1 등)를 먼저 파악해야 합니다.

- **공격 모드 (Attack Modes):**
    1. **사전 공격 (Dictionary Attack):** 미리 준비된 단어 목록을 대입합니다.
    2. **조합 공격 (Combinator Attack):** 두 개의 단어를 합쳐서 대입합니다.
    3. **마스크 공격 (Mask Attack):** 알려진 패턴(예: `Pass` + 숫자4자리)을 기반으로 대입합니다.
    4. **규칙 기반 (Rule-based):** 단어의 대소문자 변경, 특수문자 추가 등 규칙을 적용합니다.

5. 핵심내용 (Summary)

1. **SQLMap 활용:** `-r`(요청 파일 로드), `--delay`(지연), `--risk`(위험도 조절) 옵션은 실전 버그헌팅의 필수입니다. `--os-shell` 옵션으로 시스템 장악이 가능하지만, 반드시 사전에 수동 검증을 거쳐야 합니다.
2. **JWT의 약점:** JWT 자체는 안전할지라도, 서명에 사용된 **비밀키가 단순**하면 크랙 도구를 통해 쉽게 뚫리며, 이는 계정 탈취로 이어집니다.
3. **Hashcat의 위력:** 해시값만 탈취했다고 끝이 아닙니다. **Hashcat**을 이용해 다양한 공격 모드(사전, 마스크 등)로 원본 패스워드를 복구하여 2차 공격에 활용할 수 있습니다.
4. **도구의 한계:** 자동화 도구는 로직을 이해하지 못합니다. 도구에만 의존하면 복잡한 취약점을 놓치거나 시스템에 장애를 줄 수 있으므로, **'수동 분석 후 자동화 검증'** 프로세스를 지켜야 합니다.

6. 마무리 키워드 퀴즈

오늘 배운 도구들의 핵심을 확인해 봅시다.

- **Q1.** SQL Injection 취약점을 자동으로 탐지하고 데이터를 추출하는 도구로, `--os-shell` 옵션을 통해 시스템 쉘 획득까지 지원하는 도구는 무엇인가요?
    -
- **Q2.** JWT(JSON Web Token)의 무결성을 검증하는 서명(Signature) 부분을 생성할 때 사용되는 알고리즘으로, 대칭키 방식을 사용하는 대표적인 알고리즘은 무엇인가요?
    -
- **Q3.** 획득한 해시값을 바탕으로 원본 패스워드를 복구하는 도구로, 사전 공격, 마스크 공격 등 다양한 모드를 지원하는 대표적인 해시 크래킹 툴은 무엇인가요?
    -

7. 다음 장 소개

자동화 도구라는 강력한 무기를 손에 넣으셨군요. 다음 시간에는 전 세계 보안 취약점의 '주민등록번호'라고 할 수 있는 "CVE(Common Vulnerabilities and Exposures)의 이해"에 대해 알아보겠습니다. CVE 번호가 무엇인지, 이를 어떻게 분석하고 활용하여 최신 취약점을 공략할 수 있는지 상세히 다루겠습니다.

15장. CVE의 이해와 메타스플로잇(Metasploit) 활용 실습

지난 시간까지 우리는 자동화 도구(SQLMap 등)를 통해 효율적으로 취약점을 찾는 법을 배웠습니다. 이제 여러분은 전 세계 해커들이 공통으로 사용하는 '언어'를 배워야 합니다. 보안 뉴스에서 "CVE-2021-44228 로그포쉘(Log4Shell) 발생" 같은 헤드라인을 본 적이 있나요? 여기서 **CVE**가 바로 그 언어입니다.

이 번호 하나에 취약점의 모든 역사가 담겨 있습니다. 이번 15장에서는 이 CVE가 도대체 무엇인지, 그리고 해커들의 만능 칼이라 불리는 메타스플로잇(Metasploit)을 활용해 이 CVE 취약점을 어떻게 실습(Exploit)하는지 상세히 정리해 드리겠습니다.

---

1. 목적

- **지식 (Knowledge):** 정보 보안 취약점 표준 코드인 CVE(Common Vulnerabilities and Exposures)의 정의, 할당 체계(CNA), 부여 기준을 이해하고, 메타스플로잇 프레임워크(Metasploit Framework)의 모듈 구조를 파악한다.
- **기술 (Skills): Searchsploit**을 사용하여 Exploit-DB의 취약점 코드를 검색하고, Metasploit(msfconsole)을 활용해 특정 CVE(예: CVE-2021-41773)에 대한 모듈을 로드, 설정, 실행(Exploit)하여 쉘을 획득하는 실습 능력을 갖춘다.
- **태도 (Attitude):** 이미 알려진 CVE 취약점(1-day)이라도 버그바운티에서 활용할 수 있는 전략적 가치를 인지하고, 공개된 PoC 코드를 분석하여 원리를 이해하려는 탐구적인 자세를 갖춘다.

2. 사전지식

- **기본 리눅스 명령어:** Kali Linux 환경에서의 터미널 조작 및 Docker 기본 명령어.
- **네트워크 기초:** IP 주소, 포트(Port), 리버스 쉘(Reverse Shell) 개념.

3. 들어가기 전에

여러분, 보안 취약점에도 '주민등록번호'가 있다는 사실을 아시나요? 바로 **CVE**입니다. 1999년 처음 시작된 이래, 전 세계 모든 주요 취약점은 이 번호로 관리됩니다. "아파치 서버에 구멍이 났대"라고 말하는 것보다 "CVE-2021-41773 취약점이 발견됐어"라고 말하는 것이 훨씬 정확하고 전문적입니다.

또한, 이번 시간에는 해킹 영화에서 보던 검은 화면에 명령어를 입력해 시스템을 장악하는 도구, **메타스플로잇**을 실습합니다. 단순히 도구를 돌리는 '스크립트 키디'가 아니라, 도구의 원리를 이해하고 CVE 정보를 바탕으로 전략적으로 공격을 수행하는 '전문가'의 길로 안내하겠습니다.

4. 수업내용

4.1. CVE (Common Vulnerabilities and Exposures) 개요

- **정의:** 정보 보안 취약점 표준 코드로, 컴퓨터 보안 취약점에 고유한 식별 코드를 부여하는 국제 표준입니다.
- **목적:** 취약점을 일관된 방식으로 식별하고 추적하여 정보 공유와 대응을 용이하게 합니다. 보안 결함에 대한 '공통 언어' 역할을 합니다.
- **역사 및 형식:** 과거에는 4자리(CVE-YYYY-NNNN)였으나, 취약점 급증으로 2016년부터 4자리 이상으로 확대되었습니다.
- **CNA (CVE Numbering Authorities):** CVE 번호를 부여할 권한을 가진 기관입니다. MITRE를 중심으로 운영되며, 한국에서는 KISA와 네이버 등이 CNA 권한을 가지고 있습니다. 2017년부터는 오픈소스 프로젝트도 자체적으로 CNA가 될 수 있게 되었습니다.

4.2. CVE 부여 기준

모든 버그에 CVE를 주지 않습니다. 다음 세 가지 조건을 충족해야 합니다.

1. **독립적 수정 가능:** 다른 버그와 독립적으로 수정(패치)할 수 있어야 합니다.
2. **공급자의 인정:** 해당 소프트웨어 공급자가 버그를 인정하거나 문서화해야 합니다.
3. **단일 코드베이스:** 하나의 코드베이스에 영향을 미쳐야 합니다. (여러 제품에 동일 결함이 있어도 코드베이스가 다르면 별도 CVE 부여).

4.3. 버그헌팅에서의 CVE 활용 전략

- **활용법:** 타겟 서비스에서 사용하는 서드파티(CMS, 라이브러리 등)를 식별한 후, 해당 버전의 알려진 CVE가 있는지 조회합니다.
- **장점:**
  - **시간 단축:** 제로데이(0-day) 발굴보다 빠르게 취약점을 찾을 수 있습니다.
  - **학습 용이성:** 공개된 PoC(Proof of Concept) 코드를 통해 취약점 원리를 쉽게 배울 수 있습니다.
  - **파생 연구:** 비슷한 유형의 다른 취약점을 발견하는 단서가 됩니다.

- **단점:** 이미 알려진 취약점이기에 중복(Duplicate) 제보가 많을 수 있고, 기업이 이미 패치했을 확률이 높습니다.

4.4. 메타스플로잇(Metasploit) 프레임워크 실습

- **개요:** 침투 테스트를 위한 오픈소스 프레임워크로, 수많은 Exploit 모듈과 Payload를 내장하고 있습니다. 칼리 리눅스에 기본 설치되어 있습니다.

- **주요 모듈:**
  - **Exploit:** 실제 공격을 수행하는 코드.
  - **Payload:** 공격 성공 후 실행할 코드 (예: 쉘 획득, 계정 생성).
  - **Auxiliary:** 정보 수집, 스캔 등 보조 기능.
  - **Encoder:** 보안 장비 탐지를 피하기 위해 페이로드를 변조.
- **실습: Apache Path Traversal & RCE (CVE-2021-41773)**
  - **환경 구축:** Docker를 이용해 취약한 버전(Apache 2.4.49)의 서버를 구축합니다.
  - **정보 수집 (Searchsploit):** `searchsploit apache 2.4.49` 명령어로 Exploit-DB에 등록된 취약점 정보를 검색합니다.
  - **메타스플로잇 실행 (`msfconsole`):**
    1. **검색:** `search cve:2021-41773`으로 관련 모듈을 찾습니다.
    2. **모듈 선택:** `use exploit/multi/http/apache_normalize_path_rce` 명령어로 모듈을 로드합니다.
    3. **옵션 확인:** `show options`로 설정해야 할 항목(RHOSTS, LHOST 등)을 확인합니다.
    4. **설정:** `set RHOSTS [타겟IP]`, `set LHOST [내IP]` 등으로 공격 대상을 지정합니다.
    5. **공격 실행:** `exploit` 명령어를 입력하면 공격이 수행되고, 성공 시 **Meterpreter 세션**이 열려 원격 제어(RCE)가 가능해집니다.

5. 핵심내용 (Summary)

1. **CVE는 표준:** CVE는 취약점을 식별하는 전 세계 공통 식별자이며, 버그헌터는 이를 통해 알려진 취약점(1-day)을 빠르게 학습하고 적용할 수 있습니다.
2. **Searchsploit 활용:** `searchsploit` 명령어는 Exploit-DB의 방대한 데이터베이스를 로컬에서 빠르게 검색하여 PoC 코드를 찾는 데 유용합니다.
3. **Metasploit의 강력함:** `search` -> `use` -> `set` -> `exploit`으로 이어지는 메타스플로잇의 워크플로우를 익히면, 복잡한 공격 과정(Payload 생성, 핸들러 대기 등)을 자동화하여 손쉽게 시스템 권한을 획득할 수 있습니다.
4. **실전 적용:** CVE-2021-41773 사례처럼 단순한 경로 탐색(Path Traversal) 취약점이 RCE(원격 코드 실행)로 이어질 수 있음을 확인하고, 도구를 통해 이를 검증하는 능력이 필요합니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 컴퓨터 보안 취약점에 고유한 식별 코드를 부여하여, 전 세계적으로 취약점 정보를 공유하고 추적하기 위해 만든 국제 표준 식별자는 무엇인가요?
    -
- **Q2.** 메타스플로잇(Metasploit) 프레임워크에서 실제 취약점을 공격하는 코드는 'Exploit' 모듈이고, 공격 성공 후 타겟 시스템에서 실행되어 공격자에게 쉘을 연결해주는 등의 행위를 하는 코드를 무엇이라 하나요?
    -
- **Q3.** Exploit-DB에 등록된 취약점 정보를 리눅스 터미널에서 오프라인으로 빠르게 검색할 수 있게 해주는 칼리 리눅스 내장 도구 명령어는 무엇인가요?
    -

7. 다음 장 소개

이제 CVE의 개념과 이를 공격하는 도구 사용법까지 익혔습니다. 다음 시간에는 **"최근 CVE 취약점 분석과 이해"** 파트로 넘어가, 최근 보안 업계를 뜨겁게 달궜던 **Log4Shell**이나 **Spring4Shell** 같은 실제 최신 사례들을 깊이 있게 분석해 보겠습니다. 이론을 넘어 트렌드를 읽는 눈을 키워드리겠습니다.

16장. CVE 버그헌팅 사례

지난 시간에는 CVE의 개념과 메타스플로잇의 기초 사용법을 익혔습니다. 이제는 '실전'입니다. 단순히 도구 사용법을 아는 것과, 실제 공개된 취약점(CVE)을 분석하여 공격 코드를 수정하고 성공시키는 것은 천지 차이입니다.

이번 16장에서는 CVSS 점수 9.8점, 즉 '치명적(Critical)' 등급을 받은 PrimeFaces RCE 취약점(CVE-2017-1000486)을 깊이 있게 분석하고, 공개된 익스플로잇 코드가 작동하지 않을 때 이를 해결하는 **트러블슈팅(Troubleshooting)** 과정을 실습해 보겠습니다. 해커에게 가장 필요한 덕목인 '집요함'을 배우는 시간이 될 것입니다.

---

1. 목적

- **지식 (Knowledge):** CVE-2017-1000486 취약점의 원인이 되는 **PBE(Password Based Encryption)** 알고리즘과 EL(Expression Language)의 개념을 이해하고, CVSS 점수 9.8점이 가지는 위험성을 설명할 수 있다.
- **기술 (Skills):** Exploit-DB에서 확보한 외부 모듈을 메타스플로잇에 로드(`reload_all`)하고, 공격 실패 시 프록시(Burp Suite)를 연동하여 패킷을 분석, 인코딩 문제 등을 수정하여 **최종적으로 쉘을 획득**하는 능력을 갖춘다.
- **태도 (Attitude):** 공개된 공격 코드가 환경에 따라 즉시 작동하지 않을 수 있음을 인지하고, 단순히 포기하는 것이 아니라 원인을 분석하고 코드를 수정하여 기어이 성공시키는 **끈기 있는 연구 자세**를 함양한다.

2. 사전지식

- **메타스플로잇 기초:** 모듈 검색 및 옵션 설정 (`use`, `set`, `exploit`).
- **암호학 기초:** 대칭키 암호화와 해시 함수(MD5)에 대한 기본 이해.
- **웹 프레임워크:** 자바 기반 웹 프레임워크(JSF)의 구조.

3. 들어가기 전에

여러분, 공격 코드를 구했는데 막상 실행해보니 에러가 뜨거나 작동하지 않는다면 어떻게 하시겠습니까? 초보자는 거기서 멈추지만, 전문가는 그때부터 눈을 반짝입니다. "왜 안 되지?"라는 질문이 해킹 실력을 키우는 가장 빠른 지름길이기 때문입니다.

오늘 다룰 **PrimeFaces 취약점**은 아주 단순한 실수에서 비롯되었습니다. 오픈소스 프레임워크가 '기본 비밀번호(default password)'를 사용했고, 그 사실이 전 세계에 공개되었기 때문이죠. 이 취약점을 이용해 시스템을 장악하는 과정을 실습하되, 일부러 오류가 발생하는 상황을 연출하여 여러분이 직접 문제를 해결하는 힘을 길러드리겠습니다.

4. 수업내용

4.1. CVE-2017-1000486 취약점 개요 및 CVSS 분석

- **취약점 정보:** PrimeFaces 5.x 버전에서 발생한 원격 코드 실행(RCE) 취약점입니다.
- **CVSS 스코어 9.8 (Critical):**
  - **공격 벡터(Network):** 원격에서 공격 가능.
  - **권한 요구(None):** 로그인하지 않은 공격자도 수행 가능.
  - **사용자 상호작용(None):** 피해자가 클릭 등을 할 필요 없음.
  - **영향성(High):** 기밀성, 무결성, 가용성 모든 측면에서 시스템이 완전히 장악될 수 있습니다.

4.2. 취약점 발생 원리: 암호화와 EL 표현식

- **PBE (Password Based Encryption):** 패스워드를 기반으로 암호화 키를 생성하는 방식입니다. PrimeFaces는 데이터를 보호하기 위해 이 방식을 사용했습니다.

- **문제점:** PrimeFaces가 오픈소스이다 보니, 암호화에 사용되는 **기본 패스워드("primefaces")**, 솔트(Salt), 반복 횟수(Iterations) 등의 설정값이 소스코드에 그대로 노출되어 있었습니다. 해커는 이를 이용해 암호화된 페이로드를 위조할 수 있게 되었습니다.
- **EL (Expression Language):** `#{...}` 형태의 스크립트 언어로, 자바 객체에 쉽게 접근하게 해줍니다. 해커는 위조된 암호화 페이로드 안에 악성 EL 구문을 삽입하여 서버가 이를 실행하게 만듭니다.

4.3. 파이썬 PoC 코드를 이용한 공격 실습

- **PoC 분석:** 공개된 파이썬 스크립트는 `primefaces`라는 기본 비밀번호를 사용하여 악성 명령(`whoami` 등)을 암호화하고, 이를 서버에 전송합니다.
- **실행 결과:** 스크립트 실행 시 서버로부터 `root`라는 응답을 받아 RCE가 성공했음을 확인합니다.

4.4. 메타스플로잇 외부 모듈 로드 및 트러블슈팅

이 과정이 오늘 수업의 하이라이트입니다.

1. **모듈 확보:** 메타스플로잇 DB에 없는 경우, `searchsploit`을 통해 Exploit-DB의 루비(.rb) 스크립트를 찾습니다.
2. **모듈 로드 에러 해결:** 다운로드한 스크립트를 그냥 실행하면 `Msf` 상수를 찾지 못해 에러가 납니다.
    - **해결:** 스크립트를 `~/.msf4/modules/...` 경로(사용자 모듈 경로)에 복사한 뒤, msfconsole에서 **`reload_all`** 명령어를 입력하여 프레임워크가 인식하게 해야 합니다.
3. **공격 실패 및 원인 분석:**
    - 모듈을 로드하고 `exploit`을 실행했으나, "Server is not vulnerable"이라며 실패합니다.
    - **디버깅(Debugging):** 메타스플로잇의 `PROXIES` 옵션을 설정하여 **Burp Suite**로 패킷을 가로채 확인합니다. 이때 `ReverseAllowProxy true` 설정이 필요합니다.
4. **문제 해결 (인코딩 수정):**
    - **원인:** 메타스플로잇 모듈이 페이로드를 이중 URL 인코딩(Double URL Encoding)하여 보내고 있어, 서버가 이를 제대로 해석하지 못하는 상황이었습니다.
    - **수정:** 파이썬 등으로 올바르게 인코딩된 페이로드를 생성하거나, 스크립트 내에서 인코딩 방식을 수정하여 재전송합니다.
    - **결과:** 수정된 페이로드 전송 결과 `root` 권한 획득에 성공합니다.

5. 핵심내용 (Summary)

1. **취약점의 근본:** 오픈소스 소프트웨어에서 기본 설정(Default Configuration)이나 하드코딩된 비밀번호를 변경하지 않고 사용하는 것은 치명적인 RCE 취약점(CVSS 9.8)으로 이어질 수 있습니다.
2. **EL 인젝션:** 자바 기반 웹 앱에서 사용자 입력값이 검증 없이 EL 표현식(`#{...}`)으로 처리되면, 서버 내부 객체를 조작하거나 코드를 실행할 수 있습니다.

3. **도구의 커스터마이징:** 공개된 공격 도구(Exploit)는 내 환경에 100% 맞지 않을 수 있습니다. `reload_all`을 통해 외부 모듈을 불러오고, 프록시(Burp Suite)를 연동하여 패킷을 눈으로 확인하며 디버깅하는 능력이 필수적입니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 자바 기반 웹 애플리케이션(JSF)에서 사용되는 스크립트 언어로, `#{...}`와 같은 형식을 가지며 취약하게 구현될 경우 RCE로 이어질 수 있는 것은 무엇인가요?
  -
- **Q2.** 메타스플로잇 실행 중에 사용자가 직접 추가한 외부 모듈(.rb)을 인식시키거나, 수정된 코드를 반영하기 위해 콘솔에서 입력해야 하는 명령어는 무엇인가요?
  -
- **Q3.** CVE-2017-1000486 취약점의 근본 원인 중 하나로, PrimeFaces 프레임워크가 암호화 과정에서 사용한 '이것'이 오픈소스에 공개되어 있어 해커가 암호문을 위조할 수 있었습니다. '이것'은 무엇인가요?
  -

7. 다음 장 소개

이번 시간에는 과거에 발생했던 특정 CVE를 깊이 있게 파고들었습니다. 다음 시간에는 시야를 넓혀 "최근 CVE 취약점 분석과 이해"를 주제로, **Log4Shell**이나 **Spring4Shell**과 같이 최근 보안계를 뒤흔들었던 최신 트렌드 취약점들을 분석해보겠습니다. 과거의 교훈을 바탕으로 현재의 위협에 대응하는 방법을 배워봅시다.

17장. CVE 버그헌팅 사례 2

이번 강의는 그동안 배운 이론과 도구들이 실제 최신 보안 위협 속에서 어떻게 발현되는지 확인하는 **'실전 사례 분석'** 시간입니다. 단순히 "취약점이 있다"는 사실을 넘어, 사소해 보이는 설정 실수 하나가 어떻게 시스템 전체 장악(RCE)으로 이어지는지, 그 치명적인 연결고리를 파헤쳐 보겠습니다.

---

1. 목적

- **지식 (Knowledge):** Flask 프레임워크의 세션 서명 방식과 **기본 비밀키(Default Secret Key)** 사용의 위험성(CVE-2023-23524), 그리고 XSS와 CSRF를 연계하여 RCE로 발전시키는 **취약점 체이닝(Chaining)** 원리를 이해한다.
- **기술 (Skills):** `flask-unsign` 도구를 사용하여 서명된 쿠키를 크랙 및 위조하고, PostgreSQL의 `COPY FROM PROGRAM` 기능을 악용하여 시스템 명령을 실행하는 시나리오를 구상할 수 있다.
- **태도 (Attitude):** 취약점 발굴 시 단순히 시스템 침투에만 몰두하지 않고, 윤리적 해커 강령(Ethical Hacking Guidelines)을 준수하며 서비스 가용성을 해치지 않는 범위 내에서 연구를 수행하는 책임감을 갖춘다.

2. 사전지식

- **웹 프레임워크:** Python Flask의 세션 관리 메커니즘 (클라이언트 사이드 세션).
- **데이터베이스:** PostgreSQL의 기본 명령어 및 권한 체계.
- **클라우드/컨테이너:** Azure Service Fabric 및 Docker 컨테이너의 기본 개념.

3. 들어가기 전에

여러분, "설마 관리자가 비밀번호를 기본값(Default)으로 놔뒀겠어?"라는 안일한 생각이 보안 사고의 시작입니다. 놀랍게도 전 세계 수천 개의 서버가 설명서에 적힌 '예시 비밀키'를 그대로 사용하여 뚫리고 있습니다.

오늘 다룰 **Apache Superset** 사례는 `SECRET_KEY` 하나가 노출되었을 때 어떻게 관리자 권한 탈취를 넘어 서버 장악까지 이어지는지 보여주는 완벽한 예시입니다. 또한, 마이크로소프트 **Azure** 사례를 통해 "XSS는 단순히 알림 창 띄우는 것"이라는 편견을 깨고, 클라우드 인프라 전체를 위협하는 무기로 변모하는 과정을 보여드리겠습니다. 해커의 시선으로 '점'을 연결하여 '선'을 만드는 과정을 주목해 주십시오.

4. 수업내용

4.1. Apache Superset 취약점 (CVE-2023-23524) 분석

- **취약점 개요:** 데이터 시각화 도구인 Apache Superset에서 세션 서명에 사용되는 `SECRET_KEY`가 기본값(Default Value)으로 설정되어 있어 발생하는 인증 우회 취약점입니다.
- **발생 원인:**
  1. Superset은 Python Flask 프레임워크 기반이며, 세션 정보를 서버가 아닌 **클라이언트의 쿠키**에 저장합니다.
  2. 이 쿠키는 `Base64 인코딩된 데이터 + 타임스탬프 + 서명(Hash)` 구조로 되어 있어 내용은 누구나 볼 수 있지만, 서명 때문에 변조는 불가능합니다.
  3. 하지만 통계적으로 2,000개 이상의 서버가 공식 문서나 소스 코드에 있는 **기본 시크릿 키**(`thisismyscretkey` 등)를 변경하지 않고 그대로 사용했습니다.
- **공격 시나리오 (Exploit):**
  1. **키 확보:** 오픈소스 코드나 문서에서 기본 키 목록을 수집합니다.
  2. **세션 위조:** `flask-unsign` 도구를 사용해 기본 키로 서명을 검증하고, 관리자 권한(user_id=1)을 가진 위조 세션 쿠키를 생성합니다.
  3. **권한 획득:** 위조된 쿠키를 브라우저에 주입하여 관리자 계정으로 로그인합니다.

4.2. 심화 공격: PostgreSQL을 이용한 RCE 연계

- 관리자 계정 탈취에서 멈추지 않고, 원격 코드 실행(RCE)까지 확장하는 방법입니다.
- **PostgreSQL RCE (CVE-2019-9193):**
  - PostgreSQL의 `COPY` 명령어는 파일이나 프로그램의 출력을 테이블로 복사하는 기능을 제공합니다.
  - **Payload:** `COPY cmd_exec FROM PROGRAM 'id';`와 같이 입력하면, DB 서버 운영체제에서 `id` 명령어를 실행하고 그 결과를 테이블에 저장하여 탈취할 수 있습니다.
- **체이닝 결과:** Superset 관리자 권한으로 DB 설정을 변경하여 이 기능을 활성화한 뒤, 시스템 명령어를 실행해 서버를 완전히 장악합니다.

4.3. Azure Service Fabric 취약점 (Super FabriXss / CVE-2023-23383)

- **취약점 개요:** 마이크로소프트의 Azure Service Fabric Explorer(SFX)에서 발견된 **Stored XSS** 취약점입니다.
- **발현 조건:** URL 파라미터가 필터링 없이 화면에 출력되지만, 단순히 스크립트를 넣는다고 실행되지 않습니다. UI에서 **'Cluster' 이벤트 탭**을 클릭하는 특정 상호작용이 있어야만 렌더링되면서 스크립트가 실행됩니다.
- **공격 시나리오 (Chaining):**
  1. **XSS 트리거:** 공격자가 조작된 URL을 관리자에게 보내 클릭을 유도합니다. 관리자가 'Cluster' 탭을 누르는 순간 XSS가 실행됩니다.
  2. **CSRF 연계:** 실행된 XSS 스크립트는 백그라운드에서 `UpgradeApplication` API를 호출하는 CSRF 공격을 수행합니다.
  3. **RCE 달성:** 이 명령을 통해 악성 Docker 이미지가 포함된 설정으로 클러스터를 '업그레이드'시킵니다. 결과적으로 관리자의 컨테이너 환경에서 공격자의 코드가 실행(RCE)됩니다.

- **의의:** 단순한 XSS(Low/Medium)가 다른 기능과 결합하여 Critical 등급(CVSS 8.2)의 위협으로 진화한 모범 사례입니다.

4.4. 버그헌팅 윤리 및 주의사항

- **시스템 보호:** 취약점 검증 과정에서 서비스 거부(DoS), 데이터 삭제, 과도한 자원 소모 등 실제 피해를 주는 행위는 엄격히 금지됩니다.
- **규칙 준수:**
  - **개인정보 보호:** 타인의 데이터를 열람하거나 수집해서는 안 됩니다.
  - **제약 사항:** 기업이 제시한 규칙(예: 파일 생성 시 특정 이름 사용, User-Agent 변경 등)을 반드시 따라야 합니다.
  - **책임:** 좋은 의도라도 규칙을 어겨 발생한 피해는 본인의 과실이며 법적 책임을 질 수 있습니다.

5. 핵심내용 (Summary)

1. **기본 설정의 위험성:** 오픈소스나 프레임워크의 기본 시크릿 키(Default Secret Key)를 변경하지 않는 것은 인증 체계를 무너뜨리는 치명적인 실수입니다 (Apache Superset 사례).
2. **취약점의 확장성:** 단순히 관리자 계정을 얻는 것에 그치지 않고, DB 기능(PostgreSQL COPY) 등을 악용하여 RCE(시스템 장악)까지 시나리오를 확장해야 파급력(Impact)을 증명할 수 있습니다.
3. **체이닝의 미학:** XSS가 단순한 정보 탈취가 아니라, 클러스터 배포 명령(CSRF)과 결합하여 클라우드 인프라를 장악하는 도구가 될 수 있음을 확인했습니다 (Super FabriXss).
4. **윤리적 해킹:** 버그헌팅은 '파괴'가 아닌 '보호'를 위한 활동입니다. 가용성을 해치지 않고 규칙을 준수하는 것이 전문가의 기본 소양입니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** Apache Superset 취약점 사례에서, 공격자가 관리자 세션을 위조하기 위해 악용한 것으로, Flask 프레임워크가 쿠키 변조를 막기 위해 사용하는 핵심 데이터는 무엇인가요? (이것이 기본값으로 설정되어 있어 문제가 되었습니다.)
  -
- **Q2.** Flask 세션 쿠키와 같이 서명된 데이터를 크랙하거나 위조할 때 사용하는 대표적인 커맨드 라인 도구의 이름은 무엇인가요?
  -
- **Q3.** Azure Service Fabric 사례처럼, XSS와 같은 낮은 위험도의 취약점을 다른 기능이나 취약점(CSRF 등)과 연결하여 전체 시스템 장악과 같은 높은 위험도로 발전시키는 기법을 무엇이라고 합니까?
  -

7. 다음 장 소개

실제 사례를 통해 취약점의 파괴력을 확인하셨나요? 다음 시간에는 공격을 수행하기 전 가장 중요한 단계인 "버그헌팅 정보수집 전략(Reconnaissance)"에 대해 다루겠습니다. 공격 대상을 식별하고,

숨겨진 서브도메인이나 API 엔드포인트를 찾아내는 정찰 기술을 OSINT 기법과 함께 상세히 배워보겠습니다.

18장. CVE 취약점 버그헌팅 사례 3

이전 시간까지 도구 사용법과 이론적인 CVE 분석을 마쳤습니다. 이제는 "그래서 실제로 어떻게 뚫었는데?"에 대한 답을 드릴 차례입니다. 이번 18강에서는 전 세계적으로 큰 파장을 일으켰던 두 가지 실제 CVE 버그헌팅 사례(VMware, ImageMagick)를 심층 분석합니다.

특히, 단순한 이미지 업로드가 어떻게 서버의 비밀번호 파일(`/etc/passwd`) 유출로 이어지는지, 그 마법 같은 과정을 코드로 보여드리겠습니다.

---

1. 목적

- **지식 (Knowledge):** Java 템플릿 엔진인 **Freemarker**의 SSTI(Server-Side Template Injection) 취약점 원리와, 이미지 처리 라이브러리인 **ImageMagick**의 논리적 결함으로 인한 파일 유출 원리를 이해한다.
- **기술 (Skills):** 공개된 PoC(Proof of Concept) 코드를 활용하여 취약한 버전의 프레임워크를 공격하고, 이미지 파일의 메타데이터(Chunk)를 조작 및 해석하여 유출된 데이터를 복원하는 기술을 익힌다.
- **태도 (Attitude):** 오픈소스 소프트웨어의 변경 사항(Commit)과 패치 노트를 분석하여 제로데이(0-day)나 원데이(1-day) 취약점을 찾아내는 연구 중심의 자세를 함양한다.

2. 사전지식

- **SSTI (Server-Side Template Injection):** 템플릿 엔진의 동작 방식 이해.
- **이미지 포맷 구조:** PNG 파일의 Chunk 구조 및 메타데이터 개념.
- **리눅스 기초:** `/etc/passwd` 등 주요 시스템 파일의 경로와 의미.

3. 들어가기 전에

여러분, 해커원(HackerOne)에서 단일 취약점으로 약 3,300만 원($25,000)을 받은 사례가 있습니다. 놀랍게도 아주 복잡한 기술이 아니었습니다. 우리가 흔히 쓰는 **'이미지 크기 변경'** 기능에서 발생한 문제입니다.

또한, VMware 같은 거대 기업의 솔루션도 **'템플릿 엔진'** 설정을 소홀히 하여 원격 코드 실행(RCE)이라는 치명적인 구멍을 내주었습니다. 이번 시간에는 이 두 가지 사례를 통해, "남들이 보지 못하는 아주 작은 설정 값 하나"가 어떻게 거대한 포상금으로 돌아오는지 확인해 보겠습니다.

4. 수업내용

4.1. 사례 1: VMware Workspace ONE Access (CVE-2022-22954)

- **취약점 개요:** VMware의 IAM(신원 관리) 플랫폼에서 발생한 **원격 코드 실행(RCE)** 취약점입니다. 공격자는 인증 없이도 서버를 장악할 수 있었습니다.
- **발생 원인: Freemarker 템플릿 인젝션 (SSTI)**
  - **Freemarker란?** Java 기반의 템플릿 엔진으로, 데이터 모델과 템플릿을 결합하여 HTML 문서를 동적으로 생성해줍니다.
  - **원인 분석:**
    1. Freemarker 엔진에는 시스템 명령어를 실행할 수 있는 `Execute` 클래스가 존재합니다.
    2. 공식 문서에는 사용자 입력값이 이 태그에 들어가지 않도록 경고하고 있으나, VMware 제품은 이를 검증 없이 사용했습니다.
    3. 보안 설정인 `NewBuiltinClassResolver`가 제한적으로 설정되지 않아(`Unregistered Resolver` 상태 아님), 공격자가 임의의 객체를 생성하고 실행할 수 있었습니다.
- **공격 방법:**
  - 공개된 Exploit 코드를 메타스플로잇이나 스크립트로 전송하여, 서버가 템플릿을 렌더링하는 과정에서 악성 명령(`cat /etc/passwd` 등)이 실행되도록 유도합니다.

4.2. 사례 2: ImageMagick 파일 유출 (CVE-2022-44268)

- **취약점 개요:** 해커원(HackerOne) 자체 버그바운티에서 발견된 건으로, 이미지 처리 도구인 **ImageMagick**의 취약점을 이용해 서버 내부 파일을 읽어낸 사례입니다. 포상금은 **$25,000**였습니다.
- **공격 시나리오:**
  1. 공격자는 프로필 사진 업로드 기능을 타겟으로 삼습니다.
  2. 악의적으로 조작된 PNG 파일을 업로드합니다. (이미지 처리 과정 유발).
  3. 리사이즈된 이미지를 다시 다운로드합니다.
  4. 다운로드한 이미지의 정보를 확인하면 서버의 파일 내용이 포함되어 있습니다.
- **심층 분석 (Root Cause Analysis):**
  1. **취약점 위치:** `ReadOnePNGImage` 함수가 PNG의 텍스트 청크(tEXt chunk)를 읽을 때 발생합니다.

    2. **키워드 'profile':** ImageMagick 7.1.0-49 버전 코드에서, 텍스트 청크의 키워드가 `profile`인 경우, 그 값을 `filename` 변수로 복사하는 로직이 발견되었습니다 (`SetImageProperty` 함수).
    3. **파일 읽기:** 이후 `FileToStringInfo` 함수가 이 `filename`에 해당하는 파일(예: `/etc/passwd`)을 읽어옵니다.
    4. **데이터 쓰기:** 읽어온 파일 내용은 `SetImageProfile` 함수를 통해, 새로 생성되는(리사이즈된) 이미지의 프로필 정보로 저장됩니다.
- **Exploit 실습 (데이터 복원):**
    1. 공격자는 PNG 파일의 `profile` 속성에 `/etc/passwd` 경로를 넣어 업로드합니다.
    2. 처리된 이미지를 다운로드 후, `identify -verbose image.png` 명령어로 정보를 확인합니다.
    3. **`Raw profile type`** 항목에 16진수(Hex) 값들이 출력됩니다.
    4. 이 16진수 값에서 개행 문자를 제거하고 ASCII로 변환(Decoding)하면, 유출된 `/etc/passwd` 파일의 내용을 볼 수 있습니다.

5. 핵심내용 (Summary)

1. **SSTI의 위험성:** VMware 사례는 템플릿 엔진(Freemarker)을 사용할 때 사용자 입력값을 신뢰하면 안 되며, 보안 설정(Resolver 제한)을 반드시 적용해야 함을 보여줍니다.
2. **논리적 버그의 파급력:** ImageMagick 사례는 메모리 커럽션 같은 어려운 기술이 아니라, "특정 키워드(profile)를 만나면 파일을 읽는다"는 소프트웨어의 **로직**을 악용하여 고액의 포상금을 획득했습니다.
3. **메타데이터의 비밀:** 서버에서 처리된 결과물(이미지 등)의 메타데이터(Raw profile type)에는 서버가 처리 과정에서 남긴 민감한 흔적이 남아있을 수 있으므로 꼼꼼히 확인해야 합니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** VMware 취약점 사례에서 사용된 Java 기반 템플릿 엔진으로, 보안 설정 미흡 시 SSTI를 통해 원격 코드 실행(RCE)이 가능한 기술은 무엇인가요?
    -
- **Q2.** ImageMagick 취약점(CVE-2022-44268)에서 공격자가 PNG 파일의 텍스트 청크(tEXt chunk)에 삽입하여, 서버가 해당 경로의 파일을 읽도록 유도한 특정 키워드는 무엇인가요?
    -
- **Q3.** ImageMagick 공격 성공 후, 다운로드한 이미지에 포함된 유출 데이터를 확인하기 위해 사용하는 명령어로, 이미지의 상세 속성(Raw profile type 등)을 출력해주는 도구 명령은 무엇인가요?
    -

7. 다음 장 소개

실제 사례들을 통해 취약점의 파괴력을 확인하셨나요? 다음 시간에는 공격을 수행하기 전 가장 기본이면서도 중요한 단계인 "버그헌팅 정보수집 전략(Reconnaissance)"에 대해 다루겠습니다. 공격

대상을 식별하고(Subdomain Enumeration), 숨겨진 자산(Content Discovery)을 찾아내는 OSINT 기법을 상세히 배워보겠습니다.

19장. 버그헌팅 정보수집 전략: OSINT, Reconnaissance

"적을 알고 나를 알면 백전백승"이라는 말이 있죠. 해킹도 마찬가지입니다. 무작정 공격 코드를 던지는 것은 하수나 하는 짓입니다. 고수는 공격 대상이 어떤 운영체제를 쓰는지, 어떤 도메인을 숨겨놨는지, 과거에 어떤 실수를 했는지 철저히 조사합니다.

이번 19장에서는 버그헌팅의 시작이자 승패를 가르는 가장 중요한 단계인 **'정보수집(Reconnaissance)'** 전략에 대해 다룹니다. 오픈소스 인텔리전스(OSINT)를 활용해 타겟을 발가벗기는 기술을 전수해 드리겠습니다.

---

1. 목적

- **지식 (Knowledge):** OSINT(Open Source Intelligence)의 개념과 버그헌팅에서의 **정찰(Reconnaissance)** 단계의 중요성을 이해하고, 서브도메인 열거(Subdomain Enumeration)와 콘텐츠 디스커버리(Content Discovery)의 차이를 구분한다.
- **기술 (Skills): Wappalyzer**로 기술 스택을 파악하고, **Subfinder, Dirsearch, Waybackurls, LinkFinder** 등의 도구를 활용해 숨겨진 자산과 엔드포인트를 식별하며, **Nmap**을 이용해 포트 및 서비스 버전을 스캔하는 실무 능력을 갖춘다.
- **태도 (Attitude):** 공격 전 충분한 정보 수집이 취약점 발견 확률을 높인다는 점을 인지하고, 자동화 도구에만 의존하기보다 수집된 데이터를 분석하여 새로운 공격 벡터를 찾아내려는 집요한 자세를 갖춘다.

2. 사전지식

- **DNS 구조:** 서브도메인과 도메인 네임 시스템에 대한 기본 이해.
- **HTTP 프로토콜:** URL 구조, 상태 코드(200, 403, 404 등)에 대한 이해.
- **네트워크 기초:** IP 주소와 포트(Port)의 개념.

3. 들어가기 전에

여러분, 해킹 영화를 보면 검은 화면에 타자를 치자마자 시스템이 뚫리죠? 현실은 다릅니다. 실제 해킹 과정의 80%는 '정보 수집'입니다. 타겟 시스템이 '아파치'인지 'Nginx'인지, 개발자가 실수로 남겨둔 '테스트 페이지'는 없는지 찾는 과정이 선행되어야 합니다.

오늘 배울 내용은 여러분의 시야를 넓혀줄 것입니다. 구글 검색으로는 나오지 않는 **숨겨진 서브도메인**을 찾고, **과거의 웹사이트 기록**을 뒤져 삭제된 취약점을 찾아내며, 자바스크립트 코드 속에 숨겨진 비밀키(Secret Key)를 채굴하는 법을 배웁니다. 이 과정을 소홀히 하면 결코 수준 높은 버그헌터가 될 수 없습니다.

4. 수업내용

4.1. OSINT와 Reconnaissance(정찰)의 개요

- **OSINT (Open Source Intelligence):** 뉴스, SNS, 깃허브(GitHub), 공식 문서 등 인터넷에 공개된 출처에서 정보를 수집하고 분석하는 것을 의미합니다.
- **Reconnaissance (Recon):** 군사 용어인 '정찰'에서 유래했습니다. 버그헌팅에서는 공격 전 대상 시스템의 정보를 수집하여 유효한 타겟(Attack Surface)을 최대한 많이 확보하는 단계입니다.
- **정보 수집의 두 축:**
    1. **서브도메인 열거 (Subdomain Enumeration):** 대상의 범위(Scope)가 넓을 때 자산을 식별하는 과정.
    2. **콘텐츠 디스커버리 (Content Discovery):** 식별된 자산 내에서 디렉터리, 파일, 기능 등을 탐색하는 과정.

4.2. 핵심 정보 수집 도구 및 기술

- **기술 식별 (Web Technologist):**
  - **Wappalyzer:** 웹사이트가 사용하는 기술 스택(CMS, 프레임워크, 서버 버전 등)을 분석해주는 브라우저 확장 도구입니다. 공개된 정보만 분석하므로 타겟에 부하를 주지 않습니다.
- **서브도메인 식별 (Subdomain Enumeration):**
  - **Subfinder:** 검색 엔진(Google, Bing), DNS 데이터베이스(VirusTotal, Shodan) 등 다양한 소스를 활용해 서브도메인을 찾아내는 **패시브(Passive)** 스캔 도구입니다. 대상 서버에 직접 접속하지 않아 로그를 남기지 않는 것이 특징입니다.
- **디렉터리 및 경로 탐색 (Content Discovery):**
  - **Dirsearch:** 사전 파일(Dictionary)을 대입하여 웹 서버의 숨겨진 디렉터리나 파일을 찾아내는 **액티브(Active)** 도구입니다. 상태 코드(200, 403 등)를 통해 유효성을 판단합니다.
  - **Waybackurls:** 인터넷 아카이브(Wayback Machine) 데이터를 활용하여, 과거부터 현재까지의 모든 URL 기록을 추출합니다. 지금은 사라진 엔드포인트나 파라미터를 찾는 데 매우 유용합니다.
- **자바스크립트 분석:**
  - **LinkFinder:** 복잡한 JS 파일 내에서 정규표현식을 이용해 **URL 엔드포인트**를 추출해줍니다.
  - **JSLeak:** LinkFinder 기능에 더해 소스코드 내의 민감 정보(API Key, Token, Password 등)를 탐색합니다.
- **통합 프레임워크:**
  - **Amass, BBOT, BBRF:** 위에서 언급한 다양한 기능(DNS 조회, 스크래핑, 아카이브 검색 등)을 통합하여 자동화한 도구들입니다.

4.3. 네트워크 포트 스캐닝 (Nmap 활용)

- **Nmap (Network Mapper):** 네트워크 보안 감사를 위한 필수 도구로, 열려 있는 포트와 서비스 버전을 식별합니다.
- **주요 옵션:**
  - **`-p`:** 특정 포트 스캔 (예: `-p22`, `-p-`는 전체 포트).
  - **`-sV`:** 실행 중인 서비스의 버전 정보 확인.
  - **`-O`:** 운영체제(OS) 탐지.
  - **`-sC`:** 기본 스크립트(NSE)를 사용하여 취약점이나 상세 정보를 스캔.
- **NSE (Nmap Scripting Engine):** Lua 언어로 작성된 스크립트를 통해 더욱 강력한 진단(취약점 감지, SSL 암호화 방식 확인 등)이 가능합니다.

4.4. 실전 사례: 포트 스캐닝을 통한 버그헌팅

- **사례 (Andy Gill vs Pornhub):**
  - **상황:** Pornhub 버그바운티 프로그램 참여.
  - **접근:** 서브도메인 식별 후 `stage.pornhub.com`(스테이징 서버)에 집중했습니다.
  - **발견:** Nmap으로 포트 스캔을 수행(`-sV -p- -O -T4`)한 결과, **60893번 포트**가 열려있음을 확인했고, 이것이 **Memcached** 서비스임을 식별했습니다.
  - **공격:** Memcached는 인증 없이 접근 가능한 경우가 많습니다. 간단한 명령어로 내부 데이터에 접근할 수 있음을 증명하여 **$2,500**를 포상금으로 받았습니다.

- **교훈:** 복잡한 웹 해킹 기술 없이도, **기본적인 포트 스캔과 서비스 식별**만으로도 치명적인 취약점을 찾을 수 있습니다.

5. 핵심내용 (Summary)

1. **Recon의 중요성:** 정찰은 유효한 공격 표면(Attack Surface)을 넓히는 과정이며, 버그헌팅의 성공 여부를 결정짓는 기초입니다.
2. **도구의 적절한 조합:** 패시브 스캔(Subfinder)으로 조용히 정보를 모으고, 액티브 스캔(Dirsearch, Nmap)으로 실제 구동 여부를 확인하는 등 도구를 적재적소에 활용해야 합니다.
3. **과거의 흔적: Waybackurls**와 **LinkFinder**를 통해 현재 웹사이트에서는 보이지 않는 개발자 실수나 방치된 API를 찾아내는 것이 고수의 노하우입니다.
4. **포트 스캐닝:** 웹(80/443)뿐만 아니라, **Nmap**을 통해 비표준 포트에서 실행되는 관리 도구(Memcached 등)나 취약한 서비스를 찾는 시야를 가져야 합니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 대상 서버에 직접 접속하지 않고 검색 엔진이나 공개된 데이터베이스를 활용해 서브도메인 정보를 수집하는 '패시브(Passive)' 방식의 대표적인 도구는 무엇인가요?
  -
- **Q2.** 인터넷 아카이브(Wayback Machine)에 저장된 데이터를 조회하여, 과거부터 현재까지 해당 도메인과 관련된 모든 URL 목록을 추출해주는 도구는 무엇인가요?
  -
- **Q3.** 네트워크 스캔 도구인 Nmap에서, 타겟 시스템에 기본 내장된 스크립트(NSE)를 실행하여 더 상세한 정보나 취약점을 탐지하도록 하는 옵션은 무엇인가요?
  -

7. 다음 장 소개

정보 수집을 통해 공격 대상을 확보하셨나요? 이제는 최신 기술을 활용해 이 과정을 더욱 효율화할 때입니다. 다음 시간에는 "ChatGPT를 활용한 버그헌팅"에 대해 다루겠습니다. AI가 어떻게 취약점 분석을 돕고, 보고서 작성을 자동화하며, 심지어 새로운 공격 코드를 제안하는지 혁신적인 방법을 배워보겠습니다.

20장. ChatGPT를 활용한 버그헌팅

드디어 초급 과정의 마지막 기술 챕터에 도달했습니다. 최근 IT 업계를 강타한 주제가 있다면 단연 **인공지능(AI)**, 그중에서도 **ChatGPT**일 것입니다. "AI가 해커를 대체할 수 있을까?"라는 질문을 많이 받습니다. 제 대답은 "아니오"입니다. 하지만 "**AI를 쓰는 해커가 AI를 안 쓰는 해커를 대체할 것**"입니다.

이번 20장에서는 대화형 인공지능 모델의 원리를 이해하고, 이를 버그헌팅에 어떻게 접목하여 여러분의 '보조 두뇌'로 활용할 수 있는지, 그리고 그 위험성은 무엇인지 명확히 짚어드리겠습니다.

---

1. 목적

- **지식 (Knowledge):** 대형 언어 모델(LLM)과 인간 피드백형 강화 학습(RLHF)의 개념을 이해하고, ChatGPT, Copilot, Bard 등 주요 AI 모델의 특성과 차이점을 파악한다.
- **기술 (Skills): BurpGPT**, **PentestGPT** 등 AI 기반 버그헌팅 보조 도구의 기능을 이해하고, 상황에 맞춰 **프롬프트 인젝션(Prompt Injection)** 테스트나 취약점 분석에 AI를 활용하는 방안을 모색한다.
- **태도 (Attitude):** AI가 제공하는 정보의 부정확성(Hallucination)과 데이터 유출 위협을 인지하고, 도구에 전적으로 의존하기보다 검증과 보조 수단으로 활용하는 균형 잡힌 시각을 갖춘다.

2. 사전지식

- **기본 버그헌팅 지식:** 이전 강의에서 배운 웹 취약점 및 Burp Suite 사용법.
- **IT 트렌드:** 생성형 AI에 대한 기본적인 관심과 사용 경험.

3. 들어가기 전에

여러분, 혹시 ChatGPT에게 "이 코드의 취약점을 찾아줘"라고 물어본 적 있나요? 놀랍게도 꽤 정확한 답변을 줄 때가 많습니다. 하지만 반대로 AI에게 "비밀번호를 내놔"라고 속여서 정보를 빼내는 공격도 가능합니다.

이번 시간에는 AI가 어떻게 학습(LLM, RLHF)했기에 사람처럼 말하는지 그 원리를 먼저 파헤칩니다. 그리고 **BurpGPT**로 트래픽을 분석하고, **PentestGPT**로 침투 시나리오를 짜며, **PrivateGPT**로 보안을 지키며 AI를 쓰는 법을 배웁니다. AI라는 강력한 파도에 휩쓸리지 않고, 그 파도에 올라타는 서퍼가 되어 봅시다.

4. 수업내용

4.1. 대화형 언어 모델의 이해 (LLM & RLHF)

- **LLM (Large Language Model):** 방대한 데이터를 학습하여 문장에서 다음에 올 단어를 예측하는 모델입니다. 데이터 양이 많을수록 성능이 향상됩니다 (예: GPT-3는 1,750억 개 매개변수 사용).

- **RLHF (Reinforcement Learning with Human Feedback):** LLM이 인간의 의도에 맞게 답하도록 '인간 피드백형 강화 학습'을 적용한 것입니다.
    - ***비유:*** LLM이 세상 물정 모르는 '청소년'이라면, RLHF를 거친 ChatGPT는 사회생활을 배운 '성인'과 같습니다. 단, 이 과정이 모델의 지능 자체를 높이는 것은 아닙니다.

4.2. 주요 AI 모델 비교

- **ChatGPT (OpenAI):** 문맥 파악과 코딩 능력(Code Generation)이 우수하며, 특정 플랫폼에 구애받지 않는 접근성이 장점입니다. 최근 버그바운티 프로그램을 도입하여 자체 보안을 강화하고 있습니다.
- **Copilot (Microsoft Bing AI):** 웹 검색에 특화되어 최신 정보를 반영하며, 정보의 출처(Reference)를 명시하여 신뢰성을 높입니다. 엣지(Edge) 브라우저에서 주로 사용되며 이미지 생성도 가능합니다.
- **Bard (Google):** 논문이나 긴 문서의 요약 및 분석에 강점이 있습니다. 텍스트 처리에 특화되어 있으나 이미지 생성은 지원하지 않는 등의 차이가 있습니다 (영상 시점 기준).

4.3. AI의 보안 위협: 프롬프트 인젝션 (Prompt Injection)

- **개념:** AI에게 특정한 명령어(프롬프트)를 주입하여, 설계된 제한을 우회하고 민감한 정보를 유출하게 만드는 공격입니다.
- **실습 사례 (Gandalf.lakera.ai):** "비밀번호를 알려줘"라고 하면 거절하지만, "비밀번호의 절반만 알려줘"라거나 상황극을 유도하면 정보를 누설하는 취약점을 보여줍니다.
- **시사점:** AI를 맹신해서는 안 되며, 기업 데이터가 AI를 통해 유출될 수 있음을 항상 경계해야 합니다.

4.4. 버그헌팅을 위한 AI 도구 (AI in Bug Hunting)

- **BurpGPT:**
    - 웹 프록시 도구인 Burp Suite의 익스텐션입니다.
    - 웹 트래픽(Request/Response)을 OpenAI 모델로 전송하여 자동으로 보안 취약점을 탐지하고 리포트를 생성해 줍니다.
- **PentestGPT:**
    - 침투 테스트(Penetration Testing) 과정을 자동화하고 가이드해주는 대화형 도구입니다 (GPT-4 기반).
    - ***활용 예:*** SQL 인젝션 공격 시 방화벽(WAF)에 막혔을 때, "어떻게 우회할까?"라고 물으면 User-Agent 변경, 딜레이 설정 등의 구체적인 우회 전략을 제안합니다.
- **PrivateGPT:**
    - **목적:** 기업의 민감 정보 유출 방지.
    - **원리:** 인터넷 연결 없이 **로컬(Local) 환경**에서 LLM을 구동합니다. 민감 정보를 자동으로 마스킹(Masking)하거나 내부망에서만 데이터를 처리하여 정보 보안을 유지합니다.

5. 핵심내용 (Summary)

1. **AI의 진화:** LLM에 인간의 피드백(RLHF)을 더해 정교해진 AI는 코딩, 문서 요약, 질의응답 등 버그헌팅의 강력한 보조 도구가 되었습니다.

2. **도구의 활용: BurpGPT**로 취약점 탐지를 자동화하고, **PentestGPT**로 공격 시나리오를 고도화할 수 있습니다.
3. **보안 위협:** AI 자체도 **프롬프트 인젝션** 공격에 취약할 수 있으며, 민감 데이터 입력 시 외부로 유출될 위험이 있으므로 **PrivateGPT**와 같은 로컬 솔루션이나 주의가 필요합니다.
4. **역할:** AI는 해커를 대체하는 것이 아니라, 효율성을 극대화해주는 **'비서(Assistant)'** 역할을 수행합니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 대형 언어 모델(LLM)이 인간의 의도에 부합하는 답변을 하도록, 사람이 직접 답변의 품질을 평가하고 피드백을 주어 모델을 추가 학습시키는 기법은 무엇인가요?
    -
- **Q2.** Burp Suite의 확장 프로그램으로, 웹 트래픽을 AI 모델에 전송하여 자동으로 보안 취약점을 분석하고 리포트를 생성해 주는 도구는 무엇인가요?
    -
- **Q3.** 기업 내부 데이터나 민감한 정보가 외부(OpenAI 서버 등)로 유출되는 것을 막기 위해, 인터넷 연결 없이 로컬 환경에서 LLM을 구동하여 문서를 분석할 수 있게 해주는 도구는 무엇인가요?
    -

7. 다음 장 소개

AI라는 최신 무기까지 장착하셨군요. 이제 기술적인 준비는 모두 끝났습니다. 하지만 아무리 뛰어난 해킹 실력이 있어도, 이를 기업에 제대로 전달하지 못하면 포상금을 받을 수 없습니다.

다음 시간에는 초급 과정의 마지막 단계인 "리포트 작성 전략"을 다루겠습니다. 심사관을 설득하는 '좋은 리포트'의 조건, 마크다운(Markdown) 작성법, 그리고 실제 포상을 이끌어내는 문서화 노하우를 전수해 드리겠습니다. 유종의 미를 거둘 준비 되셨나요?

21장. ChatGPT 활용 버그헌팅 및 리포트 작성 전략

드디어 초급 과정의 대미를 장식할 마지막 강의입니다. "구슬이 서 말이라도 꿰어야 보배"라는 말이 있죠. 아무리 치명적인 취약점을 찾았어도, 이를 상대방(보안 담당자)이 이해하도록 **설득**하지 못하면 포상금은 없습니다.

이번 21장에서는 **ChatGPT**를 활용해 취약점 분석과 PoC 작성을 효율화하는 최신 기법부터, 심사관의 마음을 움직이는 **리포트 작성 전략**까지, 프로 버그헌터가 되기 위한 마지막 퍼즐을 맞춰보겠습니다.

---

1. 목적

- **지식 (Knowledge):** 대화형 AI(ChatGPT)를 활용하여 코드를 분석하고 취약점(Race Condition, DoS)을 식별하는 원리와, 플랫폼별(HackerOne, Google, Naver) 리포트 양식의 차이를 이해한다.
- **기술 (Skills):** ChatGPT를 이용해 HTTP 요청을 **Python** 또는 **Fetch API** PoC 코드로 자동 생성하고, **마크다운(Markdown)** 문법을 활용하여 가독성 높은 전문가 수준의 보고서를 작성할 수 있다.
- **태도 (Attitude):** 자동화 도구의 결과를 맹신하지 않고 반드시 검증(Verify)하며, 심사관의 입장에서 명확한 **공격 시나리오**와 **임팩트**를 전달하려는 프로페셔널한 자세를 갖춘다.

2. 사전지식

- **기본 프로그래밍:** Python 및 JavaScript 코드 독해 능력
- **HTTP 프로토콜:** 요청/응답 구조 및 리다이렉션의 이해
- **버그헌팅 기초:** 이전 강의들에서 배운 취약점 개념 및 도구 사용법

3. 들어가기 전에

여러분, 해킹 기술만 좋으면 돈을 많이 벌까요? 아닙니다. '글쓰기'가 돈을 벌어줍니다. 제가 현업에서 수많은 리포트를 검토해봤지만, "여기 에러 나요"라고 쓴 리포트와 "이 에러로 인해 당신 회사의 서비스가 멈춥니다"라고 쓴 리포트의 대우는 천지 차이입니다.

오늘은 여러분의 든든한 조수, **ChatGPT**에게 코딩과 초안 작성을 맡겨 효율을 극대화하는 법을 배웁니다. 그리고 구글(Google)이 말하는 '좋은 리포트'의 조건을 분석하여, 여러분의 발견이 정당한 가치를 인정받도록 만드는 **문서화(Documentation) 기술**을 전수해 드리겠습니다. 유종의 미를 거둘 준비 되셨나요?

4. 수업내용

4.1. ChatGPT를 활용한 취약점 분석 실습

- **사례 1: Hugging Face Transformers (CVE-2023-2800)**
  - **분석:** `tempfile.mktemp` 함수 사용 코드를 ChatGPT에 입력하여 분석 요청.
  - **결과:** ChatGPT는 해당 함수가 파일 이름을 생성하지만 파일을 생성하지는 않아, 그 사이에 공격자가 파일을 가로챌 수 있는 **레이스 컨디션(Race Condition)** 취약점이 있음을 정확히 지적함.
  - **해결:** `mkstemp` 함수 사용을 권고함.
- **사례 2: Draw.io Proxy (DoS 취약점)**
  - **분석:** 리다이렉션 처리 로직 코드를 ChatGPT에 입력.
  - **결과:** 리다이렉션된 URL이 현재 호스트와 동일한지 검증하는 로직이 없어, 무한 루프를 통한 **서비스 거부(DoS)** 공격이 가능함을 식별함.

4.2. ChatGPT를 활용한 PoC(개념 증명) 코드 생성

- **Python PoC 생성:**
  - Burp Suite의 HTTP 요청 패킷을 그대로 ChatGPT에 입력하고 "Python PoC 코드를 짜줘"라고 요청.
  - 헤더, 쿠키, 파라미터가 포함된 완성된 `requests` 코드를 즉시 생성해주며, 이를 통해 제보 시간을 단축할 수 있음.
- **Fetch API PoC 생성 (꿀팁):**
  - 브라우저 개발자 도구 콘솔에서 바로 실행 가능한 **Fetch API** 코드로 변환 요청.
  - 별도의 도구 없이 브라우저만으로 공격을 재현할 수 있어 매우 유용함.

4.3. 버그헌팅 플랫폼별 리포트 포맷 분석

- **HackerOne:** Asset(대상), Weakness(취약점 유형), Severity(심각도), PoC(재현 방법) 5단계 구성.
- **Google:** Description, Location, The Cause, Public 여부, Technical Problem(기술적 세부사항), Impact(공격자 이득) 등으로 구성. 특히 공격 시나리오를 중시함.
- **Naver:** 제목, 환경(OS), 서비스, 도메인, 취약점 종류, 설명, 조치 방안 등으로 구성. 조치 방안 제시는 선택 사항임.

4.4. 좋은 리포트 작성을 위한 전략 (Google 가이드 기반)

- **실수 방지 (Avoiding Mistakes):**
  - **범위(Scope) 확인:** 인수 합병 직후의 '블랙아웃' 기간이나 제외 대상을 반드시 확인해야 함.
  - **타겟 식별:** 로고만 보고 타겟이라 착각하지 말고, WHOIS나 DNS 조회로 자산 소유권을 검증해야 함.
  - **자동화 도구 검증:** 자동화 도구의 결과(SQLi, SSL 등)를 맹신하지 말고, **직접 익스플로잇**이 가능함을 증명해야 함.
  - **동영상 제보:** 꼭 필요한 경우가 아니면 지양하되, 첨부할 경우 해상도를 높이고 과정을 짧게 편집해야 함.
- **공격 시나리오 (Attack Scenario):**
  - **3요소:** 공격자의 의도, 얻을 수 있는 이익, 공격 방식을 명확히 기술.
  - **팁:** 공격자의 시작 위치(권한), 피해자에 대한 가정, 전제 조건과 파급력을 고려하여 단계별로 작성.

4.5. 마크다운(Markdown) 활용법

- **개요:** 텍스트 기반의 경량 마크업 언어로, 대부분의 플랫폼에서 표준으로 사용.
- **핵심 문법:**
  - **헤더:** # 개수로 제목 크기 조절 (구조화).
  - **강조:** *이탤릭*, 볼드체 (중요 키워드 강조).
  - **코드 블록:** 백틱 3개(```)로 코드를 감싸 가독성 확보.
  - **인용:** > 를 사용하여 참고 자료나 이전 리포트 인용.
  - **테이블:** | 와 - 를 사용하여 정보를 구조화.

5. 핵심내용 (Summary)

1. **AI는 최고의 조수:** ChatGPT는 코드의 논리적 결함(Race Condition, DoS 등)을 찾거나, HTTP 요청을 PoC 코드(Python, Fetch API)로 변환하는 데 탁월한 성능을 보여줍니다.
2. **검증의 중요성:** 자동화 도구나 AI가 찾아준 결과는 반드시 수동으로 검증(Verify)해야 하며, 실제 공격이 가능한 **시나리오**를 만들어야 유효한 리포트로 인정받습니다.
3. **리포트의 품격:** 좋은 리포트는 **명확한 타겟 식별**, **따라 하기 쉬운 재현 단계(Step-by-step)**, 그리고 **마크다운**을 활용한 깔끔한 포맷팅을 갖춰야 합니다.
4. **임팩트 전달:** 단순한 버그 설명이 아니라, "이 취약점으로 공격자가 무엇을 얻을 수 있는가(Impact)"를 설득하는 것이 포상금 액수를 결정짓습니다.

6. 마무리 키워드 퀴즈

강의 내용을 점검해 봅시다.

- **Q1.** 브라우저 개발자 도구(Console)에서 바로 실행하여 네트워크 요청을 보낼 수 있는 JavaScript API로, ChatGPT를 통해 패킷을 이 코드로 변환하면 PoC로 활용하기 좋은 것은 무엇인가요?
  -

- **Q2.** 버그헌팅 리포트 작성 시 텍스트를 굵게()하거나 코드를 블록(```)으로 감싸는 등 서식을 지정하기 위해 사용하는 경량 마크업 언어는 무엇인가요?
    -
- **Q3.** 구글의 리포트 작성 가이드에서 강조하는 것으로, 공격자의 의도, 이익, 방식을 고려하여 작성해야 하며, 취약점의 파급력을 증명하는 핵심 요소는 무엇인가요?
    -

7. 다음 장 소개

이것으로 버그헌팅 초급 과정의 모든 강의를 마쳤습니다. 1강부터 21강까지 긴 여정을 함께해주신 여러분, 고생 많으셨습니다.

이제 여러분은 취약점의 원리를 이해하고, 도구를 다루며, AI를 활용해 보고서까지 쓸 수 있는 **준비된 버그헌터**입니다. 이제 남은 것은 실전뿐입니다. 배운 내용을 바탕으로 실제 플랫폼에 도전하여 첫 포상금의 기쁨을 누리시길 바랍니다. 언제나 윤리적인 자세를 잊지 마십시오. 감사합니다.

마무리 키워드 퀴즈 정답표

1장부터 21장까지의 키워드 퀴즈를 정리해 드립니다.

대제목 (Chapter)	퀴즈 문제 (Question)	정답 (Answer)
1장. 버그헌팅 입문	Q1. 기업이 자사 서비스의 보안 취약점을 찾은 해커에게 포상금을 지급하는 제도를 뜻하는 용어는 무엇인가요?	버그바운티 (Bug Bounty)
	Q2. 취약점의 위험도를 평가할 때, 시스템을 장악하거나 계정 탈취할 수 있는 가장 높은 수준의 위험 등급은 무엇인가요?	Critical (크리티컬)
	Q3. 버그헌팅을 시작하기 전, 법적인 보호를 받고 불법 행위가 되지 않기 위해 가장 먼저 확인해야 할 '허용된 대상 및 규칙'을 무엇이라고 합니까?	범위 (Scope)
2장. 버그헌팅의 역사	Q1. 1995년, 웹 브라우저의 안정성을 위해 역사상 최초로 버그바운티 프로그램을 시작한 기업의 이름은 무엇인가요?	넷스케이프 (Netscape)
	Q2. 버그바운티 프로그램 중, 누구나 참여할 수 있어 잠재적으로 가장 많은 취약점을 발견할 수 있지만, 저품질 보고서가 접수될 확률도 높은 유형은 무엇인가요?	퍼블릭 (Public) 프로그램

	Q3. 최근 버그헌팅 트렌드 중 하나로, ChatGPT나 Copilot 등을 활용하여 취약점 탐색과 분석을 고도화하는 기술적 흐름은 무엇의 발전인가요?	인공지능(AI) 기반 도구의 발전
3장. VDP와 플랫폼	Q1. 버그헌터가 법적 처벌을 받지 않고 안전하게 취약점을 제보할 수 있도록, 기업이 제보자에게 면책권을 보장하는 VDP의 핵심 조항을 무엇이라고 합니까?	세이프 하버 (Safe Harbor)
	Q2. iOS나 안드로이드 등의 고난이도 '제로데이(Zero-day)' 취약점을 주로 고가에 매입하는 프리미엄 버그바운티 플랫폼의 이름은?	제로디움 (Zerodium)
	Q3. 국내 공공기관인 KISA에서 운영하며, 신고된 취약점에 대해 CVE 또는 KVE 번호를 부여할 권한을 가진 제도는 무엇입니까?	보안 취약점 신고포상제
4장. 윤리적 해킹	Q1. 윤리적 해커가 지켜야 할 6가지 원칙 중, 시스템 소유자로부터 명백한 허가를 받은 후에만 활동해야 한다는 원칙은 무엇인가요?	사전 동의 (Prior Consent)
	Q2. 악의적인 의도는 없지만, 허가받지 않은 시스템을 해킹하거나 규정을 위반하여 활동하는 해커를 모자 색깔에 비유하여 무엇이라고 합니까?	그레이 햇 (Grey Hat)
	Q3. 국내법상 정당한 접근 권한 없이 정보통신망에 침입하거나, 허용된 범위를 넘어 침입하는 행위를 금지하고 있는 법률의 명칭은 무엇인가요?	정보통신망법
5장. 버그헌팅의 이해	Q1. 취약점의 심각도를 평가하는 국제 표준 점수 체계로, 공격 벡터, 복잡도, 필요한 권한 등의 요소를 통해 0~10점 사이의 점수를 매기는 시스템은 무엇인가요?	CVSS

	Q2. CVSS 평가 항목 중, 공격자가 취약점을 악용하기 위해 피해자(사용자)의 특정 행동(클릭 등)이 필요한지 여부를 나타내는 지표는 무엇인가요?	사용자 상호작용 (User Interaction, UI)
	Q3. 정해진 기간과 인력으로 수행하는 '모의해킹'과 달리, 불특정 다수의 해커가 참여하여 성과(취약점 발견)에 따라 보상을 받는 방식은 무엇인가요?	버그바운티 (Bug Bounty)
6장. 웹 기본 이해 (1)	Q1. 웹 문서의 '동작'을 정의하며, 클라이언트 사이드 취약점 분석을 위해 필수적으로 알아야 하는 언어는 무엇인가요?	자바스크립트 (JavaScript)
	Q2. HTTP 메시지에서 헤더(Header)와 바디(Body)를 구분하거나, 헤더의 각 줄을 구분하기 위해 사용되는 특수 문자는 무엇인가요?	CRLF (\r\n)
	Q3. 도메인 이름을 IP 주소로 연결할 때, IP 주소가 아닌 '다른 도메인 이름(별칭)'으로 연결하는 DNS 레코드 방식은 무엇인가요?	CNAME (Canonical Name)
7장. 웹 기본 이해 (2)	Q1. HTTP 응답 헤더 중 하나로, 자바스크립트를 통한 쿠키 접근을 차단하여 XSS 공격으로부터 세션 ID를 보호하는 보안 옵션은 무엇인가요?	HttpOnly
	Q2. 서버에 데이터를 전송하여 상태를 변경할 때 주로 사용되며, 요청 본문(Body)에 데이터를 담아 보내고 실행 결과가 매번 달라질 수 있는(비멱등성) HTTP 메서드는 무엇인가요?	POST
	Q3. 클라이언트의 브라우저 및 운영체제 정보를 담고 있는 헤더로, 버그헌팅 시 이를 조작하여 모바일 환경으로 위장하거나 접근 제한을 우회할 때 사용하는 헤더는 무엇인가요?	User-Agent

8장. 웹 기본 이해 (3)	Q1. 클라이언트의 요청에는 문제가 없으나, 서버 내부의 로직 에러나 DB 오류 등으로 인해 요청을 처리할 수 없을 때 반환되는 HTTP 상태 코드 대역은 무엇인가요?	500번대
	Q2. 웹 프록시 도구인 Burp Suite에서, 캡처한 요청 패킷을 하나씩 수동으로 변조하여 서버로 재전송하고 응답을 분석하는 데 특화된 탭(기능)의 이름은 무엇인가요?	리피터 (Repeater)
	Q3. 클라이언트가 요청한 리소스가 서버에 존재하지 않을 때 발생하는 에러 코드로, 해커가 이를 통해 서버의 디렉터리 구조나 URL 패턴을 유추하는 데 활용하는 코드는 무엇인가요?	404 (Not Found)
9장. 웹 기본 이해 (4)	Q1. 클라이언트의 상태 정보를 서버에 저장하지 않고, 토큰 자체에 정보를 담아 서명하여 검증하는 'Stateless'한 인증 방식의 토큰 표준은 무엇인가요?	JWT (JSON Web Token)
	Q2. 사용자가 비밀번호를 제공하지 않고도, 구글이나 페이스북 같은 서비스의 자원 접근 권한을 다른 애플리케이션에 위임할 수 있게 해주는 표준 프로토콜은 무엇인가요?	OAuth 2.0
	Q3. 입력 값 검증 방식 중, 허용할 안전한 리스트(예: 숫자만 허용)를 미리 정의하고 그 외의 모든 입력을 차단하는 보안성이 높은 방식은 무엇인가요?	화이트리스트 (Whitelist)
10장. 웹 취약점 이해	Q1. 게시판 등에 악성 스크립트를 작성하여 서버 DB에 저장하고, 해당 글을 열람하는 다수의 사용자에게 스크립트가 실행되게 만드는 XSS 유형은 무엇인가요?	Stored XSS (저장형 XSS)
	Q2. 공격자가 웹 애플리케이션의 로직을 악용하여, 서버가 방화벽 내부의 시스템이나 클라우드 메타데이터 서비스 등 공격자가 직접 접근할 수 없는 곳으로 요청을 보내도록 만드는 취약점은 무엇인가요?	SSRF

	Q3. 사용자가 입력한 파라미터(예: user_id=1234)를 서버가 적절한 검증 없이 내부 객체의 키값으로 사용하여, 타인의 정보에 접근할 수 있게 되는 취약점을 무엇이라고 합니까?	IDOR
11장. 시나리오 & 체이닝	Q1. 단일 취약점으로는 위협이 낮지만, 두 개 이상의 보안 이슈를 연결하여 강력한 공격 시나리오를 만드는 기법을 무엇이라고 합니까?	취약점 체이닝
	Q2. 사용자가 스스로 브라우저 콘솔에 악성 코드를 입력하게 유도해야 실행되는 XSS 유형으로, 단독으로는 포상 대상에서 제외되는 경우가 많은 취약점은?	셀프 XSS (Self-XSS)
	Q3. XSS 공격 시 자바스크립트로 세션 쿠키(document.cookie)에 접근하지 못하도록 막는 보안 설정 옵션은 무엇입니까? (체이닝을 통해 이를 우회하는 시나리오가 필요함)	HttpOnly
12장. 버그헌팅 실제 사례	Q1. 유튜브 비공개 영상 탈취 사례에서, 해커가 영상을 프레임 단위로 가져오기 위해 공략했던 연동 서비스의 기능은 무엇인가요?	마크 모먼트 (Mark Moment)
	Q2. 소니(Sony) 해킹 사례에서, 사용자의 입력값(ID/PW) 필터링을 우회하기 위해 공격자가 SQL 인젝션 코드를 삽입한 HTTP 헤더의 이름은 무엇인가요?	User-Agent
	Q3. 소니 사례에서 초기 정찰(Recon) 시 Sublist3r로는 찾지 못한 숨겨진 서브도메인을 발견하게 해 준 오픈소스 정보 수집 도구의 이름은 무엇인가요?	Amass

13장. 심화 사례 분석	Q1. Sony 해킹 사례에서, MSSQL 데이터베이스 환경에서 시스템 운영체제 명령어를 실행하기 위해 공격자가 활성화하고 사용한 저장 프로시저의 이름은 무엇인가요?	xp_cmdshell
	Q2. GitLab 사례에서 공격자는 Path Traversal 취약점을 이용하여 자신의 SSH 공개키를 서버의 특정 파일에 덮어씌웠습니다. SSH 접속 시 인증된 키 목록을 담고 있는 이 파일의 이름은 무엇인가요?	authorized_keys
	Q3. Dropbox SSRF 사례에서, 공격자가 내부망 IP 직접 호출이 차단되자 이를 우회하기 위해 자신의 서버에서 사용한 HTTP 응답 헤더 기술은 무엇인가요?	리다이렉션 (Location 헤더)
14장. 자동화 도구 실습	Q1. SQL Injection 취약점을 자동으로 탐지하고 데이터를 추출하는 도구로, --os-shell 옵션을 통해 시스템 쉘 획득까지 지원하는 도구는 무엇인가요?	SQLMap
	Q2. JWT(JSON Web Token)의 무결성을 검증하는 서명(Signature) 부분을 생성할 때 사용되는 알고리즘으로, 대칭키 방식을 사용하는 대표적인 알고리즘은 무엇인가요?	HS256
	Q3. 획득한 해시값을 바탕으로 원본 패스워드를 복구하는 도구로, 사전 공격, 마스크 공격 등 다양한 모드를 지원하는 대표적인 해시 크래킹 툴은 무엇인가요?	Hashcat
15장. CVE와 메타스플로잇	Q1. 컴퓨터 보안 취약점에 고유한 식별 코드를 부여하여, 전 세계적으로 취약점 정보를 공유하고 추적하기 위해 만든 국제 표준 식별자는 무엇인가요?	CVE

	Q2. 메타스플로잇(Metasploit) 프레임워크에서 실제 취약점을 공격하는 코드는 'Exploit' 모듈이고, 공격 성공 후 타겟 시스템에서 실행되어 공격자에게 쉘을 연결해주는 등의 행위를 하는 코드를 무엇이라 하나요?	페이로드 (Payload)
	Q3. Exploit-DB에 등록된 취약점 정보를 리눅스 터미널에서 오프라인으로 빠르게 검색할 수 있게 해주는 칼리 리눅스 내장 도구 명령어는 무엇인가요?	searchsploit
16장. CVE 버그헌팅 사례	Q1. 자바 기반 웹 애플리케이션(JSF)에서 사용되는 스크립트 언어로, #{...}와 같은 형식을 가지며 취약하게 구현될 경우 RCE로 이어질 수 있는 것은 무엇인가요?	EL (Expression Language)
	Q2. 메타스플로잇 실행 중에 사용자가 직접 추가한 외부 모듈(.rb)을 인식시키거나, 수정된 코드를 반영하기 위해 콘솔에서 입력해야 하는 명령어는 무엇인가요?	reload_all
	Q3. CVE-2017-1000486 취약점의 근본 원인 중 하나로, PrimeFaces 프레임워크가 암호화 과정에서 사용한 '이것'이 오픈소스에 공개되어 있어 해커가 암호문을 위조할 수 있었습니다. '이것'은 무엇인가요?	기본 비밀번호 (Default Password)
17장. 최신 CVE 분석	Q1. Apache Superset 취약점 사례에서, 공격자가 관리자 세션을 위조하기 위해 악용한 것으로, Flask 프레임워크가 쿠키 변조를 막기 위해 사용하는 핵심 데이터는 무엇인가요?	시크릿 키 (Secret Key)
	Q2. Flask 세션 쿠키와 같이 서명된 데이터를 크랙하거나 위조할 때 사용하는 대표적인 커맨드 라인 도구의 이름은 무엇인가요?	flask-unsign
	Q3. Azure Service Fabric 사례처럼, XSS와 같은 낮은 위험도의 취약점을 다른 기능이나 취약점(CSRF 등)과 연결하여 전체 시스템 장악과 같은 높은 위험도로 발전시키는 기법을 무엇이라고 합니까?	취약점 체이닝

18장. CVE 심화 분석	Q1. VMware 취약점 사례에서 사용된 Java 기반 템플릿 엔진으로, 보안 설정 미흡 시 SSTI를 통해 원격 코드 실행(RCE)이 가능한 기술은 무엇인가요?	프리마커 (FreeMarker)
	Q2. ImageMagick 취약점(CVE-2022-44268)에서 공격자가 PNG 파일의 텍스트 청크(tEXt chunk)에 삽입하여, 서버가 해당 경로의 파일을 읽도록 유도한 특정 키워드는 무엇인가요?	profile
	Q3. ImageMagick 공격 성공 후, 다운로드한 이미지에 포함된 유출 데이터를 확인하기 위해 사용하는 명령어로, 이미지의 상세 속성(Raw profile type 등)을 출력해주는 도구 명령은 무엇인가요?	identify
19장. 정보수집 전략	Q1. 대상 서버에 직접 접속하지 않고 검색 엔진이나 공개된 데이터베이스를 활용해 서브도메인 정보를 수집하는 '패시브(Passive)' 방식의 대표적인 도구는 무엇인가요?	Subfinder
	Q2. 인터넷 아카이브(Wayback Machine)에 저장된 데이터를 조회하여, 과거부터 현재까지 해당 도메인과 관련된 모든 URL 목록을 추출해주는 도구는 무엇인가요?	waybackurls
	Q3. 네트워크 스캔 도구인 Nmap에서, 타겟 시스템에 기본 내장된 스크립트(NSE)를 실행하여 더 상세한 정보나 취약점을 탐지하도록 하는 옵션은 무엇인가요?	-sC
20장. ChatGPT와 버그헌팅	Q1. 대형 언어 모델(LLM)이 인간의 의도에 부합하는 답변을 하도록, 사람이 직접 답변의 품질을 평가하고 피드백을 주어 모델을 추가 학습시키는 기법은 무엇인가요?	RLHF

	Q2. Burp Suite의 확장 프로그램으로, 웹 트래픽을 AI 모델에 전송하여 자동으로 보안 취약점을 분석하고 리포트를 생성해 주는 도구는 무엇인가요?	BurpGPT
	Q3. 기업 내부 데이터나 민감한 정보가 외부(OpenAI 서버 등)로 유출되는 것을 막기 위해, 인터넷 연결 없이 로컬 환경에서 LLM을 구동하여 문서를 분석할 수 있게 해주는 도구는 무엇인가요?	PrivateGPT
21장. 리포트 작성 전략	Q1. 브라우저 개발자 도구(Console)에서 바로 실행하여 네트워크 요청을 보낼 수 있는 JavaScript API로, ChatGPT를 통해 패킷을 이 코드로 변환하면 PoC로 활용하기 좋은 것은 무엇인가요?	Fetch API
	Q2. 버그헌팅 리포트 작성 시 텍스트를 굵게()하거나 코드를 블록(```)으로 감싸는 등 서식을 지정하기 위해 사용하는 경량 마크업 언어는 무엇인가요?	마크다운 (Markdown)
	Q3. 구글의 리포트 작성 가이드에서 강조하는 것으로, 공격자의 의도, 이익, 방식을 고려하여 작성해야 하며, 취약점의 파급력을 증명하는 핵심 요소는 무엇인가요?	공격 시나리오 (Attack Scenario)

[버그헌팅 초급 과정 필수 용어집(Glossary]

1. 버그헌팅 기초 및 일반 (General Concepts)

- **버그헌팅 (Bug Hunting):** 소프트웨어나 서비스의 취약점(버그)을 찾아내는 일련의 활동입니다.
- **버그바운티 (Bug Bounty):** 기업이 자사 서비스의 보안 취약점을 찾아낸 해커에게 포상금(Bounty)을 지급하는 제도입니다.
- **VDP (Vulnerability Disclosure Program):** 취약점 공개 정책 프로그램으로, 포상금은 지급하지 않지만 법적 보호 하에 취약점을 제보할 수 있는 채널을 의미합니다.
- **화이트 해커 (White Hat):** 악의적인 의도 없이, 시스템의 보안을 개선하기 위해 취약점을 식별하고 조치 방안을 제시하는 윤리적 해커입니다.
- **CVSS (Common Vulnerability Scoring System):** 취약점의 심각도를 평가하는 국제 표준 점수 시스템입니다. 공격 복잡도, 영향 범위 등을 고려하여 점수(0~10점)를 매깁니다.
- **CVE (Common Vulnerabilities and Exposures):** 공개적으로 알려진 컴퓨터 보안 취약점에 부여하는 고유 식별 번호입니다. (예: CVE-2021-44228).
- **CNA (CVE Numbering Authority):** CVE 번호를 부여할 권한을 가진 기관(MITRE, KISA 등)이나 기업을 말합니다.
- **제로데이 (Zero-day):** 아직 개발사나 대중에게 알려지지 않았거나, 패치가 나오지 않은 보안 취약점을 의미합니다.

2. 웹 기술 기초 (Web Fundamentals)

- **HTTP/HTTPS:** 웹 서버와 클라이언트가 데이터를 주고받기 위한 통신 규약(프로토콜)입니다. HTTPS는 이를 암호화하여 보안을 강화한 버전입니다.
- **쿠키 (Cookie):** 사용자의 브라우저에 저장되는 작은 데이터 조각으로, 사용자 식별이나 로그인 상태 유지 등에 사용됩니다.
- **세션 (Session):** 서버 측에 저장되는 사용자 정보입니다. 클라이언트는 세션 ID만 가지고 서버에 저장된 자신의 정보에 접근합니다.
- **JWT (JSON Web Token):** JSON 객체를 사용하여 정보를 안전하게 전달하기 위한 토큰 표준입니다. 주로 인증(Authentication) 및 인가(Authorization)에 사용되며, 서명(Signature)을 통해 무결성을 보장합니다.
- **OAuth:** 사용자가 비밀번호를 제공하지 않고도, 다른 웹사이트나 앱의 접근 권한을 부여할 수 있게 하는 개방형 표준 인증 프로토콜입니다 (예: 구글로 로그인).
- **인증 (Authentication):** "당신이 누구인가?"를 확인하는 절차입니다 (로그인 등).
- **인가 (Authorization):** "당신이 무엇을 할 수 있는가?"를 확인하는 절차입니다 (권한 부여).

3. 주요 웹 취약점 (Vulnerabilities)

- **SQL 인젝션 (SQL Injection):** 공격자가 웹 애플리케이션의 입력값에 악의적인 SQL 구문을 삽입하여 데이터베이스를 조작하거나 정보를 탈취하는 공격입니다.
- **XSS (Cross-Site Scripting):** 웹 페이지에 악성 스크립트를 삽입하여, 이를 열람하는 사용자의 브라우저에서 스크립트가 실행되게 만드는 공격입니다. 세션 탈취 등에 주로 사용됩니다.

    - **Reflected XSS:** 악성 스크립트가 URL 파라미터 등을 통해 서버로 전송되었다가 즉시 응답 페이지에 반사되어 실행되는 유형입니다.
    - **Stored XSS:** 악성 스크립트가 게시판 댓글 등 서버 DB에 저장되어, 해당 페이지를 방문하는 모든 사용자에게 실행되는 유형입니다.
- **CSRF (Cross-Site Request Forgery):** 사용자의 의지와 무관하게, 공격자가 의도한 행위(비밀번호 변경, 결제 등)를 사용자의 권한으로 서버에 요청하게 만드는 공격입니다.
- **IDOR (Insecure Direct Object References):** 부적절한 인증 인가로 인해, URL의 파라미터 값(예: id=123)을 변조하여 타인의 정보에 접근하는 취약점입니다.
- **LFI (Local File Inclusion):** 서버 내부의 로컬 파일(예: `/etc/passwd`)을 페이지에 포함시켜 읽어오거나 실행시키는 취약점입니다.
- **RCE (Remote Code Execution):** 원격 코드 실행 취약점으로, 해커가 원격에서 서버의 시스템 명령어를 실행하여 서버를 장악할 수 있는 치명적인 공격입니다.
- **SSRF (Server-Side Request Forgery):** 공격자가 서버를 조종하여 서버가 내부망이나 외부의 다른 시스템으로 요청을 보내게 만드는 공격입니다. 클라우드 환경(AWS 메타데이터 등)에서 특히 위험합니다.
- **SSTI (Server-Side Template Injection):** 템플릿 엔진(Freemarker 등) 사용 시 사용자 입력값을 적절히 처리하지 않아, 서버 측에서 임의의 코드가 실행되는 취약점입니다.

4. 해킹 도구 및 기법 (Tools & Techniques)

- **버프 스위트 (Burp Suite):** 클라이언트와 서버 사이의 통신(패킷)을 가로채서 분석하고 변조할 수 있는 프록시(Proxy) 도구입니다. 리피터(Repeater), 인트루더(Intruder) 등의 기능이 있습니다.
- **메타스플로잇 (Metasploit):** 취약점 진단 및 침투 테스트를 위한 방대한 프레임워크입니다. 다양한 익스플로잇 모듈과 페이로드를 제공합니다.
- **SQLMap:** SQL 인젝션 취약점을 자동으로 탐지하고 데이터베이스를 탈취하는 과정을 자동화한 도구입니다.
- **Hashcat:** 해시(Hash)된 비밀번호 등을 크래킹(복구)하기 위해 사용하는 고성능 도구입니다.
- **OSINT (Open Source Intelligence):** 인터넷에 공개된 정보(뉴스, SNS, 깃허브 등)를 수집하여 대상을 분석하는 첩보 활동입니다.
- **레콘 (Reconnaissance):** 공격 전 대상을 정찰하는 단계로, 서브도메인 수집이나 포트 스캔 등을 포함합니다.
- **익스플로잇 (Exploit):** 취약점을 이용하여 공격자가 의도한 동작(제어 권한 획득 등)을 수행하도록 만든 코드나 절차를 말합니다.
- **페이로드 (Payload):** 익스플로잇 성공 후, 타겟 시스템에서 실제로 실행되어 공격자에게 쉘을 연결해주거나 특정 행위를 하는 코드 부분입니다.
- **체이닝 (Chaining):** 여러 개의 낮은 위험도 취약점을 연결하여, 더 높은 파급력을 가진 공격 시나리오(예: XSS + CSRF = 계정 탈취)를 만드는 기법입니다.
- **PoC (Proof of Concept):** 개념 증명 코드로, 발견한 취약점이 실제로 작동함을 증명하기 위해 작성하는 간단한 코드나 스크립트입니다